C #

CHEAT SHEETS

A Complete Reference for Modern C# and .NET

Syntax & Types · Strings · Pattern Matching · Collections · OOP & Records · Generics · async / await · LINQ · Files & JSON · Testing · Dependency Injection · EF Core

130 REFERENCE SHEETS

CONTENTS

FOUNDATIONS

001–007 Introduction & Setup
008–010 Output, Input & Comments
011–020 Variables & Data Types
021–026 Strings
027–033 Operators
034–043 Control Flow

CODE & TYPES

044–048 Error Handling
049–060 Methods & Functions
061–074 Data Structures & Collections
075–090 Object-Oriented Programming
091–097 Files & I/O

PLATFORM & ADVANCED

098–104 Namespaces, Packages & the Base Class Library
105–109 External Libraries
110–119 Advanced Topics
120–125 LINQ
126–128 Entity Framework Core
129–130 Quick Reference

C#

Table of Contents

130 sheets · syntax · examples · notes

INTRODUCTION & SETUP 001 – 007

001 Introduction & History
002 Installing the .NET SDK
003 The dotnet CLI
004 Projects & Solutions
005 Program Structure
006 File-Based Apps
007 Compilation & the Ru…

OUTPUT, INPUT & COMMENTS 008 – 010

008 Console Output
009 Console Input
010 Comments & XML Do…

VARIABLES & DATA TYPES 011 – 020

011 Variables & Constants
012 Integer Types
013 Floating-Point & Deci…
014 bool & char
015 Value vs Reference Ty…
016 Nullable Value Types
017 Nullable Reference Ty…
018 Type Conversion & C…
019 Type Checking & Insp…
020 Enums

STRINGS 021 – 026

021 String Basics
022 Interpolation & Forma…
023 String Methods — Se…
024 String Methods — Tra…
025 Indexing, Ranges & S…
026 StringBuilder & Comp…

OPERATORS 027 – 033

027 Arithmetic Operators
028 Comparison & Equalit…
029 Logical Operators
030 Bitwise & Shift Opera…
031 Assignment & Comp…
032 Null Operators
033 Operator Overloading

CONTROL FLOW 034 – 043

034 if / else if / else
035 Ternary & Conditional…
036 switch Statement
037 switch Expressions
038 Pattern Matching — B…
039 Pattern Matching — A…
040 for Loops
041 foreach Loops
042 while & do-while
043 Loop Control & Jumps

ERROR HANDLING 044 – 048

044 Exception Basics
045 Throwing Exceptions
046 Exception Types & Fil…
047 Custom Exceptions
048 using & IDisposable

METHODS & FUNCTIONS 049 – 060

049 Defining & Calling Me…
050 Parameters & Argum…
051 params & Variadic M…
052 Method Overloading
053 Local Functions & Sc…
054 Delegates
055 Lambda Expressions
056 Closures & Captured …
057 Events
058 Recursion
059 Iterators & yield
060 Extension Methods &…

DATA STRUCTURES & COLLECTIONS 061 – 074

061 Arrays — Basics
062 Arrays — Methods
063 Collection Expressions
064 List<T> — Basics
065 List<T> — Methods
066 Dictionary — Basics
067 Dictionary — Advanc…
068 Sets
069 Stacks, Queues & Lin…
070 Tuples & Deconstruct…
071 Immutable & Frozen …
072 Concurrent Collectio…
073 Collection Interfaces
074 Sorting & Searching

OBJECT-ORIENTED PROGRAMMING 075 – 090

075 Classes & Objects
076 Constructors
077 Access Modifiers
078 Properties
079 Static Members & Cla…
080 Indexers & Partial Typ…
081 Inheritance
082 Polymorphism & Cast…
083 Abstract Classes
084 Interfaces
085 Structs
086 Records
087 Equality & Object Me…
088 Generics — Basics
089 Generic Constraints
090 Covariance & Contra…

FILES & I/O 091 – 097

091 Reading Files
092 Writing & Appending …
093 Paths & Directories
094 JSON — Serialization
095 JSON — DOM & Read…
096 XML
097 Streams & Binary Files

NAMESPACES, PACKAGES & THE BASE CLASS LIBRARY 098 – 104

098 Namespaces & using
099 NuGet Packages
100 Math & Random
101 Dates & Times
102 Environment & Proce…
103 Configuration
104 Generic Host & Depe…

EXTERNAL LIBRARIES 105 – 109

105 xUnit
106 NUnit & MSTest
107 Mocking & Assertion …
108 Logging
109 HttpClient

ADVANCED TOPICS 110 – 119

110 Regular Expressions
111 async / await — Basics
112 async / await — Patte…
113 async / await — Pitfalls
114 Threads & Synchroniz…
115 Parallelism & Channels
116 Span, Memory & Perf…
117 Garbage Collection &…
118 Reflection & Attributes
119 Union Types & Close…

LINQ 120 – 125

120 LINQ — Basics
121 LINQ — Filtering & Pr…
122 LINQ — Ordering & Pa…
123 LINQ — Aggregation
124 LINQ — Grouping & J…
125 LINQ — Elements, Qu…

ENTITY FRAMEWORK CORE 126 – 128

126 EF Core — DbContext…
127 EF Core — Modeling …
128 EF Core — Querying …

QUICK REFERENCE 129 – 130

129 Quick Reference — La…
130 Quick Reference — C…

Sheet 001

Introduction & History

Anders Hejlsberg · .NET Framework vs .NET · CLR · managed code · C# version timeline · LTS vs STS releases

WHAT C# IS

C# is a statically typed, object-oriented language from Microsoft, designed by **Anders Hejlsberg** and released in 2002 with .NET Framework 1.0. Code compiles to **IL** (intermediate language) that the **CLR** runs as **managed code** with garbage collection. Today it runs cross-platform on modern **.NET** (Windows, macOS, Linux) for web APIs, desktop apps, games (Unity), cloud services and tools.

KEY FACTS

- File extension: `.cs` ; project file: `.csproj`
- Current: **C# 14 / .NET 10** (LTS, Nov 2025)
- Next: C# 15 / .NET 11 ships Nov 2026 (in preview now)
- A new major .NET ships every November
- Open source: Roslyn compiler and runtime on GitHub

HELLO, WORLD (C# 9+ TOP-LEVEL STATEMENTS)

```
// Program.cs — the whole program
Console.WriteLine("Hello, World!");   // Hello, World!

// Run it:
//   dotnet new console -o Hello
//   cd Hello
//   dotnet run                        // Hello, World!

// Or as a single file (.NET 10):
//   dotnet run hello.cs
```

CODE EXPLAINED

- → `Console.WriteLine` — writes a line to standard output
- → Top-level statements — no `class` or `Main` needed in one file
- → `dotnet new console` — creates a project from a template
- → `dotnet run` — builds, then runs the project
- → `dotnet run hello.cs` — file-based app, no `.csproj` needed

VERSION TIMELINE

VERSION	YEAR	HEADLINE FEATURES
`C# 1–2`	2002–05	classes, generics, nullable value types, iterators
`C# 3`	2007	LINQ, lambdas, var, extension methods
`C# 5`	2012	async / await
`C# 7–8`	2017–19	tuples, pattern matching, nullable reference types
`C# 9–10`	2020–21	records, top-level statements, global using
`C# 11–12`	2022–23	raw strings, required, primary constructors, [ ] collections
`C# 13`	2024	params collections, Lock type, partial properties
`C# 14`	2025	extension members, field keyword, ?.= assignment

.NET FAMILY

NAME	WHAT IT IS
`.NET Framework`	Windows-only original (4.8.x); maintenance only
`.NET Core`	Cross-platform rewrite (2016–2019)
`.NET 5+`	Unified successor; just ".NET"
`LTS`	Even versions (8, 10): 3 years support
`STS`	Odd versions (9, 11): 2 years support
`CLR`	Runtime: JIT, GC, type safety
`BCL`	Base Class Library: System.*

⚠ COMMON MISTAKES

- Confusing **.NET Framework** with modern **.NET** — new projects target .NET 8/10, not 4.8
- Assuming C# is Windows-only — it runs on macOS and Linux with the same SDK
- Mixing up C# version and .NET version — each .NET release sets the default C# version

✓ TIPS & BEST PRACTICES

- Start new work on the current **LTS** (.NET 10) unless you need a preview feature
- Check your version with `dotnet --version`
- Use `dotnet new list` to see every project template

QUICK SUMMARY

- `C#` — typed, OOP language that compiles to IL
- `CLR` — runtime that JITs IL and manages memory
- `.NET 10` — current LTS, ships C# 14
- `dotnet run` — the fastest way to run code

Installing the .NET SDK

SDK vs runtime · dotnet --info · global.json · Visual Studio · VS Code C# Dev Kit · Rider

SDK VS RUNTIME

To **write** C# you install the **.NET SDK**, which includes the compiler, the `dotnet` CLI, templates and every runtime. To only **run** apps, a machine needs just the **runtime** (or nothing at all for self-contained apps). Several SDKs can sit side by side; `global.json` pins which one a folder uses.

KEY FACTS

- Download: dotnet.microsoft.com/download
- SDK version ≠ runtime version (SDK 10.0.1xx → runtime 10.0)
- SDKs install side by side; newest is used by default
- Editors: Visual Studio, VS Code + C# Dev Kit, JetBrains Rider

INSTALL & VERIFY

```
# macOS (Homebrew)
brew install --cask dotnet-sdk

# Windows
winget install Microsoft.DotNet.SDK.10

# Linux (Ubuntu)
sudo apt-get install -y dotnet-sdk-10.0

# Any OS: official install script
./dotnet-install.sh --channel 10.0

dotnet --version          # 10.0.100
dotnet --list-sdks        # every installed SDK
dotnet --list-runtimes    # every installed runtime
dotnet --info             # full environment report
```

CODE EXPLAINED

- → `--version` — SDK that this folder will use
- → `--list-sdks` — all SDKs with install paths
- → `--list-runtimes` — ASP.NET Core / .NET / Desktop runtimes
- → `--info` — OS, RID, SDKs and runtimes in one report
- → `dotnet-install.sh` — no admin rights; good for CI

PIN AN SDK — GLOBAL.JSON

```
{
  "sdk": {
    "version": "10.0.100",
    "rollForward": "latestFeature"
  }
}
```

EDITORS & IDES

TOOL	NOTES
Visual Studio 2026	Windows; full IDE, debugger, profiler
VS Code + C# Dev Kit	Any OS; Solution Explorer, tests
JetBrains Rider	Any OS; strong refactoring
dotnet CLI	Works with any editor
dotnet new global.json	Creates the pin file

⚠ COMMON MISTAKES

- Installing only the runtime, then wondering why `dotnet new` is missing — you need the **SDK**
- Forgetting a `global.json` pins an old SDK — `dotnet --version` shows what's active
- Installing VS Code's old C# extension only — add **C# Dev Kit** for project and test support

✓ TIPS & BEST PRACTICES

- Commit `global.json` so the team and CI use the same SDK
- Use `rollForward: latestFeature` to accept SDK patch updates
- Run `dotnet --info` first when reporting build problems

QUICK SUMMARY

- `.NET SDK` — compiler, CLI, templates, runtimes
- `dotnet --list-sdks` — see what's installed
- `global.json` — pin the SDK per folder
- `C# Dev Kit` — VS Code support for projects and tests

The dotnet CLI

dotnet new · dotnet run · dotnet build · dotnet test · dotnet add package · dotnet publish · dotnet watch

ONE TOOL FOR EVERYTHING

The **`dotnet` CLI** creates, builds, runs, tests, packages and publishes projects. Every IDE calls it under the hood, so anything you do in Visual Studio can be scripted. Commands act on the project or solution in the current folder unless you pass a path.

KEY FACTS

- `dotnet <command> --help` lists every option
- `run`, `test` and `publish` build first automatically
- Default configuration is `Debug`; use `-c Release`
- Output goes to `bin/` and `obj/` (don't commit them)

TYPICAL WORKFLOW

```
dotnet new console -o MyApp          # create project in ./MyApp
cd MyApp
dotnet add package Serilog           # add a NuGet package
dotnet build                         # compile → bin/Debug/net10.0/
dotnet run -- --name Bob             # run; args after -- go to app
dotnet watch                         # rebuild + rerun on save

dotnet new xunit -o MyApp.Tests      # test project
dotnet add MyApp.Tests reference MyApp
dotnet test                          # run all tests

dotnet publish -c Release -o out     # deployable output
dotnet clean                         # remove build output
```

CODE EXPLAINED

- → `-o` — output folder (also sets the project name)
- → `--` — everything after it is passed to your program
- → `watch` — hot reload for supported edits
- → `add reference` — project-to-project reference
- → `publish` — trimmed, ready-to-deploy files
- → `-c Release` — optimized build

CORE COMMANDS

COMMAND	DOES
`dotnet new <template>`	Create project/file from a template
`dotnet new list`	Show installed templates
`dotnet build`	Compile project or solution
`dotnet run`	Build and run
`dotnet test`	Run unit tests
`dotnet format`	Apply .editorconfig style fixes

PACKAGES & TOOLS

COMMAND	DOES
`dotnet add package X`	Add NuGet PackageReference
`dotnet remove package X`	Remove a package
`dotnet list package --outdated`	Find upgrades
`dotnet restore`	Download dependencies
`dotnet pack`	Build a .nupkg
`dotnet tool install -g X`	Install a global CLI tool

⚠ COMMON MISTAKES

- Passing app args without `--` — `dotnet run --name Bob` is read as a CLI option
- Running `dotnet test` from a folder with several projects and no solution — it can't pick one
- Shipping a `Debug` build — always `publish -c Release`

✓ TIPS & BEST PRACTICES

- Use `dotnet watch test` for a fast test loop
- `dotnet new gitignore` adds a correct .NET `.gitignore`
- `dotnet format` in CI keeps style consistent

QUICK SUMMARY

- `dotnet new` — scaffold projects
- `dotnet run` / `watch` — run and hot-reload
- `dotnet test` — run tests
- `dotnet publish -c Release` — ship it

Projects & Solutions

PROJECTS AND SOLUTIONS

A **project** (`.csproj`) is an MSBuild file that lists the target framework, settings and dependencies; it produces one assembly. A **solution** (`.sln` or the newer XML `.slnx`) groups related projects. SDK-style projects include every `.cs` file in the folder automatically, so the file stays short.

KEY FACTS

- `.slnx` is the default for `dotnet new sln` in .NET 10
- `Directory.Build.props` applies settings to every project below it
- `<Nullable>enable</Nullable>` turns on nullable warnings
- `<ImplicitUsings>` adds common `using` directives

A TYPICAL .CSPROJ

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net10.0</TargetFramework>
    <LangVersion>latest</LangVersion>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <TreatWarningsAsErrors>true</TreatWarningsAsErrors>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Serilog" Version="4.2.0" />
    <ProjectReference Include="..\Core\Core.csproj" />
  </ItemGroup>
</Project>
```

CODE EXPLAINED

- → `Sdk=` — picks build rules (Web, Worker, Razor…)
- → `OutputType` — `Exe` or `Library` (default)
- → `TargetFramework` — `net10.0`; plural `TargetFrameworks` for multi-target
- → `LangVersion` — `latest`, `preview`, or a number
- → `PackageReference` — NuGet dependency
- → `ProjectReference` — another project in the repo

SOLUTION COMMANDS

COMMAND	DOES
`dotnet new sln -n App`	Create App.slnx
`dotnet sln add src/**/*.csproj`	Add projects
`dotnet sln list`	List projects
`dotnet sln migrate`	Convert .sln → .slnx
`dotnet build App.slnx`	Build all projects

DIRECTORY.BUILD.PROPS

```
<Project>
  <PropertyGroup>
    <TargetFramework>net10.0</TargetFramework>
    <Nullable>enable</Nullable>
    <AnalysisLevel>latest</AnalysisLevel>
  </PropertyGroup>
</Project>
```

⚠ COMMON MISTAKES

- Listing `.cs` files in the `.csproj` — SDK projects include them already, so you get duplicates
- Setting `LangVersion` higher than the SDK supports — the compiler errors out
- Repeating the same settings in every project instead of using `Directory.Build.props`

✓ TIPS & BEST PRACTICES

- Put shared settings in `Directory.Build.props` at the repo root
- Turn on `Nullable` and `TreatWarningsAsErrors` for new projects
- Prefer `.slnx` — it's readable and merges cleanly

QUICK SUMMARY

- `.csproj` — one project, one assembly
- `.slnx` — groups projects
- `TargetFramework` — which .NET to build for
- `Directory.Build.props` — shared settings

Program Structure

top-level statements · static void Main · string[] args · namespace · file-scoped namespace · global using

WHERE A PROGRAM STARTS

Every C# program has one **entry point**. Since C# 9, a single file can use **top-level statements** and the compiler generates the `Main` method for you. Larger apps still split code into **namespaces** and **classes** across files; braces `{ }` mark blocks and every statement ends with `;`.

KEY FACTS

- Only **one** file may contain top-level statements
- `args` is available automatically in top-level code
- `Main` may return `int` and/or be `async Task`
- File-scoped `namespace X;` saves one indentation level

CLASSIC VS TOP-LEVEL

```
// ── Classic (explicit Main) ──────────────────
using System;                       // import a namespace

namespace Demo;                     // file-scoped namespace (C# 10)

internal static class Program
{
    static int Main(string[] args)  // entry point
    {
        Console.WriteLine($"{args.Length} args");
        return 0;                   // exit code
    }
}

// ── Top-level (same program, C# 9+) ──────────
Console.WriteLine($"{args.Length} args");
return 0;
```

CODE EXPLAINED

- → `using` — lets you write `Console` instead of `System.Console`
- → `namespace Demo;` — applies to the whole file
- → `static int Main(string[] args)` — classic entry point
- → `return 0` — process exit code (0 = success)
- → Top-level — compiler wraps it in a generated `Main`

VALID MAIN SIGNATURES

SIGNATURE	USE
`static void Main()`	No args, no exit code
`static int Main(string[] args)`	Args + exit code
`static async Task Main()`	Await inside Main
`static async Task<int> Main(string[] a)`	Async + exit code

SYNTAX BASICS

RULE	EXAMPLE
`Statements end with ;`	`int x = 1;`
`Blocks use braces`	`if (ok) { … }`
`Case-sensitive`	`Main ≠ main`
`global using (C# 10)`	`global using System.Text;`
`Naming: PascalCase`	`OrderService, GetTotal()`

⚠ COMMON MISTAKES

- Putting top-level statements in two files — error CS8802; only one file may have them
- Declaring types **above** top-level statements — statements must come first in the file
- Writing `main` in lower case — C# is case-sensitive, so no entry point is found

✓ TIPS & BEST PRACTICES

- Use top-level statements for small tools and samples
- Use file-scoped namespaces in every new file
- Return a non-zero exit code on failure so scripts can detect it

QUICK SUMMARY

- `Main` — the single entry point
- Top-level statements — generated `Main`
- `namespace X;` — file-scoped namespace
- `using` — import namespaces

File-Based Apps

dotnet run app.cs · #:package · #:sdk · #:property · shebang · converting to a project

C# WITHOUT A PROJECT FILE

.NET 10 lets you run a single `.cs` file directly: `dotnet run app.cs`. Special `#:` directives at the top of the file add NuGet packages, choose an SDK or set MSBuild properties. It's ideal for scripts and quick experiments, and you can convert it to a full project later.

KEY FACTS

- Requires the **.NET 10 SDK** or newer
- Directives must appear at the top of the file
- Builds are cached, so later runs start fast
- On Unix a `#!` shebang makes the file executable

APP.CS

```
#!/usr/bin/env dotnet
#:package Humanizer@2.14.1
#:property Nullable=enable

using Humanizer;

var since = DateTime.Now - new DateTime(2002, 2, 13);
Console.WriteLine($"C# is {since.Humanize(2)} old");

// Run:      dotnet run app.cs
// Or:       chmod +x app.cs && ./app.cs
// Convert:  dotnet project convert app.cs
```

CODE EXPLAINED

- → `#!/usr/bin/env dotnet` — shebang for Unix shells
- → `#:package Name@Version` — NuGet reference
- → `#:property` — any MSBuild property
- → `#:sdk Microsoft.NET.Sdk.Web` — switch SDK (e.g. web apps)
- → `dotnet project convert` — creates a `.csproj` from the file

DIRECTIVES

DIRECTIVE	EXAMPLE
`#:package`	`#:package Spectre.Console@0.49.1`
`#:sdk`	`#:sdk Microsoft.NET.Sdk.Web`
`#:property`	`#:property LangVersion=preview`
`#:project`	`#:project ../Lib/Lib.csproj`

FILE-BASED VS PROJECT

NEED	USE
One-off script, demo	File-based app
Multiple source files	Project
Tests, CI, publishing	Project
Teaching / trying a feature	File-based app

⚠ COMMON MISTAKES

- Putting `#:` directives below `using` lines — they must come first
- Omitting the version in `#:package` — pin it so the script stays reproducible
- Running with an older SDK — `dotnet run app.cs` needs .NET 10+

✓ TIPS & BEST PRACTICES

- Great replacement for small PowerShell or Python helper scripts
- Use `#:sdk Microsoft.NET.Sdk.Web` to spin up a minimal API in one file
- Convert with `dotnet project convert` once the script grows

QUICK SUMMARY

- `dotnet run app.cs` — run a single file
- `#:package` — add NuGet packages
- `#:property` / `#:sdk` — configure the build
- `dotnet project convert` — graduate to a project

Compilation & the Runtime

FROM SOURCE TO MACHINE CODE

The **Roslyn** compiler turns C# into **IL** stored in an **assembly** (`.dll`/`.exe`). At run time the CLR's **JIT** compiles IL to native code method by method, using **tiered compilation** to start fast and re-optimize hot code. **ReadyToRun** and **Native AOT** compile ahead of time for faster startup.

KEY FACTS

- Assembly = IL + metadata + manifest
- Tier 0: quick JIT; Tier 1: optimized after ~30 calls
- Dynamic PGO tunes hot paths using real usage
- Native AOT: no JIT, small single binary, some limits

PUBLISHING OPTIONS

```
# Framework-dependent (needs .NET runtime installed)
dotnet publish -c Release

# Self-contained (bundles the runtime)
dotnet publish -c Release -r linux-x64 --self-contained

# Single file
dotnet publish -c Release -r win-x64 -p:PublishSingleFile=true

# ReadyToRun (precompiled IL, JIT still available)
dotnet publish -c Release -r osx-arm64 -p:PublishReadyToRun=true

# Native AOT (set in .csproj: <PublishAot>true</PublishAot>)
dotnet publish -c Release -r linux-x64
```

CODE EXPLAINED

- → `-r` — runtime identifier (RID): `win-x64`, `linux-arm64`, `osx-arm64`
- → `--self-contained` — no runtime needed on target
- → `PublishSingleFile` — one executable
- → `PublishReadyToRun` — faster startup, larger files
- → `PublishAot` — native code; reflection is limited

PIPELINE

STAGE	TOOL / OUTPUT
`Compile`	Roslyn → IL in .dll
`Load`	CLR loads assembly
`JIT`	IL → native, per method
`Tier up`	Hot methods re-JIT optimized
`GC`	Frees unreachable objects

DEPLOYMENT MODES

MODE	STARTUP	SIZE
`Framework-dependent`	Normal	Smallest
`Self-contained`	Normal	Large
`ReadyToRun`	Faster	Larger
`Native AOT`	Fastest	Small, native

⚠ COMMON MISTAKES

- Expecting Native AOT to support everything — reflection-heavy code and `dynamic` may fail
- Benchmarking only the first call — Tier 0 code is slower until the JIT tiers up
- Forgetting `-r` for self-contained or AOT builds — a RID is required

✓ TIPS & BEST PRACTICES

- Use Native AOT for CLIs and serverless where startup matters
- Inspect IL with **ILSpy** or `sharplab.io`
- Use BenchmarkDotNet for timing — it handles warm-up for you

QUICK SUMMARY

- `Roslyn` — C# → IL
- `JIT` — IL → native at run time
- `ReadyToRun` — precompiled for startup
- `Native AOT` — no JIT, single native binary

Console Output

WRITING TO THE CONSOLE

`System.Console` writes text to **standard output** (`Console.Out`) and **standard error** (`Console.Error`). `WriteLine` adds a newline, `Write` does not. Both accept any value (calling `ToString()`) and support **composite formatting** with `{0}` placeholders, though string interpolation `$"..."` is usually clearer.

KEY FACTS

- `using System;` is implicit with ImplicitUsings
- `Console.WriteLine()` with no args prints a blank line
- `null` prints as an empty string
- `Debug.WriteLine` goes to the debugger, not the console

OUTPUT VARIANTS

```
Console.WriteLine("Hello");             // Hello⏎
Console.Write("A"); Console.Write("B"); // AB (no newline)
Console.WriteLine(42);                  // 42
Console.WriteLine(3.5 * 2);             // 7
Console.WriteLine(true);                // True

string name = "Ada"; int age = 36;
Console.WriteLine($"{name} is {age}");      // Ada is 36
Console.WriteLine("{0} is {1}", name, age); // Ada is 36
Console.WriteLine($"[{name,-6}][{age,4}]"); // [Ada   ][  36]
Console.WriteLine("Tab:\tNew:\nX");         // Tab:    New:⏎X

Console.Error.WriteLine("Something failed"); // to stderr
System.Diagnostics.Debug.WriteLine("dbg");   // debugger only
```

CODE EXPLAINED

- → `WriteLine` — value + newline
- → `Write` — value only; build a line in pieces
- → `$"{x}"` — interpolation (preferred)
- → `{0}`, `{1}` — composite format placeholders
- → `{x,-6}` / `{x,4}` — left / right align in 6 / 4 chars
- → `Console.Error` — stderr; survives `> out.txt` redirects

ESCAPE SEQUENCES

ESCAPE	MEANING
`\n`	New line
`\t`	Tab
`\"`	Double quote
`\\`	Backslash
`é`	Unicode char (é)
`\e`	Escape char (C# 13)

CONSOLE MEMBERS

MEMBER	DOES
`ForegroundColor`	= ConsoleColor.Green
`BackgroundColor`	= ConsoleColor.DarkBlue
`ResetColor()`	Restore default colors
`Clear()`	Clear the window
`SetCursorPosition(x, y)`	Move the cursor
`OutputEncoding`	= Encoding.UTF8 for emoji on Windows

⚠ COMMON MISTAKES

- Writing errors to `Console.WriteLine` — they disappear into redirected output; use `Console.Error`
- Forgetting `ResetColor()` after changing colors — the terminal stays colored
- Expecting `Debug.WriteLine` in Release builds — calls are removed by `[Conditional("DEBUG")]`

✓ TIPS & BEST PRACTICES

- Prefer `$"..."` over `{0}` placeholders for readability
- Use alignment `{x,-10}` to print simple tables
- For logs in real apps use `ILogger`, not `Console`

QUICK SUMMARY

- `Console.WriteLine` — print with newline
- `Console.Write` — print without newline
- `Console.Error` — write to stderr
- `{x,10}` — alignment inside interpolation

Console Input

READING USER INPUT

`Console.ReadLine()` returns the next line as a `string?` — it is `null` when input ends (Ctrl+D / Ctrl+Z or a closed pipe). Input is always text, so convert it with `int.Parse` or, better, `int.TryParse`, which returns `false` instead of throwing on bad input. `Console.ReadKey()` reads a single key press.

KEY FACTS

- `ReadLine` blocks until Enter is pressed
- Return type is `string?` — check for `null`
- `TryParse` uses an `out` parameter for the result
- Command-line args arrive in `args` (string[])

SAFE INPUT LOOP

```
Console.Write("Enter your age: ");
string? line = Console.ReadLine();      // "36" or null at EOF

if (int.TryParse(line, out int age))    // true → age = 36
    Console.WriteLine($"Next year: {age + 1}");
else
    Console.WriteLine("Not a number"); // "abc" → false

int n;
do { Console.Write("1-10: "); }
while (!int.TryParse(Console.ReadLine(), out n) || n is < 1 or > 10);

ConsoleKeyInfo key = Console.ReadKey(intercept: true); // not echoed
if (key.Key == ConsoleKey.Escape) return;

string name = args.Length > 0 ? args[0] : "guest";  // CLI args
```

CODE EXPLAINED

- → `string?` — ReadLine may return `null`
- → `int.TryParse(s, out int x)` — no exception on bad text
- → `do…while` — re-prompt until input is valid
- → `n is < 1 or > 10` — range check with patterns
- → `ReadKey(intercept: true)` — read a key without echoing it
- → `args[0]` — first command-line argument

PARSE VS TRYPARSE

CALL	INPUT	RESULT
`int.Parse(s)`	`"42"`	42
`int.Parse(s)`	`"4x"`	FormatException
`int.Parse(s)`	`null`	ArgumentNullException
`int.TryParse(s, out n)`	`"4x"`	false, n = 0
`double.TryParse(s, out d)`	`"3.5"`	true, d = 3.5
`bool.TryParse(s, out b)`	`"TRUE"`	true, b = true

READING METHODS

METHOD	RETURNS
`Console.ReadLine()`	string? — line or null
`Console.ReadKey()`	ConsoleKeyInfo — echoes key
`Console.Read()`	int — next char code, -1 at end
`Console.KeyAvailable`	bool — key waiting?
`Console.In.ReadToEnd()`	All piped input

⚠ COMMON MISTAKES

- Calling `int.Parse(Console.ReadLine())` — throws on empty or bad input; use `TryParse`
- Ignoring `null` from `ReadLine` — piped input can end and cause a `NullReferenceException`
- Parsing decimals without a culture — `"3,5"` vs `"3.5"` depends on the user's locale

✓ TIPS & BEST PRACTICES

- Pass `CultureInfo.InvariantCulture` when parsing machine-generated text
- Wrap prompting in a helper like `int ReadInt(string prompt)`
- Use `System.CommandLine` or Spectre.Console for richer CLIs

QUICK SUMMARY

- `Console.ReadLine()` — read a line (`string?`)
- `int.TryParse` — safe conversion
- `Console.ReadKey()` — single key press
- `args` — command-line arguments

Comments & XML Docs

COMMENTS AND XML DOCS

C# has **single-line** `//` and **block** `/* */` comments, plus **XML documentation comments** `///` that tools read. XML docs appear as IntelliSense hover text and can be exported to an `.xml` file for doc generators like DocFX. Enable `GenerateDocumentationFile` to get warnings for undocumented public members.

KEY FACTS

- Block comments do not nest
- `///` must sit directly above the member it documents
- `<inheritdoc/>` copies docs from a base or interface
- `#region` only folds code in the editor

DOCUMENTED METHOD

```
// Single-line comment
/* Block comment
   across lines */

/// <summary>Calculates the final price after a discount.</summary>
/// <param name="price">Original price; must be ≥ 0.</param>
/// <param name="percent">Discount from 0 to 100.</param>
/// <returns>The discounted price, rounded to 2 places.</returns>
/// <exception cref="ArgumentOutOfRangeException">percent &gt; 100</exception>
/// <example><code>Discount(50m, 10) // 45.00</code></example>
public static decimal Discount(decimal price, int percent)
{
    ArgumentOutOfRangeException.ThrowIfGreaterThan(percent, 100);
    return Math.Round(price * (100 - percent) / 100m, 2); // TODO: currency
}
```

CODE EXPLAINED

- → `<summary>` — shown in IntelliSense
- → `<param name="x">` — one per parameter
- → `<returns>` — what the method gives back
- → `<exception cref="T">` — documents a thrown exception
- → `>` — XML needs `<` and `>` escaped
- → `// TODO` — shows in the IDE's task list

COMMON XML TAGS

TAG	USE
`<summary>`	One-line description
`<remarks>`	Longer details
`<param>` / `<paramref>`	Describe / refer to a parameter
`<typeparam>`	Generic type parameter
`<see cref="…"/>`	Link to another member
`<inheritdoc/>`	Reuse base/interface docs

SETTINGS & DIRECTIVES

ITEM	EFFECT
`<GenerateDocumentationFile>`	true → builds .xml docs
`CS1591`	Warning: missing XML comment
`#pragma warning disable CS1591`	Silence it locally
`#region Name … #endregion`	Collapsible block

⚠ COMMON MISTAKES

- Writing `List<int>` inside `///` text — use `List{T}` in `cref` or `<int>` in prose
- Comments that restate the code (`i++; // add one`) — explain **why**, not what
- Leaving stale `<param>` names after a rename — the compiler warns with CS1572/CS1573

✓ TIPS & BEST PRACTICES

- Type `///` above a member and the IDE scaffolds the tags
- Use `<inheritdoc/>` on interface implementations
- Avoid `#region` to hide long classes — split the class instead

QUICK SUMMARY

- `//` and `/* */` — normal comments
- `///` — XML documentation
- `<summary>` , `<param>` , `<returns>` — core tags
- `GenerateDocumentationFile` — export docs

Sheet 011

Variables & Constants

var · explicit types · const · readonly · definite assignment · default · naming conventions

DECLARING VARIABLES

C# is **statically typed**: every variable has a type fixed at compile time. You can write the type explicitly or let the compiler infer it with `var` — the variable is still strongly typed. `const` declares a compile-time constant; `readonly` fields can be set once, in a declaration or constructor.

KEY FACTS

- Locals must be **definitely assigned** before use
- `var` needs an initializer and can't be `null` alone
- `const` works only with primitives, `string`, enums, `null`
- Use `@` to name a variable after a keyword: `@class`

VARIABLES IN PRACTICE

```
int count = 10;                  // explicit type
var total = 19.99m;              // inferred: decimal
string? note = null;             // nullable reference
int a = 1, b = 2;                // several at once
int x = default;                 // 0
DateTime start = new(2026, 1, 1); // target-typed new

const double Pi = 3.14159;       // compile-time constant
const string Greeting = "Hi";

class Config
{
    public readonly string Path;       // set once
    public static readonly DateTime Loaded = DateTime.Now;
    public Config(string path) => Path = path;
}

int y;
// Console.WriteLine(y);         // CS0165: unassigned local
```

CODE EXPLAINED

- → `var` — type inferred from the right side; still static
- → `default` — zero value for the target type
- → `new(…)` — type taken from the declaration
- → `const` — baked into callers at compile time
- → `readonly` — assigned in field initializer or constructor only
- → `static readonly` — runtime value, set once per type

NAMING CONVENTIONS

KIND	STYLE	EXAMPLE
Local / parameter	camelCase	`orderTotal`
Private field	_camelCase	`_logger`
Public member	PascalCase	`OrderTotal`
Constant	PascalCase	`MaxRetries`
Interface	I + PascalCase	`IRepository`
Type parameter	T + Pascal	`TResult`

CONST VS READONLY

	CONST	READONLY
Set when	compile time	run time
Where	field or local	field only
Types	primitives, string	any type
Implicitly static	yes	no
Versioning	copied into callers	read at run time

⚠ COMMON MISTAKES

- `var x = null;` — error CS0815; the compiler can't infer a type from `null`
- Using `public const` for values that may change — other assemblies keep the old value until rebuilt
- Thinking `readonly` makes an object immutable — `readonly List<T>` can still be modified

✓ TIPS & BEST PRACTICES

- Use `var` when the type is obvious from the right-hand side
- Prefer `static readonly` over `public const` for shared values
- Declare variables in the smallest scope that needs them

QUICK SUMMARY

- `var` — inferred, still strongly typed
- `const` — compile-time constant
- `readonly` — assign once at run time
- `default` — zero value of a type

Integer Types

byte · short · int · long · unsigned types · nint / nuint · Int128 · 0x / 0b / _ literals

WHOLE-NUMBER TYPES

C# has signed and unsigned integers from 8 to 128 bits. **`int`** (32-bit) is the default for literals and the usual choice; use `long` for larger values. Arithmetic overflow **wraps around silently** unless you use a `checked` context. Every keyword is an alias for a `System` struct, e.g. `int` = `System.Int32`.

KEY FACTS

- Integer literals are `int` unless too big or suffixed
- `_` digit separators: `1_000_000`
- `nint` / `nuint` are 32 or 64 bits depending on the platform
- `Int128` / `UInt128` exist since .NET 7

INTEGER LITERALS & OVERFLOW

```
int  dec = 1_000_000;          // 1000000
int  hex = 0xFF;               // 255
int  bin = 0b_1010_1010;       // 170
long big = 9_000_000_000L;     // L suffix → long
uint u   = 42U;                // U → uint
ulong ul = 42UL;               // UL → ulong

int max = int.MaxValue;        // 2147483647
int wrap = max + 1;            // -2147483648 (silent wrap)
int safe = checked(max + 1);   // OverflowException

Console.WriteLine(7 / 2);      // 3 (integer division)
Console.WriteLine(-7 / 2);     // -3 (truncates toward zero)
Console.WriteLine(-7 % 3);     // -1 (sign of dividend)
Console.WriteLine(int.Parse("42") + 1); // 43
```

CODE EXPLAINED

- → `0x` / `0b` — hex / binary literals
- → `L`, `U`, `UL` — choose the literal's type
- → `int.MaxValue + 1` — wraps to `int.MinValue`
- → `checked(...)` — throw on overflow instead
- → `/` on integers — discards the remainder
- → `%` — remainder takes the sign of the left operand

INTEGER TYPES

TYPE	BITS	RANGE
`sbyte / byte`	8	-128…127 / 0…255
`short / ushort`	16	-32,768…32,767 / 0…65,535
`int / uint`	32	±2.1 billion / 0…4.29 billion
`long / ulong`	64	$\pm 9.2\times10^{18}$ / $0\ldots1.8\times10^{19}$
`nint / nuint`	32/64	Platform pointer size
`Int128 / UInt128`	128	$\pm 1.7\times10^{38}$

USEFUL MEMBERS

MEMBER	EXAMPLE → RESULT
`MinValue / MaxValue`	`byte.MaxValue → 255`
`Parse / TryParse`	`int.Parse("7") → 7`
`ToString("N0")`	`1234567 → "1,234,567"`
`ToString("X")`	`255 → "FF"`
`Math.Abs / Math.Sign`	`Math.Sign(-5) → -1`
`int.Clamp(v, lo, hi)`	`int.Clamp(15, 0, 10) → 10`

⚠ COMMON MISTAKES

- Expecting `5 / 2` to be `2.5` — both operands are `int`; write `5 / 2.0` or cast first
- Silent overflow in sums of large values — use `long` or `checked`
- Using `uint` for counts that can go negative — `0u - 1` wraps to 4,294,967,295

✓ TIPS & BEST PRACTICES

- Set the `CheckForOverflowUnderflow` project property in Debug builds
- Use `long` for IDs, file sizes and timestamps
- Use `byte` only for raw binary data, not for small numbers

QUICK SUMMARY

- `int` — default 32-bit integer
- `long` — 64-bit, suffix `L`
- `checked` — throw on overflow
- `/` — integer division truncates

Floating-Point & Decimal

FRACTIONAL NUMBERS

`double` (64-bit) is the default for fractional literals and is fast but **binary**, so values like 0.1 are approximate. `float` (32-bit) saves memory. `decimal` (128-bit) stores **base-10** digits exactly and is the right choice for money. Floating-point math never throws: it produces `Infinity` or `NaN` instead.

KEY FACTS

- `3.14` is `double`; use `f` for float and `m` for decimal
- `NaN` is never equal to anything, even itself
- decimal ÷ 0 throws; double ÷ 0 gives Infinity
- `Half` (16-bit) exists for ML and graphics

FLOATING-POINT BEHAVIOR

```
double d = 3.14;            // default
float  f = 3.14f;           // f suffix required
decimal m = 3.14m;          // m suffix required

Console.WriteLine(0.1 + 0.2);          // 0.30000000000000004
Console.WriteLine(0.1 + 0.2 == 0.3);   // False
Console.WriteLine(0.1m + 0.2m == 0.3m);// True

Console.WriteLine(1.0 / 0);            // ∞ (PositiveInfinity)
Console.WriteLine(0.0 / 0);            // NaN
Console.WriteLine(double.NaN == double.NaN); // False
Console.WriteLine(double.IsNaN(0.0 / 0));    // True
// Console.WriteLine(1m / 0);          // DivideByZeroException

bool close = Math.Abs(a - b) < 1e-9;   // compare with tolerance
Console.WriteLine(1.5e3);              // 1500 (scientific)
```

CODE EXPLAINED

- → `f` / `m` — float and decimal literals need suffixes
- → `0.1 + 0.2` — binary rounding error
- → decimal — exact base-10 arithmetic
- → `1.0 / 0` — Infinity, no exception
- → `double.IsNaN` — the only reliable NaN test
- → Tolerance compare — never `==` on computed doubles

TYPES COMPARED

TYPE	SIZE	PRECISION	USE
`float`	32-bit	~6–9 digits	Graphics, large arrays
`double`	64-bit	~15–17 digits	Science, general math
`decimal`	128-bit	28–29 digits	Money, exact decimals
`Half`	16-bit	~3 digits	ML, GPU data

SPECIAL VALUES

MEMBER	MEANING
`double.NaN`	Not a number
`double.PositiveInfinity`	Result of 1.0 / 0
`double.Epsilon`	Smallest positive double
`double.IsFinite(x)`	Not NaN or ±∞
`decimal.Round(x, 2)`	Round money

⚠ COMMON MISTAKES

- Using `double` for currency — rounding errors add up; use `decimal`
- Testing `x == double.NaN` — always false; use `double.IsNaN(x)`
- Writing `float f = 3.14;` — error CS0664; add the `f` suffix

✓ TIPS & BEST PRACTICES

- Compare doubles with a tolerance or `Math.Abs(a - b) < eps`
- Format output with `ToString("F2")` instead of rounding stored values
- Mixing `decimal` and `double` needs an explicit cast — on purpose

QUICK SUMMARY

- `double` — default, fast, approximate
- `decimal` — exact base-10, use for money
- `NaN` / `Infinity` — no exceptions in float math
- `f` / `m` — literal suffixes

bool & char

BOOLEANS AND CHARACTERS

`bool` holds only `true` or `false` and does **not** convert to or from numbers. `char` is a single **UTF-16 code unit** written in single quotes. Most characters fit in one `char`, but emoji and some scripts need two (a surrogate pair), which is what `System.Text.Rune` represents.

KEY FACTS

- `if (1)` is an error — conditions must be `bool`
- `bool.ToString()` gives `"True"` / `"False"`
- `char` arithmetic produces `int`
- `"😀".Length` is 2 — two `char` s, one Rune

BOOL AND CHAR

```
bool done = false;
bool ok = bool.Parse("true");        // True
bool isAdult = age >= 18;            // comparison → bool

char c = 'A';
char tab = '\t';
char e = 'é';                   // é
Console.WriteLine(c + 1);            // 66 (int)
Console.WriteLine((char)(c + 1));    // B
Console.WriteLine(char.IsDigit('7'));        // True
Console.WriteLine(char.IsLetter('_'));       // False
Console.WriteLine(char.ToUpper('q'));        // Q
Console.WriteLine((int)'a');                 // 97
Console.WriteLine("7"[0] - '0');             // 7 (digit value)

foreach (Rune r in "hi😀".EnumerateRunes())
    Console.Write(r.Value + " ");    // 104 105 128512
```

CODE EXPLAINED

- → `bool.Parse` — case-insensitive `"true"` / `"false"`
- → `'A' + 1` — char promotes to int
- → `(char)` — cast back to a character
- → `char.IsDigit` , `IsLetter` , `IsWhiteSpace` — classify
- → `ch - '0'` — digit character to its value
- → `EnumerateRunes()` — iterate real code points

CHAR METHODS

METHOD	EXAMPLE → RESULT
`char.IsDigit`	`'5'` → `True`
`char.IsLetterOrDigit`	`'_'` → `False`
`char.IsWhiteSpace`	`'\t'` → `True`
`char.IsUpper / IsLower`	`'a'` → `False / True`
`char.ToUpper / ToLower`	`'b'` → `'B'`
`char.IsAsciiHexDigit`	`'F'` → `True`

CHAR LITERALS

LITERAL	VALUE
`'\n' '\r' '\t'`	Newline, return, tab
`'\'' '\\'`	Quote, backslash
`'\0'`	Null char (U+0000)
`'\u0041'`	Unicode escape → 'A'
`'\x41'`	Hex escape → 'A'
`'\e'`	ESC (U+001B), C# 13

⚠ COMMON MISTAKES

- Writing `"A"` when you mean `'A'` — double quotes make a `string` , not a `char`
- Assuming `string.Length` counts characters users see — emoji and accents can use several `char` s
- Comparing letters with `==` across cultures — use `char.ToUpperInvariant` for case-insensitive checks

✓ TIPS & BEST PRACTICES

- Name bools as questions: `isValid` , `hasItems` , `canRetry`
- Use `Rune` or `StringInfo` when handling user-visible characters
- Use `char.IsAscii*` helpers for fast, culture-free checks

QUICK SUMMARY

- `bool` — true/false only, no numeric conversion
- `char` — one UTF-16 code unit in `' '`
- `char.IsDigit` & friends — classify chars
- `Rune` — a full Unicode code point

Value vs Reference Types

stack vs heap · struct vs class · copy semantics · boxing · unboxing · ValueType

TWO KINDS OF TYPES

Value types (`struct`, `enum`, all numeric types, `bool`, `char`) hold their data directly, and assignment **copies** the value. **Reference types** (`class`, `string`, arrays, delegates, `record`) hold a reference to an object on the heap, and assignment copies the **reference**, so both variables see the same object.

KEY FACTS

- Value types can't be `null` (unless `T?`)
- Locals of value type usually live on the stack
- **Boxing** wraps a value in an object on the heap
- `string` is a reference type but behaves like a value (immutable)

COPY VS SHARE

```
struct PointS { public int X; }
class  PointC { public int X; }

var s1 = new PointS { X = 1 };
var s2 = s1;          // copies the data
s2.X = 99;
Console.WriteLine(s1.X);   // 1  (unchanged)

var c1 = new PointC { X = 1 };
var c2 = c1;          // copies the reference
c2.X = 99;
Console.WriteLine(c1.X);   // 99 (same object)

object boxed = 42;          // boxing: int → object (heap)
int unboxed = (int)boxed;   // unboxing
// long bad = (long)boxed;  // InvalidCastException

Console.WriteLine(ReferenceEquals(c1, c2)); // True
```

CODE EXPLAINED

- → `struct` assignment — independent copy
- → `class` assignment — two names, one object
- → Boxing — allocation each time a value becomes `object`
- → Unboxing — must cast to the **exact** boxed type
- → `ReferenceEquals` — same object?

VALUE VS REFERENCE

	VALUE TYPE	REFERENCE TYPE
`Examples`	int, double, bool, DateTime, struct	class, string, array, record
`Variable holds`	The data	A reference
`Assignment`	Copies data	Copies reference
`Default`	0 / false / zeroed	null
`== (default)`	Compares values	Compares references
`Inheritance`	No (sealed)	Yes

PASSING TO METHODS

CALL	EFFECT
`Method(valueVar)`	Method gets a copy
`Method(refObj)`	Method can mutate the object
`Method(ref valueVar)`	Method can change caller's value
`Method(in bigStruct)`	Read-only, no copy

⚠ COMMON MISTAKES

- Mutating a struct inside a `List<T>` — `list[0].X = 5` won't compile because the indexer returns a copy
- Unboxing to a different numeric type — `(long)(object)42` throws; unbox then convert
- Assuming `class` parameters are passed "by reference" — the reference is copied; reassigning it doesn't affect the caller

✓ TIPS & BEST PRACTICES

- Make structs small and immutable (`readonly struct`)
- Avoid boxing in hot loops — use generics instead of `object`
- Use `record` when you want reference types with value equality

QUICK SUMMARY

- Value types — copy the data
- Reference types — copy the reference
- Boxing — value → `object`, costs an allocation
- `ReferenceEquals` — identity check

Nullable Value Types

VALUE TYPES THAT CAN BE NULL

Adding `?` to a value type (`int?`, `DateTime?`) creates `Nullable<T>`, which can hold a value or **no value**. It's ideal for optional database columns or missing input. Operators are **lifted**: if either operand is null, arithmetic gives null and comparisons give `false`.

KEY FACTS

- `int?` is shorthand for `Nullable<int>`
- Accessing `.Value` when empty throws
- `??` supplies a fallback value
- `x is int n` unwraps and tests in one step

WORKING WITH INT?

```
int? score = null;
Console.WriteLine(score.HasValue);          // False
Console.WriteLine(score ?? -1);             // -1
Console.WriteLine(score.GetValueOrDefault()); // 0
// int bad = score.Value;                   // InvalidOperationException

score = 85;
if (score is int s) Console.WriteLine(s + 1);  // 86
int sure = score.Value;                     // 85 (safe now)

int? a = null, b = 5;
Console.WriteLine(a + b);         // (empty) — null
Console.WriteLine(a > b);         // False
Console.WriteLine(a < b);         // False (!)
Console.WriteLine(a == null);     // True

DateTime? shipped = order.ShippedAt;
string label = shipped?.ToString("d") ?? "Not shipped";
```

CODE EXPLAINED

- → `HasValue` — true when a value is present
- → `??` — use right side when left is null
- → `GetValueOrDefault()` — value or `default(T)`
- → `is int s` — pattern match and unwrap
- → Lifted ops — `null + 5` is `null`
- → `?.` — call a member only when not null

NULLABLE<T> MEMBERS

MEMBER	RESULT
`x.HasValue`	bool
`x.Value`	T, or throws if empty
`x.GetValueOrDefault()`	T or default(T)
`x.GetValueOrDefault(7)`	T or 7
`Nullable.GetUnderlyingType(t)`	int for typeof(int?)

LIFTED OPERATORS (A = NULL)

EXPRESSION	RESULT
`a + 1`	`null`
`a == null`	`true`
`a > 1 / a <= 1`	`false / false`
`(bool?)null & false`	`false`
`(bool?)null | true`	`true`

⚠ COMMON MISTAKES

- Reading `.Value` without checking — throws `InvalidOperationException` when empty
- Assuming `!(a > b)` means `a <= b` — with nulls both are false
- Using `int?` where `0` is a fine default — nullability adds checks everywhere

✓ TIPS & BEST PRACTICES

- Prefer `x is int v` or `x ?? fallback` over `.HasValue` + `.Value`
- Map nullable DB columns to `T?` properties
- Use `??=` to set a default only once

QUICK SUMMARY

- `T?` — value type that can be null
- `HasValue` / `Value` — test and read
- `??` — fallback when null
- Lifted operators — null propagates

Nullable Reference Types

NULL SAFETY FOR REFERENCES

With <Nullable>enable</Nullable>, reference types are **non-nullable by default**: `string` means "never null" and `string?` means "may be null". The compiler tracks null-state through your code and **warns** on possible null dereferences. It's a compile-time analysis only — nothing changes at run time.

KEY FACTS

- On by default in new projects since .NET 6
- `!` (null-forgiving) silences a warning — no runtime check
- `#nullable enable|disable|restore` toggles per file/region
- Attributes live in `System.Diagnostics.CodeAnalysis`

NULLABLE ANNOTATIONS

```
#nullable enable
string name = "Ada";        // never null
string? nick = null;        // may be null

int a = name.Length;        // fine
int b = nick.Length;        // ⚠ CS8602: possible null dereference
int c = nick?.Length ?? 0; // safe
if (nick is not null)
    Console.WriteLine(nick.Length);  // flow analysis: not null here

string forced = nick!;      // "trust me" — no runtime check

public class User
{
    public required string Email { get; init; } // must be set
    public string? Phone { get; set; }
}

static bool TryFind(int id, [NotNullWhen(true)] out User? user)
{ user = id == 1 ? new User { Email = "a@b.c" } : null; return user != null; }
```

CODE EXPLAINED

- → `string?` — opt in to nulls
- → CS8602 — dereference of a maybe-null value
- → `is not null` — narrows to non-null in that branch
- → `!` — suppresses the warning only
- → `required` — caller must set it, avoids CS8618
- → `[NotNullWhen(true)]` — non-null when method returns true

COMMON WARNINGS

CODE	MEANING
CS8600	Null assigned to non-nullable
CS8602	Dereference of possibly null
CS8603	Possible null return
CS8604	Possible null argument
CS8618	Non-nullable member not initialized
CS8625	null literal to non-nullable

ATTRIBUTES

ATTRIBUTE	TELLS THE COMPILER
`[NotNullWhen(true)]`	out arg is non-null if true
`[MaybeNull]`	May return/hold null
`[NotNull]`	Non-null after the call
`[AllowNull]`	Accepts null input
`[MemberNotNull(nameof(X))]`	Method initializes X

⚠ COMMON MISTAKES

- Sprinkling `!` to silence warnings — it hides real bugs; fix the flow instead
- Thinking annotations prevent nulls at run time — callers with nullable off can still pass null
- Leaving constructors that don't set non-nullable fields — use `required` or initialize them

✓ TIPS & BEST PRACTICES

- Set `WarningsAsErrors` to `nullable` in new projects
- Still validate public API arguments with `ArgumentNullException.ThrowIfNull`
- Migrate old projects file by file with `#nullable enable`

QUICK SUMMARY

- `string` vs `string?` — non-null vs maybe-null
- Compiler warnings, not runtime checks
- `!` — null-forgiving operator
- `[NotNullWhen]` — teach the flow analysis

Type Conversion & Casting

implicit · explicit (cast) · Convert.ToInt32 · Parse · TryParse · checked · unchecked

CHANGING A VALUE'S TYPE

Implicit conversions happen automatically when no data can be lost (`int` → `long`, `int` → `double`). **Explicit** casts `(T)x` are required when data may be lost, and they truncate or wrap silently. Text is converted with `Parse`/`TryParse`, and `Convert.ToX` handles many types and rounds instead of truncating.

KEY FACTS

- `(int)3.9` → 3 (truncates)
- `Convert.ToInt32(3.5)` → 4 (banker's rounding)
- `checked` turns silent overflow into an exception
- Every type has `ToString()`

CONVERSIONS

```
int i = 42;
long l = i;                    // implicit widening
double d = i;                  // 42
int back = (int)9.99;          // 9  (truncate)
byte wrap = unchecked((byte)300); // 44 (300 - 256)
// byte boom = checked((byte)300); // OverflowException

Console.WriteLine(Convert.ToInt32(3.5));  // 4
Console.WriteLine(Convert.ToInt32(2.5));  // 2 (to even)
Console.WriteLine(Convert.ToInt32("12")); // 12
Console.WriteLine(Convert.ToBoolean(1));  // True

int p = int.Parse("123");                 // 123
bool ok = double.TryParse("1e3", out var x); // True, 1000
string s = 3.75.ToString("F1");           // "3.8"

decimal price = (decimal)19.99;           // double → decimal
int code = (int)DayOfWeek.Friday;         // 5 (enum → int)
```

CODE EXPLAINED

- → Widening — implicit and safe
- → `(int)` — explicit, drops the fraction
- → `unchecked` — overflow wraps (default)
- → `Convert.ToInt32(double)` — rounds to even
- → `Parse` / `TryParse` — string → number
- → Enum ↔ int — explicit cast both ways

IMPLICIT (SAFE)

FROM	TO
`byte, short`	`int, long, double, decimal`
`int`	`long, float, double, decimal`
`long`	`float, double, decimal`
`float`	`double`
`char`	`int, long, double`
`derived class`	`base class / interface`

WAYS TO CONVERT

METHOD	ON FAILURE
`(T)x`	Truncates / wraps / InvalidCastException
`Convert.ToInt32(x)`	Throws; null string → 0
`int.Parse(s)`	FormatException
`int.TryParse(s, out n)`	Returns false
`x as T`	null (reference types)

⚠ COMMON MISTAKES

- Expecting `(int)2.7` to round — it truncates to 2; use `Math.Round` first
- Casting large `long` to `int` without `checked` — the value silently wraps
- Parsing locale-formatted numbers with the wrong culture — pass `CultureInfo.InvariantCulture`

✓ TIPS & BEST PRACTICES

- Use `TryParse` for anything a user typed
- Use `checked` around casts that must not overflow
- `int.CreateChecked(x)` / `CreateSaturating(x)` convert between number types generically

QUICK SUMMARY

- Implicit — widening, no data loss
- `(T)x` — explicit cast, may lose data
- `Convert.ToX` — rounding conversions
- `TryParse` — safe text conversion

Type Checking & Inspection

typeof · GetType() · is · as · nameof · sizeof · default(T)

INSPECTING TYPES

`typeof(T)` gets a `System.Type` for a type name at compile time, while `obj.GetType()` returns the **runtime** type of an object. The `is` operator tests compatibility (including base types and interfaces), `as` casts or returns `null`, and `nameof` turns a symbol into a string that survives renames.

KEY FACTS

- `is` matches derived types; `GetType() ==` does not
- `as` works only with reference and nullable types
- `nameof` is evaluated at compile time
- `sizeof` on primitives needs no `unsafe`

TYPE CHECKS

```
object o = "hello";

Console.WriteLine(typeof(int));          // System.Int32
Console.WriteLine(o.GetType().Name);     // String
Console.WriteLine(o is string);          // True
Console.WriteLine(o is IComparable);     // True (interface)

if (o is string s && s.Length > 3)       // test + cast
    Console.WriteLine(s.ToUpper());      // HELLO

string? maybe = o as string;             // "hello" or null
int? n = o as int?;                      // null

Console.WriteLine(nameof(o));            // o
Console.WriteLine(nameof(Console.WriteLine)); // WriteLine
Console.WriteLine(sizeof(long));         // 8
Console.WriteLine(default(int));         // 0

Animal a = new Dog();
Console.WriteLine(a is Animal);                  // True
Console.WriteLine(a.GetType() == typeof(Animal)); // False
```

CODE EXPLAINED

- `typeof(T)` — Type object from a type name
- `GetType()` — actual runtime type
- `is T name` — test and bind in one step
- `as` — cast or `null`, never throws
- `nameof` — symbol name as a string
- `is` vs `GetType() ==` — inheritance-aware vs exact

OPERATORS

OPERATOR	RETURNS	ON MISMATCH
`x is T`	bool	false
`x is T t`	bool + variable	false
`x as T`	T?	null
`(T)x`	T	InvalidCastException
`typeof(T)`	Type	—
`x.GetType()`	Type	NullReferenceException if null

SYSTEM.TYPE MEMBERS

MEMBER	EXAMPLE
`Name / FullName`	"List`1" / "System.Collections…"
`IsValueType`	typeof(int).IsValueType → true
`IsAssignableTo(t)`	typeof(Dog).IsAssignableTo(typeof(Animal))
`BaseType`	typeof(Dog).BaseType → Animal
`IsGenericType`	typeof(List<int>) → true

⚠ COMMON MISTAKES

- Calling `x.GetType()` on a null reference — throws; use `x is T` instead
- Using `as` and then not checking for `null` — you just move the crash
- Hard-coding member names in strings — use `nameof` so renames stay in sync

✓ TIPS & BEST PRACTICES

- Prefer `if (x is Foo f)` over `as` + null check
- Use `nameof` in `ArgumentException` messages and `PropertyChanged` events
- Reach for virtual methods before long `is` chains

QUICK SUMMARY

- `typeof` — compile-time type
- `GetType()` — runtime type
- `is` / `as` — safe type tests and casts
- `nameof` — rename-safe names

Enums

NAMED CONSTANTS

An `enum` defines a set of named integral constants, making code like `status == OrderStatus.Shipped` readable and type-safe. The underlying type is `int` by default. Mark an enum `[Flags]` and use powers of two to combine values with bitwise operators.

KEY FACTS

- First member is 0 unless you set values
- Any int can be cast to an enum, even undefined ones
- `default(MyEnum)` is 0 — define a `None = 0` member
- Generic helpers: `Enum.GetValues<T>()`, `Enum.Parse<T>()`

ENUMS AND FLAGS

```
enum OrderStatus { Pending, Paid, Shipped = 10, Delivered }  // 0,1,10,11
enum Level : byte { Low = 1, High = 2 }                      // byte-backed

var s = OrderStatus.Paid;
Console.WriteLine(s);                  // Paid
Console.WriteLine((int)s);             // 1
Console.WriteLine((OrderStatus)11);    // Delivered
Console.WriteLine((OrderStatus)99);    // 99 (no error!)

var p = Enum.Parse<OrderStatus>("Shipped");          // Shipped
Enum.TryParse("paid", ignoreCase: true, out OrderStatus q); // Paid
foreach (var v in Enum.GetValues<OrderStatus>()) Console.Write($"{v} ");

[Flags] enum Perm { None = 0, Read = 1, Write = 2, Exec = 4 }
var rw = Perm.Read | Perm.Write;
Console.WriteLine(rw);                 // Read, Write
Console.WriteLine(rw.HasFlag(Perm.Write)); // True
rw &= ~Perm.Write;                     // remove a flag → Read
```

CODE EXPLAINED

- → `Shipped = 10` — later members continue from 10
- → `: byte` — choose the underlying integer type
- → Casting undefined values — allowed; validate with `Enum.IsDefined`
- → `Enum.Parse<T>` — string → enum (throws if unknown)
- → `[Flags]` + `|` — combine; `ToString` lists names
- → `&= ~flag` — clear a flag

ENUM API

CALL	RESULT
`Enum.GetValues<T>()`	T[] of all values
`Enum.GetNames<T>()`	string[] of names
`Enum.IsDefined(value)`	true if named member
`Enum.TryParse<T>(s, out v)`	bool, no exception
`value.ToString("D")`	Numeric string "10"
`value.HasFlag(f)`	Flag set?

SWITCH ON AN ENUM

```
string Label(OrderStatus s) => s switch
{
    OrderStatus.Pending   => "Waiting",
    OrderStatus.Paid      => "Paid",
    OrderStatus.Shipped   => "On the way",
    OrderStatus.Delivered => "Done",
    _ => throw new ArgumentOutOfRangeException(nameof(s))
};
```

⚠ COMMON MISTAKES

- Trusting casts from external ints — `(OrderStatus)99` is legal; check `Enum.IsDefined`
- Flags values that aren't powers of two — `Read = 1, Write = 2, Exec = 3` overlaps
- Reordering members after persisting numbers — stored values now mean something else

✓ TIPS & BEST PRACTICES

- Give explicit values to enums stored in databases or JSON
- Always include `None = 0` in `[Flags]` enums
- Add a `_ =>` arm in switches to catch undefined values

QUICK SUMMARY

- `enum` — named integer constants
- `[Flags]` — combinable bit values
- `Enum.Parse<T>` / `TryParse` — from text
- `HasFlag` — test a flag

String Basics

TEXT IN C#

`string` (alias for `System.String`) is an **immutable** sequence of UTF-16 `chars`. Every method that seems to change a string returns a **new** one. Strings are reference types, but == compares their **contents**. C# offers regular, verbatim @"...", and raw """..."""" literals.

KEY FACTS

- `string.Empty == ""` ; default of `string` is `null`
- Index with `s[0]` ; read-only, no `s[0] = 'x'`
- Raw literals (C# 11) need no escaping
- `"abc"u8` is a UTF-8 `ReadOnlySpan<byte>`

LITERALS & BASICS

```
string a = "Hello";
string path = @"C:\temp\new";      // verbatim: \ is literal
string q = "She said \"hi\"";      // escaped quotes
string json = """
    { "name": "Ada", "tags": ["c#"] }
    """;                           // raw: no escapes, indent trimmed

Console.WriteLine(a.Length);       // 5
Console.WriteLine(a[1]);           // e
Console.WriteLine(a + " World");   // Hello World
Console.WriteLine(a == "Hello");   // True (content)
Console.WriteLine(new string('-', 5)); // -----
string up = a.ToUpper();           // "HELLO"; a unchanged

Console.WriteLine(string.IsNullOrEmpty(""));       // True
Console.WriteLine(string.IsNullOrWhiteSpace("  ")); // True
```

CODE EXPLAINED

- → `@"..."` — backslashes are literal; `""` for a quote
- → `"""..."""` — raw literal; closing quotes set indentation
- → `Length` — number of UTF-16 chars
- → `==` — ordinal content comparison
- → `new string(c, n)` — repeat a char
- → Immutability — `ToUpper` returns a new string

LITERAL FORMS

FORM	EXAMPLE	NOTES
Regular	`"a\tb"`	Escapes work
Verbatim	`@"C:\dir"`	Multi-line; "" for quote
Raw	`"""x"y"""`	C# 11; any content
Interpolated	`$"{x}"`	Embeds values
UTF-8	`"hi"u8`	ReadOnlySpan<byte>

STATIC HELPERS

METHOD	RESULT
`string.IsNullOrEmpty(s)`	null or ""
`string.IsNullOrWhiteSpace(s)`	null, "" or spaces
`string.Concat(a, b, c)`	Joined string
`string.Join(",", list)`	"a,b,c"
`string.Equals(a, b, cmp)`	Comparison-aware equality

⚠ COMMON MISTAKES

- Calling `s.Trim();` and ignoring the result — strings are immutable; write `s = s.Trim();`
- Building strings with `+=` in a loop — each step copies; use `StringBuilder` or `string.Join`
- Checking only `s == ""` — misses `null` ; use `string.IsNullOrEmpty`

✓ TIPS & BEST PRACTICES

- Use raw literals for JSON, SQL and regex patterns
- Prefer `string.IsNullOrWhiteSpace` for user input checks
- Use `string` (keyword) in code, `String` only in docs if you like

QUICK SUMMARY

- `string` — immutable UTF-16 text
- `@""` / `"""` — verbatim / raw literals
- `==` — compares contents
- `IsNullOrWhiteSpace` — robust empty check

Interpolation & Formatting

INTERPOLATION AND FORMAT STRINGS

String interpolation `$"...{expr}..."` inserts values directly. Inside a hole you can add **alignment** (`{x,10}`) and a **format specifier** (`{x:N2}`). Numeric and date formats follow the **current culture** unless you pass one, which matters for decimal separators and currency symbols.

KEY FACTS

- Format: `{value,alignment:format}`
- Escape braces by doubling: `{{ }}`
- `$$"""` raw strings use `{{x}}` for holes
- Results shown are for `en-US`

FORMAT SPECIFIERS

```
double price = 1234.5; int n = 42; double p = 0.256;
var today = new DateTime(2026, 10, 11, 14, 5, 0);

Console.WriteLine($"{price:C}");       // $1,234.50
Console.WriteLine($"{price:N2}");      // 1,234.50
Console.WriteLine($"{price:F1}");      // 1234.5
Console.WriteLine($"{n:D5}");          // 00042
Console.WriteLine($"{n:X}  {n:x4}");   // 2A  002a
Console.WriteLine($"{p:P1}");          // 25.6%
Console.WriteLine($"{price:E2}");      // 1.23E+003
Console.WriteLine($"{today:yyyy-MM-dd HH:mm}"); // 2026-10-11 14:05
Console.WriteLine($"[{n,6}] [{n,-6}]");// [    42] [42    ]
Console.WriteLine($"{{literal}} {n}"); // {literal} 42

string s = string.Format("{0:N0} items", 1500);    // 1,500 items
string inv = string.Create(CultureInfo.InvariantCulture, $"{1.5}");
string j = $$"""{"id": {{n}}}""";      // {"id": 42}
```

CODE EXPLAINED

- → `C` — currency; `N2` — thousands + 2 decimals
- → `F1` — fixed decimals, no grouping
- → `D5` — zero-pad integers
- → `X` / `x4` — hex, upper / lower, padded
- → `P1` — multiply by 100, add %
- → `string.Create(Invariant…)` — culture-independent output

NUMERIC FORMATS

SPEC	1234.567 →
G	1234.567
N / N0	1,234.57 / 1,235
F3	1234.567
0.00	1234.57
#,##0	1,235
000000	001235

DATE FORMATS

SPEC	RESULT
d / D	10/11/2026 / Sunday, October 11, 2026
t / T	2:05 PM / 2:05:00 PM
o	2026-10-11T14:05:00.0000000
yyyy-MM-dd	2026-10-11
MMM d, h:mm tt	Oct 11, 2:05 PM
ddd	Sun

⚠ COMMON MISTAKES

- Using `mm` for months — `mm` is minutes; months are `MM`
- Writing machine-readable text with the current culture — `1,5` vs `1.5`; use `InvariantCulture`
- Forgetting the `$` prefix — `"{x}"` prints the braces literally

✓ TIPS & BEST PRACTICES

- Use the `o` format for round-trip dates in logs and JSON
- Use `ToString("F2", CultureInfo.InvariantCulture)` for files
- Put complex expressions in a variable before interpolating

QUICK SUMMARY

- `$"{x}"` — interpolation
- `{x,10}` — alignment
- `{x:N2}` — format specifier
- `CultureInfo` — controls separators and symbols

String Methods — Search

FINDING TEXT

Search methods return a `bool` (`Contains`, `StartsWith`, `EndsWith`) or a **position** (`IndexOf`, `LastIndexOf`) that is `-1` when nothing is found. Most have overloads that take a `char` or a `StringComparison`, which decides between **ordinal** (byte-exact) and **culture-aware** comparison and whether case matters.

KEY FACTS

- `Contains(string)` is ordinal by default
- `IndexOf(string)` and `StartsWith(string)` are **culture-aware** by default
- `char` overloads are always ordinal and fastest
- Positions are zero-based

SEARCHING

```
string s = "Hello, World";

Console.WriteLine(s.Contains("World"));            // True
Console.WriteLine(s.Contains("world"));            // False
var ic = StringComparison.OrdinalIgnoreCase;
Console.WriteLine(s.Contains("world", ic));        // True
Console.WriteLine(s.StartsWith("Hell"));           // True
Console.WriteLine(s.EndsWith('d'));                // True

Console.WriteLine(s.IndexOf('o'));                 // 4
Console.WriteLine(s.IndexOf('o', 5));              // 8 (start at 5)
Console.WriteLine(s.LastIndexOf('o'));             // 8
Console.WriteLine(s.IndexOf("xyz"));               // -1
Console.WriteLine(s.IndexOfAny(['W', ',']));       // 5

int count = s.Count(c => c == 'l');                // 3 (LINQ)
bool any = s.AsSpan().ContainsAny('x', 'W');       // True
```

CODE EXPLAINED

- → `Contains` — substring present?
- → `StringComparison.OrdinalIgnoreCase` — fast case-insensitive
- → `IndexOf(c, start)` — begin the search at `start`
- → `-1` — not found
- → `IndexOfAny` — first of several chars
- → `AsSpan().ContainsAny` — allocation-free span search

SEARCH METHODS

METHOD	RETURNS
`Contains(x)`	bool
`StartsWith(x) / EndsWith(x)`	bool
`IndexOf(x [, start])`	int, -1 if missing
`LastIndexOf(x)`	int
`IndexOfAny(char[])`	int
`string.Equals(a, b, cmp)`	bool

STRINGCOMPARISON

VALUE	USE FOR
`Ordinal`	IDs, keys, paths, protocols
`OrdinalIgnoreCase`	Case-insensitive identifiers
`CurrentCulture`	Text shown to users
`CurrentCultureIgnoreCase`	User search boxes
`InvariantCulture`	Rare; stable linguistic sort

⚠ COMMON MISTAKES

- Lower-casing both strings to compare — allocates and fails for some cultures; pass `OrdinalIgnoreCase`
- Testing `IndexOf(x) > 0` — misses a match at position 0; use `>= 0` or `Contains`
- Relying on culture-aware `IndexOf(string)` for protocol text — results vary by machine

✓ TIPS & BEST PRACTICES

- Always pass a `StringComparison` when searching with a string
- Use `SearchValues<char>` for repeated multi-char searches (.NET 8+)
- Use `Regex` once patterns get complex

QUICK SUMMARY

- `Contains` / `StartsWith` / `EndsWith` — bool tests
- `IndexOf` — position or -1
- `StringComparison` — ordinal vs culture, case
- `char` overloads — always ordinal

String Methods — Transform

ToUpper · ToLower · Trim · Replace · Split · string.Join · PadLeft · Substring

CHANGING TEXT

Transform methods build and return a **new string**: case changes, trimming, replacing, splitting into arrays, joining collections, and padding. Combine them in chains, and remember to assign the result. `Split` and `Join` are the workhorses for parsing and producing delimited data.

KEY FACTS

- `Trim()` removes whitespace; `Trim('/')` removes chars
- `Split` options: `RemoveEmptyEntries`, `TrimEntries`
- `Substring(start, length)` — length, not end index
- Use `ToUpperInvariant` for culture-free case changes

TRANSFORMATIONS

```
string s = "  Hello, World  ";

Console.WriteLine(s.Trim());                 // "Hello, World"
Console.WriteLine(s.TrimStart());            // "Hello, World  "
Console.WriteLine("//a//".Trim('/'));        // "a"
Console.WriteLine("abc".ToUpper());          // "ABC"
Console.WriteLine("cat".Replace('c', 'b'));  // "bat"
Console.WriteLine("a-b-c".Replace("-", "")); // "abc"

string[] parts = "a, b,,c".Split(',',
    StringSplitOptions.RemoveEmptyEntries | StringSplitOptions.TrimEntries);
// ["a", "b", "c"]
Console.WriteLine(string.Join(" | ", parts));   // a | b | c

Console.WriteLine("7".PadLeft(3, '0'));      // "007"
Console.WriteLine("ab".PadRight(4) + "|");   // "ab  |"
Console.WriteLine("Hello".Substring(1, 3));  // "ell"
Console.WriteLine("Hello".Insert(5, "!"));   // "Hello!"
Console.WriteLine("Hello".Remove(1, 3));     // "Ho"
```

CODE EXPLAINED

- → `Trim` / `TrimStart` / `TrimEnd` — strip edges
- → `Replace` — all occurrences, char or string
- → `Split(sep, options)` — parse delimited text
- → `string.Join(sep, items)` — the reverse of Split
- → `PadLeft(width, ch)` — fixed-width output
- → `Insert` / `Remove` — edit by position

TRANSFORM METHODS

METHOD	EXAMPLE → RESULT
`ToLower / ToLowerInvariant`	`"AbC" → "abc"`
`Replace(old, new, cmp)`	`case-insensitive replace`
`Split(string[], opts)`	`multiple separators`
`Split(',', 2)`	`"a,b,c" → ["a", "b,c"]`
`ReplaceLineEndings()`	`normalize \r\n / \n`
`string.Concat(chars.Reverse())`	`"abc" → "cba"`

CHAINING

```
string slug = "  My First Post!  "
    .Trim()
    .ToLowerInvariant()
    .Replace(" ", "-")
    .Replace("!", "");      // "my-first-post"
```

⚠ COMMON MISTAKES

- `s.Replace("a", "b");` without assignment — the original is unchanged
- `Substring(2, 5)` read as "from 2 to 5" — the second argument is a **length**
- Splitting CSV with `Split(',')` — quoted fields with commas break; use a CSV library

✓ TIPS & BEST PRACTICES

- Use `s[start..end]` ranges as a clearer alternative to `Substring`
- Pass `TrimEntries` to `Split` instead of trimming each item
- Use `ToUpperInvariant` for keys; `ToUpper` for UI text

QUICK SUMMARY

- `Trim` — strip whitespace or chars
- `Replace` — swap text
- `Split` / `Join` — parse and build lists
- `PadLeft` / `PadRight` — fixed width

Indexing, Ranges & Spans

INDEX, RANGE AND SPAN

C# 8 added `^` (index from end) and `..` (range) operators, so `s[^1]` is the last char and `s[1..4]` is a substring. On a `string` a range allocates a new string; on a **`ReadOnlySpan<char>`** it just creates a view over the same memory, which makes parsing fast and allocation-free.

KEY FACTS

- Range end is **exclusive**: `[1..4]` = indexes 1, 2, 3
- `^1` = last, `^0` = one past the end
- `..` alone = everything
- Spans live on the stack — no `async` or fields

SLICING

```
string s = "Hello";

Console.WriteLine(s[0]);       // H
Console.WriteLine(s[^1]);      // o (last)
Console.WriteLine(s[1..4]);    // ell
Console.WriteLine(s[..2]);     // He
Console.WriteLine(s[^3..]);    // llo
Console.WriteLine(s[1..^1]);   // ell (trim first & last)

Index last = ^1;  Range mid = 1..^1;
Console.WriteLine(s[mid]);     // ell

ReadOnlySpan<char> span = "2026-10-11".AsSpan();
int year  = int.Parse(span[..4]);     // 2026 — no substring alloc
int month = int.Parse(span.Slice(5, 2)); // 10
Console.WriteLine(span[^2..].ToString()); // "11"

int[] nums = [1, 2, 3, 4, 5];
int[] copy = nums[1..3];       // [2, 3] (new array)
Span<int> view = nums.AsSpan(1, 2); view[0] = 99; // nums[1] = 99
```

CODE EXPLAINED

- → `^n` — nth from the end
- → `a..b` — start inclusive, end exclusive
- → `Index` / `Range` — store them in variables
- → `AsSpan()` — view, no copy
- → `int.Parse(span)` — parse without allocating
- → Array range — copies; span — shares memory

RANGE CHEAT SHEET ("HELLO")

EXPR	RESULT
`s[0]` / `s[^1]`	`'H'` / `'o'`
`s[..3]`	`"Hel"`
`s[3..]`	`"lo"`
`s[^2..]`	`"lo"`
`s[..]`	`"Hello"`
`s[^0]`	`IndexOutOfRangeException`

SPAN MEMBERS

MEMBER	DOES
`Slice(start, len)`	Sub-view
`IndexOf` / `Contains`	Search, no allocation
`Trim()`	Trimmed view
`SequenceEqual(other)`	Compare contents
`ToString()`	Allocate a string at the end
`MemoryExtensions.Split`	Enumerate ranges (.NET 9)

⚠ COMMON MISTAKES

- Expecting the range end to be included — `s[0..3]` is 3 chars
- Using `s[^0]` for the last char — it's one past the end; use `^1`
- Storing a `Span<T>` in a field or across `await` — not allowed; use `Memory<T>`

✓ TIPS & BEST PRACTICES

- Use spans in hot parsing code to avoid substrings
- Ranges also work on arrays and `List<T>` via `CollectionsMarshal.AsSpan`
- Call `ToString()` on a span only once you need a real string

QUICK SUMMARY

- `^1` — last element
- `a..b` — exclusive-end range
- `AsSpan()` — zero-copy view
- `Slice` — span sub-range

StringBuilder & Comparison

BUILDING AND COMPARING STRINGS

`StringBuilder` is a mutable buffer for building text in many steps without creating a new string each time. For comparison, choose **ordinal** (`StringComparison.Ordinal`) for identifiers and **culture-aware** for sorting user-visible text. `Encoding` converts between strings and bytes.

KEY FACTS

- `StringBuilder` is in `System.Text`
- `string.Compare` returns <0, 0 or >0
- `StringComparer.OrdinalIgnoreCase` plugs into collections
- UTF-8: ASCII = 1 byte; é = 2; 😀 = 4

STRINGBUILDER & COMPARISON

```
var sb = new StringBuilder();
sb.Append("Items: ");
for (int i = 1; i <= 3; i++) sb.Append(i).Append(',');
sb.Length--;                          // drop last comma
sb.AppendLine().AppendLine("Done");
sb.Insert(0, ">> ").Replace("Items", "IDs");
Console.WriteLine(sb.ToString());  // >> IDs: 1,2,3⏎Done⏎

bool same = string.Equals("ABC", "abc",
    StringComparison.OrdinalIgnoreCase);       // True
int ord = string.Compare("apple", "Banana",
    StringComparison.Ordinal);                 // > 0 ('a' > 'B')
int ui  = string.Compare("apple", "Banana",
    StringComparison.CurrentCulture);          // < 0

var names = new HashSet<string>(StringComparer.OrdinalIgnoreCase) { "Ada" };
Console.WriteLine(names.Contains("ADA"));     // True

byte[] bytes = Encoding.UTF8.GetBytes("héllo"); // 6 bytes
string back = Encoding.UTF8.GetString(bytes);   // "héllo"
```

CODE EXPLAINED

- → `Append` — returns the builder, so calls chain
- → `Length--` — trim the last char cheaply
- → `Insert` / `Replace` — edit in place
- → Ordinal — compares char codes (uppercase sorts first)
- → Culture — dictionary order for people
- → `Encoding.UTF8` — string ↔ bytes

WHICH BUILDER?

SITUATION	USE
Few fixed pieces	$"..." interpolation
Joining a collection	string.Join
Loop with many appends	StringBuilder
Known final length	string.Create
2-3 strings	+ is fine

COMPARISON APIS

API	USE
`a == b`	Ordinal, case-sensitive
`string.Equals(a, b, cmp)`	Choose rules
`string.Compare(a, b, cmp)`	Sort order
`a.CompareTo(b)`	Culture-aware sort
`StringComparer.X`	For Dictionary / HashSet / Sort

⚠ COMMON MISTAKES

- Using `+=` inside a large loop — O(n²) copying; switch to `StringBuilder`
- Sorting identifiers with culture rules — results change between machines; use `Ordinal`
- Assuming `string.Length` equals UTF-8 byte count — use `Encoding.UTF8.GetByteCount`

✓ TIPS & BEST PRACTICES

- Pre-size: `new StringBuilder(capacity: 1024)` when you know the size
- Create dictionaries with `StringComparer.OrdinalIgnoreCase` for case-insensitive keys
- Use `Encoding.UTF8` unless a protocol demands another encoding

QUICK SUMMARY

- `StringBuilder` — efficient repeated appends
- `StringComparison.Ordinal` — exact, fast
- `StringComparer` — rules for collections
- `Encoding.UTF8` — text ↔ bytes

Arithmetic Operators

+ - * / % · integer division · ++ / -- · Math.DivRem · overflow · precedence

MATH OPERATORS

C# provides the usual arithmetic operators. The **result type** depends on the operands: two `int`s give an `int` (so division truncates), while mixing in a `double` promotes the result to `double`. Small types like `byte` and `short` are promoted to `int` before any arithmetic.

KEY FACTS

- No `**` power operator — use `Math.Pow`
- `%` works on floating-point too: `5.5 % 2` = 1.5
- Integer ÷ 0 throws; double ÷ 0 gives ∞
- `byte + byte` is `int`

ARITHMETIC IN ACTION

```
int a = 17, b = 5;
Console.WriteLine(a + b);         // 22
Console.WriteLine(a - b);         // 12
Console.WriteLine(a * b);         // 85
Console.WriteLine(a / b);         // 3
Console.WriteLine(a % b);         // 2
Console.WriteLine(a / (double)b); // 3.4
Console.WriteLine(Math.Pow(2, 10)); // 1024 (double)

int i = 5;
Console.WriteLine(i++);           // 5 (then i = 6)
Console.WriteLine(++i);           // 7
Console.WriteLine(-i);            // -7

var (q, r) = Math.DivRem(17, 5); // q = 3, r = 2
Console.WriteLine(2 + 3 * 4);     // 14 (* before +)
Console.WriteLine((2 + 3) * 4);   // 20
byte x = 10, y = 20;
int sum = x + y;                  // 30 — result is int
```

CODE EXPLAINED

- → `/` on ints — quotient only
- → `(double)b` — cast one operand to get a fraction
- → `i++` — returns old value, then increments
- → `++i` — increments, then returns new value
- → `Math.DivRem` — quotient and remainder at once
- → Precedence — `* / %` before `+ -`

OPERATORS

OP	NAME	EXAMPLE	RESULT
`+`	Add / concat	`7 + 2`	`9`
`-`	Subtract / negate	`7 - 2`	`5`
`*`	Multiply	`7 * 2`	`14`
`/`	Divide	`7 / 2` | `7 / 2.0`	`3` | `3.5`
`%`	Remainder	`-7 % 2`	`-1`
`++ / --`	Increment / decrement	`x++`	`x + 1`

PRECEDENCE (HIGH → LOW)

LEVEL	OPERATORS
Primary	x.y f(x) a[i] x++ new
Unary	+x -x !x ~x ++x (T)x
Multiplicative	* / %
Additive	+ -
Shift	<< >> >>>
Then	comparison → logical → ?? → ?: → =

⚠ COMMON MISTAKES

- Computing a percentage as `count / total * 100` with ints — gives 0; use `count * 100.0 / total`
- Using `i++` inside complex expressions — hard to read and easy to get wrong
- Assuming `%` is always positive — `-7 % 3` is `-1`; use `((x % n) + n) % n` for modulo

✓ TIPS & BEST PRACTICES

- Add parentheses whenever precedence isn't obvious
- Use `Math.Pow` only for real exponents; `x * x` is faster for squares
- Use `checked` for arithmetic that must not overflow

QUICK SUMMARY

- `/` — integer division when both are ints
- `%` — remainder (sign of left side)
- `++x` vs `x++` — pre vs post increment
- `Math.Pow` — exponentiation

Sheet 028

Comparison & Equality Operators

== · != · < > <= >= · Equals · ReferenceEquals · string vs object equality

COMPARING VALUES

Relational operators (<, >, <=, >=) work on numbers, chars and enums. **Equality** depends on the type: value types and `string` compare contents, most classes compare **references**, and `record`s compare their properties. `Equals()` and `ReferenceEquals()` make the intent explicit.

KEY FACTS

- Comparison results are always `bool`
- `<` / `>` aren't defined for `string` — use `string.Compare`
- `NaN` compares false with everything
- Types can overload `==` (string and records do)

EQUALITY RULES

```
Console.WriteLine(5 == 5.0);            // True (int promoted)
Console.WriteLine('a' < 'b');           // True
Console.WriteLine("abc" == "ab" + "c"); // True (string contents)

class Pt { public int X; }
var p1 = new Pt { X = 1 }; var p2 = new Pt { X = 1 };
Console.WriteLine(p1 == p2);            // False (different objects)
Console.WriteLine(p1.Equals(p2));       // False (default = reference)

record RPt(int X);
Console.WriteLine(new RPt(1) == new RPt(1)); // True (value equality)

object o1 = 42, o2 = 42;
Console.WriteLine(o1 == o2);            // False (two boxes)
Console.WriteLine(o1.Equals(o2));       // True  (Int32.Equals)
Console.WriteLine(ReferenceEquals(p1, p1)); // True

Console.WriteLine(string.Compare("a", "b")); // -1
Console.WriteLine(3.CompareTo(7));           // -1
```

CODE EXPLAINED

- → `5 == 5.0` — int converted to double first
- → `string ==` — overloaded to compare text
- → Class `==` — same object?
- → `record` — compiler-generated value equality
- → `object ==` — reference compare even for boxed ints
- → `CompareTo` — <0, 0, >0 for ordering

OPERATORS

OP	MEANING	EXAMPLE
`==`	Equal	`x == 3`
`!=`	Not equal	`name != null`
`< >`	Less / greater	`age < 18`
`<= >=`	Or equal	`score >= 90`
`is`	Pattern test	`x is > 0 and < 10`

WHAT == COMPARES

TYPE	==
`int, double, enum`	Values
`string`	Contents (ordinal)
`class (default)`	References
`record / record struct`	All members
`struct (no overload)`	Not defined — use Equals
`object variables`	References

⚠ COMMON MISTAKES

- Comparing objects with `==` expecting field equality — override `Equals` or use a `record`
- Comparing boxed values via `object ==` — always false for separate boxes; use `Equals`
- Writing `if (x = 5)` — compile error in C#; use `==`

✓ TIPS & BEST PRACTICES

- Implement `IEquatable<T>` for custom value equality
- Use `is null` instead of `== null` to bypass overloaded `==`
- Use `x is >= 1 and <= 10` for clean range checks

QUICK SUMMARY

- `== / !=` — equality (type-dependent)
- `< > <= >=` — ordering
- `Equals` — value equality when overridden
- `ReferenceEquals` — identity

Logical Operators

BOOLEAN LOGIC

&& (AND) and || (OR) **short-circuit**: the right side runs only if needed, which lets you guard against nulls in one line. ! negates. The single-character &, | and ^ also work on `bool` but **always evaluate both sides**; ^ is XOR (true when exactly one side is true).

KEY FACTS

- Operands must be `bool` — no truthy integers
- `&&` binds tighter than `||`
- `bool?` uses three-valued logic with `&` / `|`
- Patterns also offer `and`, `or`, `not`

LOGIC IN PRACTICE

```
bool isAdmin = true, isActive = false;

Console.WriteLine(isAdmin && isActive);   // False
Console.WriteLine(isAdmin || isActive);   // True
Console.WriteLine(!isActive);             // True
Console.WriteLine(isAdmin ^ isActive);    // True (XOR)

// Short-circuit guards
if (user != null && user.IsActive) { }       // safe: right side skipped if null
if (list.Count == 0 || list[0] < 0) { }      // safe: [0] not read if empty

// Non-short-circuit: both sides always run
bool a = Check1() & Check2();             // both methods called

// De Morgan
bool x = !(isAdmin && isActive);          // same as !isAdmin || !isActive

// Pattern combinators
if (age is >= 13 and <= 19) Console.WriteLine("teen");
if (input is not (null or "")) Console.WriteLine("has text");
```

CODE EXPLAINED

- → `&&` — false as soon as left is false
- → `||` — true as soon as left is true
- → `^` — exclusive or
- → Null guard — the left check protects the right
- → `&` — evaluates both (rarely what you want)
- → `and` / `or` / `not` — inside `is` patterns

TRUTH TABLE

A	B	A && B	A || B	A ^ B
T	T	T	T	F
T	F	F	T	T
F	T	F	T	T
F	F	F	F	F

OPERATORS

OP	SHORT-CIRCUITS?
&&	Yes
||	Yes
!	— (unary)
& |	No — both sides run
^	No

BOOL? LOGIC

EXPR	RESULT
`null & false`	`false`
`null & true`	`null`
`null | true`	`true`
`null | false`	`null`

⚠ COMMON MISTAKES

- Using `&` instead of `&&` for null guards — `x != null & x.Ok` still evaluates `x.Ok` and throws
- Mixing `&&` and `||` without parentheses — `a || b && c` means `a || (b && c)`
- Writing `if (flag == true)` — just write `if (flag)`

✓ TIPS & BEST PRACTICES

- Put cheap or null checks on the left of `&&`
- Extract long conditions into well-named `bool` variables
- Use `is not null` for readable negated null checks

QUICK SUMMARY

- `&&` / `||` — short-circuit AND / OR
- `!` — NOT
- `^` — XOR
- `and` / `or` / `not` — pattern logic

Bitwise & Shift Operators

WORKING WITH BITS

Bitwise operators act on each bit of an integer: & keeps bits set in both, | in either, ^ in exactly one, and ~ flips all bits. Shifts move bits left (<<, multiply by 2^n) or right (>>, divide by 2^n keeping the sign). C# 11 added >>> for an **unsigned** right shift that fills with zeros.

KEY FACTS

- Work on `int`, `uint`, `long`, `ulong` (smaller types promote)
- `~x` equals `-x - 1` for signed ints
- Shift counts are masked: `1 << 33 == 1 << 1` for int
- `Convert.ToString(x, 2)` prints binary

WORKED EXAMPLE: 12 AND 10

```
int a = 0b1100;   // 12
int b = 0b1010;   // 10

Console.WriteLine(a & b);    //  8  → 1000
Console.WriteLine(a | b);    // 14  → 1110
Console.WriteLine(a ^ b);    //  6  → 0110
Console.WriteLine(~a);       // -13
Console.WriteLine(a << 2);   // 48  → 110000
Console.WriteLine(a >> 2);   //  3  → 11
Console.WriteLine(-16 >> 2); // -4  (sign kept)
Console.WriteLine(-16 >>> 2);// 1073741820 (zero-filled)

Console.WriteLine(Convert.ToString(a, 2));      // 1100
Console.WriteLine(int.PopCount(a));             // 2 (set bits)
Console.WriteLine(BitOperations.IsPow2(64));    // True

int flags = 0;
flags |= 1 << 3;                  // set bit 3
bool on = (flags & (1 << 3)) != 0; // test bit 3 → True
flags &= ~(1 << 3);               // clear bit 3
```

CODE EXPLAINED

- → `&` — AND: mask bits
- → `|` — OR: set bits
- → `^` — XOR: toggle bits
- → `~` — NOT: invert all bits
- → `>>` vs `>>>` — sign-fill vs zero-fill
- → `1 << n` — a mask with only bit n set

BITWISE OPERATORS

OP	NAME	EXAMPLE	RESULT
`&`	AND	`1100 & 1010`	`1000`
`|`	OR	`1100 | 1010`	`1110`
`^`	XOR	`1100 ^ 1010`	`0110`
`~`	NOT	`~0000…1100`	`1111…0011`
`<<`	Left shift	`0011 << 1`	`0110`
`>> / >>>`	Right shift	`1000 >> 2`	`0010`

COMMON IDIOMS

IDIOM	MEANING
`x & 1`	Odd? (1 = odd)
`x & (x - 1)`	Clear lowest set bit
`(x & (x - 1)) == 0`	Power of two (x > 0)
`x ^ y ^ y`	Back to x
`flags |= f / flags &= ~f`	Set / clear flag
`BitOperations.Log2(x)`	Index of highest set bit

⚠ COMMON MISTAKES

- Forgetting parentheses: `x & 1 == 0` parses as `x & (1 == 0)` — a compile error; write `(x & 1) == 0`
- Expecting `>>` to zero-fill negative numbers — use `>>>` or an unsigned type
- Shifting by 32+ on an `int` — the count wraps; use `long` for wider masks

✓ TIPS & BEST PRACTICES

- Prefer `[Flags]` enums over raw bit masks for readability
- Use `BitOperations` / `int.PopCount` instead of hand-written bit loops
- Write masks in binary literals: `0b_0000_1000`

QUICK SUMMARY

- `& | ^ ~` — AND, OR, XOR, NOT
- `<< >>` — shift (×2^n, ÷2^n)
- `>>>` — unsigned right shift (C# 11)
- `int.PopCount` — count set bits

Assignment & Compound Assignment

= · += -= *= /= %= · &= |= ^= · (a, b) = (b, a) swap · ref locals · chained assignment

ASSIGNMENT OPERATORS

= stores a value and is itself an **expression** that returns the assigned value. **Compound** operators like += combine an operation with assignment and include an implicit cast back to the variable's type. Tuples allow swapping and multi-assignment, and `ref` locals let you assign through an alias.

KEY FACTS

- `x op= y` is `x = (T)(x op y)` with x evaluated once
- `a = b = 0` assigns right to left
- `??=` assigns only when the left is null
- C# 14 lets types define their own `+=`

ASSIGNMENT FORMS

```
int x = 10;
x += 5;   // 15
x -= 3;   // 12
x *= 2;   // 24
x /= 5;   // 4
x %= 3;   // 1

byte b = 250;
b += 10;              // OK: implicit cast → 4 (wraps)
// b = b + 10;        // CS0266: int can't convert to byte

int a = 1, c = 2;
(a, c) = (c, a);      // swap → a = 2, c = 1
var (name, age) = ("Ada", 36);   // deconstruct

int i, j;
i = j = 0;            // chained

int[] arr = [1, 2, 3];
ref int first = ref arr[0];
first = 99;           // arr[0] is now 99
```

CODE EXPLAINED

- → `+=  -=  *=  /=  %=` — arithmetic compound
- → `<<=  |=` — shift / bitwise compound
- → `b += 10` — hidden cast back to `byte`
- → `(a, c) = (c, a)` — tuple swap, no temp
- → `ref int` — alias to an array element
- → `(line = …) != null` — assignment as expression

ALL ASSIGNMENT OPERATORS

OP	EQUIVALENT
`x += y`	`x = x + y`
`x -= y x *= y x /= y`	`x = x op y`
`x %= y`	`x = x % y`
`x &= y x |= y x ^= y`	`bitwise`
`x <<= y x >>= y x >>>= y`	`shifts`
`x ??= y`	`x = x ?? y`

STRINGS & EVENTS

CODE	EFFECT
`s += "!"`	New string assigned
`button.Click += OnClick`	Subscribe handler
`button.Click -= OnClick`	Unsubscribe
`total += item.Price`	Running sum

⚠ COMMON MISTAKES

- Expecting `b = b + 1` to compile for `byte` — arithmetic yields `int`; use `b++` or `b += 1`
- Using `x =+ 5` — that's `x = +5`, not `x += 5`
- Hiding assignments inside conditions — legal, but keep it to the `while ((x = …) != null)` idiom

✓ TIPS & BEST PRACTICES

- Use tuple swap instead of a temporary variable
- Use `??=` to initialize caches and lazy fields
- Prefer `+=` for events; never `=` (it replaces other subscribers on fields)

QUICK SUMMARY

- `=` — assign (also an expression)
- `op=` — compound with implicit cast
- `(a, b) = (b, a)` — swap
- `ref` local — assign through an alias

Null Operators

?. · ?[] · ?? · ??= · null-conditional assignment (C# 14) · is null · is not null

HANDLING NULL CONCISELY

C# has a family of operators for null: `?.` and `?[]` access a member only when the target is not null (otherwise the result is `null`), `??` supplies a fallback, and `??=` assigns only when the variable is null. **C# 14** adds **null-conditional assignment**: `a?.B = value` assigns only when `a` isn't null.

KEY FACTS

- `?.` short-circuits the whole rest of the chain
- `?.` on a value-type member returns `T?`
- `x ?? throw new …` validates in one line
- `is null` ignores overloaded `==`

NULL OPERATORS

```
Customer? c = GetCustomer();

string? city = c?.Address?.City;     // null if c or Address is null
int? len = c?.Name.Length;           // int? (null-propagating)
char? first = c?.Name?[0];           // ?[] for indexers
c?.Orders.Clear();                   // call only if not null
OnChanged?.Invoke(this, EventArgs.Empty); // safe event raise

string name = c?.Name ?? "Guest";    // fallback
int count = c?.Orders.Count ?? 0;
_cache ??= LoadCache();              // assign once

string id = input ?? throw new ArgumentNullException(nameof(input));

// C# 14: null-conditional assignment
c?.LastSeen = DateTime.UtcNow;       // skipped if c is null
c?.Points += 10;                     // compound OK; ++ / -- not allowed

if (c is null) return;               // pattern null check
if (c is not null) Console.WriteLine(c.Name);
```

CODE EXPLAINED

- → `?.` — member access if not null
- → `?[]` — index if not null
- → `?.Invoke` — thread-safe event raising
- → `??` — value when left is null
- → `??=` — assign only if null
- → `c?.X = v` — right side not evaluated when `c` is null

OPERATORS

OP	NAME	C#
`?.`	Null-conditional member	6
`?[]`	Null-conditional index	6
`??`	Null-coalescing	2
`??=`	Null-coalescing assignment	8
`!`	Null-forgiving (no runtime effect)	8
`a?.b = x`	Null-conditional assignment	14

RESULTS WHEN C IS NULL

EXPRESSION	RESULT
`c?.Name`	null
`c?.Name.Length`	null (int?)
`c?.Name ?? "x"`	"x"
`c?.Save()`	not called
`c!.Name`	NullReferenceException

⚠ COMMON MISTAKES

- Expecting `c?.Orders.Count == 0` to be true when `c` is null — it compares `null == 0`, which is false
- Using `?.` everywhere — it hides bugs where a value should never be null
- Mixing `??` with other operators without parentheses — `??` has very low precedence

✓ TIPS & BEST PRACTICES

- Validate at the boundary with `?? throw`, then trust non-null inside
- Use `??=` for lazy initialization
- Prefer `is null` / `is not null` over `== null`

QUICK SUMMARY

- `?.` / `?[]` — skip when null
- `??` — fallback value
- `??=` — assign if null
- `a?.b = x` — conditional assignment (C# 14)

Operator Overloading

operator + · operator == · implicit / explicit operator · checked operators · user-defined compound assignment (C# 14) · true / false

CUSTOM OPERATORS

Classes and structs can define **operators** as `public static` methods with the `operator` keyword, so a `Money` or `Vector` type can be added with +. You can also define **conversion operators** (`implicit` / `explicit`). **C# 14** adds user-defined **compound assignment** (+=) and ++/−− as instance methods that mutate in place.

KEY FACTS

- `==` and `!=` must be defined in pairs (also `<` / `>`)
- Overload `==` → also override `Equals` and `GetHashCode`
- `implicit` conversions must never throw or lose data
- `&&` / `||` can't be overloaded directly

A MONEY TYPE

```
public readonly record struct Money(decimal Amount, string Currency)
{
    public static Money operator +(Money a, Money b)
    {
        if (a.Currency != b.Currency)
            throw new InvalidOperationException("Mixed");
        return new(a.Amount + b.Amount, a.Currency);
    }
    public static Money operator *(Money m, decimal f) =>
        m with { Amount = m.Amount * f };
    public static bool operator >(Money a, Money b) => a.Amount > b.Amount;
    public static bool operator <(Money a, Money b) => a.Amount < b.Amount;

    public static implicit operator decimal(Money m) => m.Amount;   // safe
    public static explicit operator Money(decimal d) => new(d, "USD");
}

var total = new Money(10, "USD") + new Money(5, "USD"); // 15 USD
decimal raw = total;            // implicit → 15
var m = (Money)3.5m;            // explicit; == comes from record
```

CODE EXPLAINED

- → `operator +` — static, takes both operands
- → `>` requires a matching `<`
- → `implicit` — no cast needed at the call site
- → `explicit` — caller must write `(Money)`
- → Records generate `==` / `!=` for you

OVERLOADABLE OPERATORS

CATEGORY	OPERATORS
Unary	+ - ! ~ ++ -- true false
Binary	+ - * / % & | ^ << >> >>>
Comparison (pairs)	== != < > <= >=
Conversion	implicit, explicit
Not overloadable	&& || = . ?: ?? new is

C# 14 COMPOUND ASSIGNMENT

```
public class Counter
{
    public int Value { get; private set; }
    // instance, void, mutates in place
    public void operator +=(int n) => Value += n;
    public void operator ++() => Value++;
}
var c = new Counter(); c += 5; c++;  // Value = 6
```

⚠ COMMON MISTAKES

- Overloading `==` but not `Equals` / `GetHashCode` — collections and LINQ behave inconsistently
- Implicit conversions that lose precision or throw — make those `explicit`
- Overloading operators with surprising meanings (`+` that removes) — keep math-like semantics

✓ TIPS & BEST PRACTICES

- Use `record struct` to get equality for free
- Implement generic math interfaces (`IAdditionOperators<,,>`) for reusable math
- Add `operator checked` variants for types that can overflow

QUICK SUMMARY

- `public static T operator +(T a, T b)` — binary operator
- `implicit` / `explicit operator` — conversions
- `==` with `!=`, `<` with `>` — pairs
- `void operator +=` — C# 14 in-place

if / else if / else

BRANCHING WITH IF

`if` runs a block when its condition is `true`; `else if` adds more conditions checked in order; `else` handles everything left. The condition must be a `bool` expression in parentheses. Braces are optional for single statements but recommended. **Guard clauses** — early `return`s — keep nesting shallow.

KEY FACTS

- Only the first matching branch runs
- No implicit truthiness: `if (count)` is an error
- `is` patterns can test and declare in the condition
- A variable declared in a pattern is in scope after a guard

IF / ELSE IF / ELSE

```
int score = 82;

if (score >= 90)
{
    Console.WriteLine("A");
}
else if (score >= 80)
{
    Console.WriteLine("B");              // ← runs
}
else Console.WriteLine("Below B");       // braces optional

// Guard clauses instead of deep nesting
static decimal Price(Order? order)
{
    if (order is null) return 0;              // early exit
    if (order.Items.Count == 0) return 0;
    return order.Items.Sum(i => i.Price);
}

if (input is string { Length: > 0 } text)     // test + bind
    Console.WriteLine(text.ToUpper());
```

CODE EXPLAINED

- → `else if` — checked only when earlier tests fail
- → `else` — fallback, no condition
- → Guard clause — reject bad input first, then the happy path
- → `is string { Length: > 0 } text` — type + property test + variable
- → Braces — safer when adding lines later

CONDITION FORMS

CONDITION	MEANING
`if (x > 0)`	Comparison
`if (ok && ready)`	Combined logic
`if (obj is null)`	Null check
`if (obj is Dog d)`	Type test + variable
`if (n is >= 1 and <= 9)`	Range pattern

ALTERNATIVES

INSTEAD OF…	CONSIDER
`if/else returning a value`	?: ternary
`Long else-if on one value`	switch expression
`Nested ifs`	Guard clauses
`if (x == null) x = y`	x ??= y

⚠ COMMON MISTAKES

- Adding a second line under a brace-less `if` — only the first line is conditional
- Ordering `else if` from loosest to strictest — `>= 80` before `>= 90` makes the A branch unreachable
- Writing `if (x = 5)` — compile error in C#; use `==`

✓ TIPS & BEST PRACTICES

- Always use braces except for one-line guard returns
- Return early to keep the main logic at the left margin
- Switch to a `switch` expression when every branch picks a value

QUICK SUMMARY

- `if` / `else if` / `else` — ordered branches
- Condition must be `bool`
- Guard clauses — early returns
- `is` patterns — test and bind in one step

Ternary & Conditional Expressions

?: · nested ternary · target-typed conditional · ref conditional · ?? as fallback

EXPRESSIONS THAT CHOOSE

The **conditional operator** `condition ? a : b` evaluates to `a` when true and `b` when false. Unlike `if`, it is an **expression**, so it can initialize variables, be returned, or be passed as an argument. Both branches must convert to a common type, which C# 9 can take from the target.

KEY FACTS

- Only the chosen branch is evaluated
- Right-associative: `a ? b : c ? d : e`
- `??` is a shorter choice for null fallbacks
- `ref` ternaries choose between variables

TERNARY FORMS

```
int age = 20;
string group = age >= 18 ? "adult" : "minor";          // "adult"
Console.WriteLine($"{n} item{(n == 1 ? "" : "s")}");  // pluralize

// Nested — keep it short or use a switch expression
string size = n < 10 ? "small" : n < 100 ? "medium" : "large";

// Target-typed (C# 9): the declared type unifies both sides
int? maybe = age > 0 ? age : null;
IShape s = useCircle ? new Circle() : new Square();

decimal Total(bool vip, decimal amt) => vip ? amt * 0.9m : amt;

string name = user?.Name ?? "Guest";      // null fallback

int a = 1, b = 2;
ref int bigger = ref (a > b ? ref a : ref b);
bigger = 99;                              // b is now 99
```

CODE EXPLAINED

- → `cond ? x : y` — value, not a statement
- → Parenthesize inside interpolation holes
- → Chained ternary — reads top to bottom
- → Target typing — `int?` lets `age : null` compile
- → Expression-bodied member — ternary as the body
- → `ref` ternary — choose which variable to alias

IF VS ?: VS ??

NEED	USE	EXAMPLE
Run statements	if / else	`if (ok) Save();`
Pick a value	?:	`ok ? 1 : 0`
Value or default when null	??	`name ?? "-"`
Pick among many values	switch expr	`x switch { … }`

TYPE RULES

EXPRESSION	TYPE
`b ? 1 : 2.5`	`double`
`b ? 1 : null (no target)`	`error before C# 9`
`int? r = b ? 1 : null`	`int?`
`b ? "a" : 'c'`	`error — no common type`

⚠ COMMON MISTAKES

- Nesting ternaries three deep — unreadable; use a `switch` expression
- Forgetting parentheses in interpolation — `$"{a ? b : c}"` treats `:` as a format specifier
- Using `?:` only for side effects (`ok ? Save() : Log()`) — use `if`

✓ TIPS & BEST PRACTICES

- Keep each branch short and side-effect free
- Put each part on its own line for longer ternaries
- Use `??` instead of `x != null ? x : y`

QUICK SUMMARY

- `c ? a : b` — conditional expression
- Only one branch is evaluated
- Target typing (C# 9) — resolves mixed types
- `ref (c ? ref a : ref b)` — conditional alias

switch Statement

THE SWITCH STATEMENT

A `switch` statement compares one value against `case` labels and runs the matching section. C# has **no implicit fall-through**: each non-empty section must end with `break`, `return`, `throw`, `continue` or `goto case`. Cases can use **patterns** and `when` guards, not just constants.

KEY FACTS

- Works on any type with patterns (C# 7+)
- Stack empty labels to share one section
- `default` can appear anywhere; runs last
- Use a switch **expression** when you just need a value

SWITCH STATEMENT

```
switch (command)
{
    case "start":
    case "run":                          // stacked labels
        Start();
        break;
    case "stop":
        Stop(); break;
    case "restart":
        Stop();
        goto case "start";               // explicit jump
    default:
        Console.WriteLine($"Unknown: {command}"); break;
}

switch (shape)
{
    case Circle c when c.Radius > 10:    // type pattern + guard
        Console.WriteLine("Big circle"); break;
    case Circle c:
        Console.WriteLine($"Circle r={c.Radius}"); break;
}
```

CODE EXPLAINED

- → Stacked `case` labels — share one block
- → `break` — required at the end of each section
- → `goto case` — deliberate fall-through
- → `default` — no other case matched
- → `case Circle c when …` — pattern + extra condition
- → Order matters — specific cases before general ones

CASE LABEL KINDS

LABEL	MATCHES
`case 3:`	Constant
`case "a" or "b":`	Either constant
`case > 100:`	Relational
`case int n:`	Type, binds n
`case Point { X: 0 }:`	Property pattern
`case var x when x.Ok:`	Any value + guard

SECTION EXITS

STATEMENT	EFFECT
`break`	Leave the switch
`return`	Leave the method
`throw`	Raise an exception
`goto case X`	Jump to another case
`goto default`	Jump to default
`continue`	Next loop iteration (in a loop)

⚠ COMMON MISTAKES

- Leaving out `break` — error CS0163: control cannot fall through to another case
- Putting a general pattern before a specific one — the compiler flags the later case as unreachable
- Using `break` inside a switch to exit a surrounding loop — it only exits the switch

✓ TIPS & BEST PRACTICES

- Prefer switch **expressions** when each case produces a value
- Combine constants with `or` instead of stacking many labels
- Throw in `default` for enums so new members don't slip through

QUICK SUMMARY

- `switch` — multi-way branch on one value
- No fall-through — end each section
- `case T x when …` — patterns with guards
- `goto case` — explicit jump

switch Expressions

SWITCH AS AN EXPRESSION

A **switch expression** (C# 8) maps an input to a value using `pattern => result` arms separated by commas. It's shorter than a switch statement, always produces a value, and the compiler **warns when arms aren't exhaustive**. The discard _ catches everything else.

KEY FACTS

- Input comes **before** `switch`
- Arms are tried top to bottom
- Unmatched input throws `SwitchExpressionException`
- Arms can `throw` instead of returning

SWITCH EXPRESSIONS

```
string Grade(int score) => score switch
{
    >= 90 => "A",
    >= 80 => "B",
    >= 70 => "C",
    < 0 or > 100 => throw new ArgumentOutOfRangeException(nameof(score)),
    _ => "F"
};

decimal Shipping(Order o) => o switch
{
    { Total: >= 100 }            => 0m,
    { Country: "US", Express: true } => 15m,
    { Country: "US" }            => 5m,
    _                            => 25m
};

string Winner(string a, string b) => (a, b) switch    // tuple switch
{
    ("rock", "scissors") or ("paper", "rock") or ("scissors", "paper") => "A",
    var (x, y) when x == y => "Tie",
    _ => "B"
};
```

CODE EXPLAINED

- → `value switch { … }` — expression form
- → Relational arms — `>= 90 => …`
- → `throw` arm — reject invalid input
- → Property patterns — match on object shape
- → Tuple switch — branch on several values at once
- → `when` — extra guard on an arm

STATEMENT VS EXPRESSION

	STATEMENT	EXPRESSION
Produces a value	No	Yes
Separators	case … : break;	pattern => value,
Exhaustiveness check	No	Yes (CS8509)
Multiple statements	Yes	No — one expression
Default	default:	_ =>

DIAGNOSTICS

CODE	MEANING
CS8509	Not all inputs handled
CS8510	Arm already handled by earlier arm
CS8524	Unnamed enum values not handled
SwitchExpressionException	Runtime: no arm matched

⚠ COMMON MISTAKES

- Putting `_ =>` first — every later arm becomes unreachable (CS8510)
- Ignoring CS8509 — unhandled values throw `SwitchExpressionException` at run time
- Forcing multi-step logic into an arm — call a method or use a switch statement

✓ TIPS & BEST PRACTICES

- Order arms from most to least specific
- Use tuple switches for state machines and decision tables
- End enum switches with `_ => throw new UnreachableException()`

QUICK SUMMARY

- `x switch { p => v, … }` — value-producing switch
- `_` — catch-all arm
- Tuple switch — match combinations
- Compiler checks exhaustiveness

Pattern Matching — Basics

PATTERN MATCHING ESSENTIALS

Patterns test a value's shape and can extract parts of it. They appear after `is`, in `switch` cases and in switch-expression arms. The basic patterns check **type**, **constant** values, **relational** ranges and combine with `and`, `or` and `not`.

KEY FACTS

- `is T x` declares `x` only when the match succeeds
- `is null` / `is not null` can't be fooled by `==` overloads
- Relational patterns need constant operands
- `not` binds tighter than `and`, which binds tighter than `or`

BASIC PATTERNS

```
object o = 42;

if (o is int n) Console.WriteLine(n + 1);       // type pattern → 43
if (o is 42) Console.WriteLine("answer");       // constant pattern
if (o is not null) { }                          // negated null

int t = 25;
string feel = t switch
{
    < 0            => "freezing",               // relational
    >= 0 and < 15  => "cold",                   // and
    >= 15 and < 25 => "mild",
    _              => "hot"
};

bool isVowel = c is 'a' or 'e' or 'i' or 'o' or 'u';  // or
bool isLetter = c is (>= 'a' and <= 'z') or (>= 'A' and <= 'Z');

if (o is var any) Console.WriteLine(any);       // var: always matches
```

CODE EXPLAINED

- → `is int n` — type check + typed variable
- → `is 42` — equality with a constant
- → `< 0`, `>= 15` — relational patterns
- → `and` / `or` / `not` — combine patterns
- → Parentheses — group pattern logic
- → `var x` — match anything, capture it

BASIC PATTERNS

PATTERN	EXAMPLE	C#
Declaration / type	`x is string s`	7 / 9
Constant	`x is 0` / `x is null`	7
Relational	`x is > 10`	9
Logical	`x is > 0 and < 10`	9
Negation	`x is not null`	9
var	`x is var v`	7
Discard	`_` (in switch)	8

WHERE PATTERNS WORK

CONTEXT	EXAMPLE
is expression	`if (x is int i)`
switch statement	`case > 0:`
switch expression	`> 0 => "pos"`
when guards	`case int i when i > 0:`
catch filter (no)	`use catch (E e) when …`

⚠ COMMON MISTAKES

- Using a pattern variable outside its definite-assignment scope — `n` exists only where the match is known true
- Writing `x is > a` with a variable `a` — relational patterns need constants; use `x > a`
- Expecting `is not A or B` to mean `not (A or B)` — `not` binds first; add parentheses

✓ TIPS & BEST PRACTICES

- Replace `as` + null check with `is T x`
- Use `is null` for null checks in generic or operator-overloaded code
- Combine ranges in one pattern: `is >= 1 and <= 100`

QUICK SUMMARY

- `is T x` — type pattern with variable
- `is > 0` — relational pattern
- `and` / `or` / `not` — pattern logic
- `var` / `_` — match anything

Pattern Matching — Advanced

property pattern { } · extended property a.b.c · positional (Deconstruct) · list patterns [1, .., var x] · slice pattern · recursive patterns

MATCHING STRUCTURE

Advanced patterns look **inside** values. **Property patterns** `{ Prop: pattern }` test members, **positional patterns** `(a, b)` use `Deconstruct`, and **list patterns** `[first, .., last]` (C# 11) match arrays and lists by element. Patterns nest, so one expression can check an entire object graph.

KEY FACTS

- Property patterns also check for non-null
- `{ A.B: x }` — extended property pattern (C# 10)
- `..` is the slice pattern; only one per list pattern
- List patterns need `Length` / `Count` + an indexer

PROPERTY, POSITIONAL & LIST PATTERNS

```
if (order is { Status: OrderStatus.Paid, Total: > 100 }) Ship(order);
if (person is { Address.City: "Paris" }) Greet(person); // C# 10

record Point(int X, int Y);
string Quadrant(Point p) => p switch
{
    (0, 0)          => "origin",
    ( > 0, > 0)     => "Q1",
    ( < 0, > 0)     => "Q2",
    _               => "Q3/Q4"
};

int[] nums = [1, 2, 3, 4];
Console.WriteLine(nums is [1, ..]);              // True (starts with 1)
if (nums is [var first, .., var last])
    Console.WriteLine($"{first}..{last}");       // 1..4
if (nums is [_, .. var middle, _])
    Console.WriteLine(middle.Length);            // 2 (slice)
```

CODE EXPLAINED

- → `{ Status: X, Total: > 100 }` — test several properties
- → `{ Address.City: … }` — dot into nested members
- → `(0, 0)` — positional via `Deconstruct`
- → `[1, ..]` — starts with 1, any length
- → `[var f, .., var l]` — capture first and last
- → `.. var middle` — capture the slice

LIST PATTERN EXAMPLES

PATTERN	MATCHES
`[]`	Empty
`[_]`	Exactly one element
`[1, 2]`	Exactly 1, 2
`[.., 0]`	Ends with 0
`[>= 0, ..]`	First is non-negative
`["--help" or "-h", ..]`	CLI flag first

NESTING PATTERNS

```
bool IsVipUs(Customer c) => c is
{
    Tier: Tier.Gold or Tier.Platinum,
    Address: { Country: "US" },
    Orders: [_, _, ..]          // 2+ orders
};
```

⚠ COMMON MISTAKES

- Forgetting that `{ }` rejects null — `x is { }` is a non-null test, which may surprise readers
- Using two `..` in one list pattern — only one slice is allowed
- Using list patterns on a plain `IEnumerable<T>` — they need `Length` / `Count` and an indexer; materialize with `ToList()` first

✓ TIPS & BEST PRACTICES

- Use `x is { } y` as a compact "not null, bind it"
- Use list patterns to parse command-line args
- Add `Deconstruct` to classes to enable positional patterns

QUICK SUMMARY

- `{ P: pat }` — property pattern
- `(a, b)` — positional pattern
- `[a, .., z]` — list pattern
- `.. var rest` — slice capture

for Loops

COUNTED LOOPS

A `for` loop has three parts: an **initializer** that runs once, a **condition** checked before each iteration, and an **iterator** that runs after each iteration. It's the right choice when you need the **index**, want to step by something other than 1, loop backwards, or modify the collection by position.

KEY FACTS

- The loop variable is scoped to the loop
- All three parts are optional: `for (;;)` loops forever
- Multiple variables are comma-separated
- Use `foreach` when you don't need the index

FOR LOOP PATTERNS

```
for (int i = 0; i < 5; i++)
    Console.Write(i);                    // 01234

for (int i = 10; i > 0; i -= 3)
    Console.Write($"{i} ");              // 10 7 4 1

string[] names = ["Ada", "Bob", "Cy"];
for (int i = names.Length - 1; i >= 0; i--)
    Console.Write(names[i]);             // CyBobAda (reverse)

for (int i = 0, j = 9; i < j; i++, j--)
    Console.Write($"({i},{j})");         // (0,9)(1,8)…(4,5)

for (int r = 1; r <= 3; r++)             // nested: multiplication table
{
    for (int c = 1; c <= 3; c++)
        Console.Write($"{r * c,4}");
    Console.WriteLine();
}

for (int i = list.Count - 1; i >= 0; i--)   // safe removal
    if (list[i] < 0) list.RemoveAt(i);

for (;;) { if (Done()) break; }          // infinite with exit
```

CODE EXPLAINED

- → `int i = 0` — initializer, runs once
- → `i < 5` — checked before every pass
- → `i++` / `i -= 3` — step after every pass
- → `i, j` — two variables moving toward each other
- → Nested — inner loop runs fully for each outer pass
- → Backwards removal — indexes stay valid

ANATOMY

PART	EXAMPLE	RUNS
Initializer	`int i = 0`	Once
Condition	`i < arr.Length`	Before each pass
Iterator	`i++`	After each pass
Body	`{ … }`	While condition true

CHOOSING A LOOP

NEED	LOOP
Index or custom step	for
Every item, read-only	foreach
Unknown count	while
Run at least once	do-while
Item + index	foreach (var (i, x) in xs.Index())

⚠ COMMON MISTAKES

- Off-by-one with `<=` — `for (i = 0; i <= arr.Length; i++)` reads past the end
- Removing items while counting up — elements shift and get skipped; iterate backwards
- Modifying the loop variable in the body — makes the loop hard to reason about

✓ TIPS & BEST PRACTICES

- Cache `Count` only if it's expensive; `.Length` on arrays is free
- Use `Enumerable.Range(0, n)` with LINQ for functional style
- Use `Parallel.For` only for CPU-heavy, independent iterations

QUICK SUMMARY

- `for (init; cond; step)` — counted loop
- Reverse loops — `i--` from `Length - 1`
- Multiple variables — `i++, j--`
- `for (;;)` — infinite loop

foreach Loops

ITERATING COLLECTIONS

`foreach` walks every item of anything that is **enumerable** (`IEnumerable<T>`, arrays, `List<T>`, `Dictionary`, spans, strings). It's simpler and safer than `for` because there's no index to get wrong. The loop variable is **read-only**, and changing the collection during the loop throws.

KEY FACTS

- Works with any type that has `GetEnumerator()`
- Dictionaries yield `KeyValuePair<K,V>`
- `.Index()` (.NET 9) adds the position
- `await foreach` iterates `IAsyncEnumerable<T>`

FOREACH FORMS

```
List<string> fruits = ["apple", "banana", "cherry"];
foreach (string f in fruits) Console.Write(f[0]);     // abc
foreach (var c in "hey") Console.Write(c);            // hey

var ages = new Dictionary<string, int> { ["Ada"] = 36, ["Bob"] = 25 };
foreach (var (name, age) in ages)                     // deconstruct pair
    Console.WriteLine($"{name}: {age}");

foreach (var (i, f) in fruits.Index())                // .NET 9
    Console.WriteLine($"{i}. {f}");                   // 0. apple …

int[] nums = [1, 2, 3];
foreach (ref int n in nums.AsSpan()) n *= 10;         // in-place → 10,20,30

foreach (var f in fruits)
    if (f == "banana") fruits.Remove(f);   // ✗ InvalidOperationException

fruits.RemoveAll(f => f == "banana");                 // ✓ safe alternative
```

CODE EXPLAINED

- → `foreach (T x in xs)` — every element in order
- → `var (k, v)` — deconstruct `KeyValuePair`
- → `.Index()` — `(int Index, T Item)` tuples
- → `ref` foreach over a span — modify elements
- → Modifying the list mid-loop — throws
- → `await foreach` — consume async streams

WHAT YOU CAN FOREACH

SOURCE	ITEM TYPE
`T[] / List<T>`	T
`string`	char
`Dictionary<K,V>`	KeyValuePair<K,V>
`Span<T>`	T or ref T
`IEnumerable<T> (LINQ, yield)`	T, lazily
`IAsyncEnumerable<T>`	T via await foreach

CHANGING WHILE ITERATING

GOAL	DO THIS
`Remove matching items`	list.RemoveAll(pred)
`Remove from a dictionary`	foreach (var k in d.Keys.ToList())
`Replace values`	for loop with index
`Build a new list`	LINQ Where / Select

⚠ COMMON MISTAKES

- Adding or removing items inside `foreach` — throws "Collection was modified"
- Assigning to the loop variable — it's read-only (CS1656)
- Enumerating a LINQ query twice in two `foreach` loops — runs the query twice; call `ToList()` once

✓ TIPS & BEST PRACTICES

- Prefer `foreach` unless you need the index or reverse order
- Use `.Index()` instead of a separate counter
- Iterate `dict.Keys.ToList()` when you must remove dictionary entries

QUICK SUMMARY

- `foreach (var x in xs)` — every item
- `var (k, v)` — deconstruct pairs
- `.Index()` — item with position
- `await foreach` — async streams

while & do-while

CONDITION-DRIVEN LOOPS

`while` checks its condition **before** each iteration, so the body may run zero times. `do … while` checks **after**, so the body always runs at least once — ideal for menus and input validation. Use these when you don't know the number of iterations in advance.

KEY FACTS

- `do-while` ends with a semicolon: `while (cond);`
- `while (true)` + `break` for loops with mid-body exits
- The condition is re-evaluated every iteration
- Make sure something in the body changes the condition

WHILE AND DO-WHILE

```
int n = 27, steps = 0;
while (n != 1)                          // Collatz sequence
{
    n = n % 2 == 0 ? n / 2 : 3 * n + 1;
    steps++;
}
Console.WriteLine(steps);               // 111

using var reader = new StreamReader("data.txt");
string? line;
while ((line = reader.ReadLine()) is not null)   // read until EOF
    Console.WriteLine(line);

int choice;
do Console.Write("1) Play  2) Quit: ");
while (!int.TryParse(Console.ReadLine(), out choice) || choice is not (1 or 2));

while (true)                            // exit from the middle
{
    var msg = queue.Receive();
    if (msg is null) break;
    Handle(msg);
}
```

CODE EXPLAINED

- → `while (cond)` — may run zero times
- → Loop until a sentinel (`n == 1`)
- → Assignment in condition — classic read-until-null
- → `do { } while (…);` — prompt runs at least once
- → `while (true)` + `break` — exit mid-body

WHILE VS DO-WHILE

	WHILE	DO-WHILE
Condition checked	Before body	After body
Minimum runs	0	1
Typical use	Read until done	Menus, retry, validation
Syntax ends with	}	while (cond);

COMMON LOOP SHAPES

SHAPE	EXAMPLE
Sentinel	`while (x != 0)`
Countdown	`while (retries-- > 0)`
Polling	`while (!task.IsCompleted)`
Cancellable	`while (!token.IsCancellationRequested)`

⚠ COMMON MISTAKES

- Forgetting to update the condition variable — the loop never ends
- Missing the semicolon after `do { } while (x)` — compile error
- Busy-waiting with `while (!done) { }` — burns CPU; await a task or use a signal

✓ TIPS & BEST PRACTICES

- Prefer `for` when there's a clear counter
- Add a maximum-iteration guard to loops driven by external data
- Use `CancellationToken` to stop long-running loops cleanly

QUICK SUMMARY

- `while` — test first, 0+ runs
- `do-while` — test last, 1+ runs
- `while (true)` + `break` — mid-body exit
- Always change the condition inside

Loop Control & Jumps

break · continue · labeled break / continue (C# 15 preview) · goto · return · yield break

CONTROLLING LOOP FLOW

`break` exits the innermost loop or `switch`, `continue` skips to the next iteration, and `return` leaves the whole method. To exit **nested** loops you previously needed a flag, `goto` or a helper method; **C# 15 (preview)** adds **labeled `break` and `continue`** that name the target loop.

KEY FACTS

- `break` / `continue` affect the **innermost** loop only
- Labeled jumps need .NET 11 SDK + `LangVersion` preview
- `yield break` ends an iterator method
- C# has no `pass` ; use an empty block `{ }`

BREAK, CONTINUE & LABELS

```
foreach (int n in Enumerable.Range(1, 10))
{
    if (n % 2 == 0) continue;           // skip evens
    if (n > 7) break;                   // stop the loop
    Console.Write(n);                   // 1357
}

// C# 15 preview: labeled break / continue
outer: for (int r = 0; r < rows; r++)
{
    for (int c = 0; c < cols; c++)
    {
        if (blocked[r, c]) continue outer;   // next row
        if (goal[r, c]) break outer;         // leave both loops
    }
}
```

CODE EXPLAINED

- → `continue` — jump to the next iteration
- → `break` — leave the innermost loop
- → `outer:` — label placed directly on the loop
- → `continue outer` — next iteration of the outer loop
- → `break outer` — exit both loops at once

JUMP STATEMENTS

STATEMENT	EXITS
`break`	Innermost loop / switch
`continue`	Current iteration
`break label (C# 15)`	Named loop / switch
`continue label (C# 15)`	To named loop's next pass
`return`	Whole method
`goto label`	Jump anywhere in the method
`yield break`	Iterator sequence
`throw`	Up the call stack

EXITING NESTED LOOPS

OPTION	NOTES
`Extract method + return`	Clearest; works everywhere
`Boolean flag`	Verbose; check at each level
`goto done;`	Works; frowned upon
`break outer;`	C# 15 preview; IDE0410 suggests it
`LINQ Any / FirstOrDefault`	Often removes the loops

⚠ COMMON MISTAKES

- Expecting `break` inside a `switch` inside a loop to exit the loop — it only exits the switch
- Using labeled jumps on C# 14 — they don't exist yet; use a method + `return`
- Long chains of `continue` — invert the conditions or filter the input first

✓ TIPS & BEST PRACTICES

- Use `continue` for early "skip this one" checks
- Turn nested search loops into a method that returns the result
- Replace search loops with LINQ (`Any` , `First`) when it reads better

QUICK SUMMARY

- `break` — exit innermost loop
- `continue` — next iteration
- `break outer` / `continue outer` — C# 15 preview
- `return` — exit everything

Exception Basics

HANDLING EXCEPTIONS

Code that might fail goes in a `try` block. If an exception is thrown, the runtime looks for the first matching `catch` (checked top to bottom), and a `finally` block runs **no matter what** — success, exception or `return`. Unhandled exceptions bubble up the call stack and terminate the program.

KEY FACTS

- All exceptions derive from `System.Exception`
- Order catches from **most specific** to most general
- `finally` runs even after `return` inside `try`
- `catch` without a type catches everything

TRY / CATCH / FINALLY

```
try
{
    string text = File.ReadAllText("config.json");
    int port = int.Parse(text);
    Console.WriteLine($"Port {port}");
}
catch (FileNotFoundException ex)              // most specific first
{
    Console.WriteLine($"Missing file: {ex.FileName}");
}
catch (FormatException)                       // no variable needed
{
    Console.WriteLine("Config is not a number");
}
catch (Exception ex)                          // everything else
{
    Console.WriteLine($"{ex.GetType().Name}: {ex.Message}");
    throw;                                    // rethrow, keep stack
}
finally
{
    Console.WriteLine("Done");                // always runs
}
```

CODE EXPLAINED

- → `try` — code that may throw
- → `catch (T ex)` — handles `T` and subclasses
- → Omit the variable when you don't use it
- → `catch (Exception)` — last-resort handler
- → `throw;` — rethrow after logging
- → `finally` — cleanup that must always happen

EXCEPTION MEMBERS

MEMBER	CONTAINS
`Message`	Human-readable description
`StackTrace`	Where it was thrown
`InnerException`	The original cause
`GetType().Name`	Exception type name
`Data`	Extra key/value info
`ToString()`	Type + message + stack (log this)

WHAT RUNS WHEN

SCENARIO	BLOCKS EXECUTED
`No exception`	try → finally
`Caught exception`	try (partial) → catch → finally
`Uncaught exception`	try → finally → caller
`return inside try`	try → finally → return
`Exception in catch`	finally → caller gets new one

⚠ COMMON MISTAKES

- Catching `Exception` first — later specific catches are unreachable (CS0160)
- Empty `catch { }` blocks — failures vanish silently; at least log
- Using exceptions for normal control flow — use `TryParse` / `TryGetValue` patterns

✓ TIPS & BEST PRACTICES

- Catch only what you can handle; let the rest bubble up
- Log `ex.ToString()`, not just `ex.Message`
- Prefer `using` over `finally` for disposing resources

QUICK SUMMARY

- `try` — guarded code
- `catch (T ex)` — handle specific types
- `finally` — always runs
- `throw;` — rethrow preserving the stack

Throwing Exceptions

RAISING EXCEPTIONS

`throw` raises an exception object. Throw the **most specific** built-in type that fits, include a clear message, and wrap lower-level errors as an **inner exception** when you translate them. Inside a `catch`, use bare `throw;` to rethrow without losing the original stack trace. .NET also offers one-line **guard helpers** like `ArgumentNullException.ThrowIfNull`.

KEY FACTS

- `throw;` keeps the stack; `throw ex;` resets it
- `throw` can be an **expression** (C# 7)
- Guard helpers fill in the parameter name for you
- `UnreachableException` (.NET 7) marks impossible paths

THROWING PATTERNS

```
public void Withdraw(Account acct, decimal amount)
{
    ArgumentNullException.ThrowIfNull(acct);               // .NET 6
    ArgumentOutOfRangeException.ThrowIfNegativeOrZero(amount); // .NET 8
    if (amount > acct.Balance)
        throw new InvalidOperationException(
            $"Insufficient funds: balance {acct.Balance}");
    acct.Balance -= amount;
}

string Name(string? s) => s ?? throw new ArgumentNullException(nameof(s));

try { db.Save(order); }
catch (SqlException ex)
{
    throw new OrderSaveException($"Order {order.Id} failed", ex); // wrap
}

catch (Exception ex) { Log(ex); throw; }      // ✓ keeps original stack
// catch (Exception ex) { throw ex; }         // ✗ stack starts here
```

CODE EXPLAINED

- → `ThrowIfNull` — guard with the right parameter name
- → `ThrowIfNegativeOrZero` — numeric range guard
- → `InvalidOperationException` — object in the wrong state
- → `?? throw` — throw expression
- → Wrap — new type, original as `InnerException`
- → `throw;` vs `throw ex;` — keep vs lose the trace

GUARD HELPERS (STATIC)

HELPER	ON TYPE · .NET
`ThrowIfNull(x)`	ArgumentNullException · 6
`ThrowIfNullOrEmpty(s)`	ArgumentException · 7
`ThrowIfNullOrWhiteSpace(s)`	ArgumentException · 8
`ThrowIfNegative(n)`	ArgumentOutOfRangeException · 8
`ThrowIfGreaterThan(n, max)`	ArgumentOutOfRangeException · 8
`ThrowIf(disposed, this)`	ObjectDisposedException · 7

WHICH EXCEPTION?

SITUATION	THROW
`Null argument`	ArgumentNullException
`Bad argument value`	ArgumentException / OutOfRange
`Wrong object state`	InvalidOperationException
`Not implemented yet`	NotImplementedException
`Operation not supported`	NotSupportedException
`Can't happen`	UnreachableException

⚠ COMMON MISTAKES

- `throw ex;` inside a catch — resets the stack trace; use `throw;`
- Throwing `Exception` or `SystemException` directly — too general to catch meaningfully
- Dropping the original error when wrapping — always pass it as `innerException`

✓ TIPS & BEST PRACTICES

- Validate public method arguments at the top with guard helpers
- Put the failing value in the message: `$"Id {id} not found"`
- Use `ExceptionDispatchInfo` to rethrow later with the original stack

QUICK SUMMARY

- `throw new T(msg)` — raise
- `throw;` — rethrow, keep stack
- `new T(msg, inner)` — wrap with cause
- `ThrowIfNull` & friends — guards

Exception Types & Filters

EXCEPTION TYPES AND FILTERS

The BCL defines exception types for common failures; knowing which ones you'll meet tells you what to catch. **Exception filters** — `catch (T ex) when (condition)` — let you catch only when a condition holds. A filter runs **before** the stack unwinds, so a filter that returns `false` can log without catching.

KEY FACTS

- Filters don't change the stack trace
- `AggregateException` wraps several errors (Tasks)
- `OperationCanceledException` signals cancellation, not failure
- `StackOverflowException` can't be caught

FILTERS IN PRACTICE

```
try
{
    await client.GetStringAsync(url, token);
}
catch (HttpRequestException ex) when (ex.StatusCode == NotFound)
{
    return null;                                // only 404s
}
catch (HttpRequestException ex) when (IsTransient(ex))
{
    await RetryAsync();
}
catch (OperationCanceledException) when (token.IsCancellationRequested)
{
    Console.WriteLine("Cancelled by caller");
}
catch (Exception ex) when (LogAndContinue(ex)) { }  // never catches

static bool LogAndContinue(Exception ex) { Log(ex); return false; }
```

CODE EXPLAINED

- → `when (…)` — catch only if the condition is true
- → Same type, different filters — separate handling
- → Cancellation filter — distinguish caller cancel from timeout
- → Filter returning `false` — log without handling
- → `IsTransient(ex)` — any bool method works as a filter

COMMON EXCEPTIONS

TYPE	THROWN WHEN
`NullReferenceException`	Member access on null
`ArgumentException`	Bad argument (+Null, +OutOfRange)
`InvalidOperationException`	Wrong state; First() on empty
`KeyNotFoundException`	dict[key] — key missing
`FormatException`	int.Parse("abc")
`InvalidCastException`	Bad cast or unbox

SPECIAL CASES

TYPE	NOTE
`OperationCanceledException`	Cancellation, not failure
`TaskCanceledException`	Also HttpClient timeouts
`AggregateException`	From .Wait() / .Result
`StackOverflowException`	Can't catch; process dies

⚠ COMMON MISTAKES

- Catching `NullReferenceException` — fix the null bug instead
- Using `.Result` / `.Wait()` and then catching the inner type — you get `AggregateException`; prefer `await`
- Side effects in filters that throw — an exception inside a filter is swallowed and treated as `false`

✓ TIPS & BEST PRACTICES

- Filter on `HttpRequestException.StatusCode` instead of parsing messages
- Treat `OperationCanceledException` separately from real failures
- Use a `when (Log(ex))` filter returning `false` for diagnostics

QUICK SUMMARY

- `catch (T e) when (cond)` — exception filter
- Filters run before unwinding
- `AggregateException` — many errors in one
- Know the common BCL exception types

Custom Exceptions

: Exception · standard constructors · extra properties · naming · when to create one · exception hierarchy

DEFINING YOUR OWN EXCEPTIONS

Create a custom exception when callers need to **catch a specific failure** of your domain (e.g. `OrderNotFoundException`) or when you want to carry extra data. Derive from `Exception` (or a more specific base), end the name with **`Exception`**, and provide the standard constructors so it can wrap inner exceptions.

KEY FACTS

- Derive from `Exception`, not `ApplicationException`
- `[Serializable]` isn't needed on modern .NET
- Make extra properties read-only
- A small hierarchy lets callers catch at different levels

CUSTOM EXCEPTION CLASS

```
public class AppException : Exception          // optional common base
{
    public AppException() { }
    public AppException(string msg) : base(msg) { }
    public AppException(string msg, Exception inner) : base(msg, inner) { }
}

public sealed class OrderNotFoundException : AppException
{
    public int OrderId { get; }

    public OrderNotFoundException(int orderId)
        : base($"Order {orderId} was not found.") => OrderId = orderId;
}

// Throw and catch
throw new OrderNotFoundException(42);

catch (OrderNotFoundException ex) { return NotFound(ex.OrderId); }
catch (AppException ex)           { return Problem(ex.Message); }
```

CODE EXPLAINED

- → `: Exception` — inherit message, stack, inner
- → Three standard constructors — empty, message, message + inner (add the same to leaf types)
- → `: base(message)` — pass the message up
- → `OrderId` — extra, read-only context
- → `sealed` — leaf types usually needn't be extended
- → Catch specific first, then the shared base

CREATE ONE WHEN…

SITUATION	CUSTOM?
`Callers handle this case differently`	Yes
`You need extra data (IDs, codes)`	Yes
`A built-in type already fits`	No — reuse it
`Argument validation`	No — ArgumentException
`Only to change the message`	No

GUIDELINES

DO	AVOID
`Name ends with Exception`	OrderError, OrderFailed
`Clear, complete message`	"Error"
`Preserve inner exception`	Swallowing the cause
`Read-only extra properties`	Mutable state
`Few, meaningful types`	One class per throw site

⚠ COMMON MISTAKES

- Skipping the `(message, inner)` constructor — callers can't wrap the original cause
- Inheriting from `ApplicationException` — it adds nothing; Microsoft recommends `Exception`
- Creating custom types for argument errors — `ArgumentException` already communicates that

✓ TIPS & BEST PRACTICES

- Group domain errors under one base so APIs can map them to responses
- Put identifiers in properties, not just the message, for logging and handling
- Map domain exceptions to HTTP `ProblemDetails` at the API edge

QUICK SUMMARY

- `class XException : Exception` — custom type
- Standard constructors — include `(msg, inner)`
- Extra read-only properties — context
- Base class — catch a family at once

using & IDisposable

DETERMINISTIC CLEANUP

The garbage collector frees memory, but **unmanaged resources** (files, sockets, DB connections) should be released promptly. Types holding them implement `IDisposable`. The `using` statement or declaration calls `Dispose()` automatically, even when an exception is thrown. Async resources use `IAsyncDisposable` with `await using`.

KEY FACTS

- `using` compiles to `try` / `finally` + `Dispose()`
- Declarations dispose at the end of the enclosing scope
- Multiple `using` s dispose in **reverse** order
- `Dispose()` should be safe to call more than once

USING & IDISPOSABLE

```
using (var reader = new StreamReader("in.txt"))      // statement form
    Console.WriteLine(reader.ReadLine());            // Dispose() after

using var input = File.OpenRead("a.bin");            // declaration (C# 8)
using var output = File.Create("b.bin");             // disposed first…
input.CopyTo(output);                                // …at end of scope

await using var conn = new SqlConnection(cs);        // IAsyncDisposable

public sealed class TempFile : IDisposable
{
    public string Path { get; } = System.IO.Path.GetTempFileName();
    private bool _disposed;
    public void Dispose()
    {
        if (_disposed) return;                       // idempotent
        File.Delete(Path); _disposed = true;
    }
}
```

CODE EXPLAINED

- → `using (…) { }` — dispose at the closing brace
- → `using var` — dispose at the end of the method
- → Reverse order — last created, first disposed
- → `await using` — calls `DisposeAsync()`
- → Sealed class — simple `Dispose` without the full pattern
- → `_disposed` flag — safe repeated calls

COMMON DISPOSABLES

TYPE	RELEASES
`FileStream` / `StreamReader`	File handle
`SqlConnection` / `DbContext`	DB connection
`HttpResponseMessage`	Network stream
`CancellationTokenSource`	Timer / registrations
`Timer`	Callback scheduling

FULL DISPOSE PATTERN (UNSEALED)

```
protected virtual void Dispose(bool disposing)
{
    if (_disposed) return;
    if (disposing) _stream.Dispose();  // managed
    _disposed = true;
}
public void Dispose()
    { Dispose(true); GC.SuppressFinalize(this); }
```

⚠ COMMON MISTAKES

- Forgetting to dispose — file locks and connection-pool exhaustion follow
- Disposing an object you didn't create (e.g. an injected `HttpClient`) — the owner disposes
- Using `using` on `HttpClient` per request — exhausts sockets; reuse one or use `IHttpClientFactory`

✓ TIPS & BEST PRACTICES

- Prefer `using var` declarations to cut nesting
- Seal classes when possible so a simple `Dispose()` is enough
- Guard members with `ObjectDisposedException.ThrowIf` after disposal

QUICK SUMMARY

- `IDisposable` — explicit cleanup contract
- `using` — automatic `Dispose()`
- `await using` — async cleanup
- Dispose is idempotent and reverse-ordered

Defining & Calling Methods

METHODS

A **method** is a named block of code inside a class, struct or record. It declares an **access modifier**, a **return type** (`void` when it returns nothing), a **name** in PascalCase and a parameter list. **Static** methods belong to the type; **instance** methods act on an object and can use its fields.

KEY FACTS

- Every code path of a non-void method must `return`
- `=>` creates an **expression-bodied** member
- Local functions can live inside other methods
- Methods can return tuples for multiple values

DECLARING & CALLING

```
public class Calculator
{
    private int _calls;                                  // instance state

    public int Add(int a, int b)                         // instance method
    {
        _calls++;
        return a + b;
    }

    public int Square(int x) => x * x;                   // expression-bodied
    public void Reset() => _calls = 0;                   // void
    public static double Average(int a, int b) => (a + b) / 2.0; // static

    public (int Min, int Max) Range(int[] xs) => (xs.Min(), xs.Max());
}

var calc = new Calculator();
Console.WriteLine(calc.Add(2, 3));                  // 5
Console.WriteLine(calc.Square(4));                  // 16
Console.WriteLine(Calculator.Average(3, 4));        // 3.5
var (min, max) = calc.Range([4, 1, 9]);             // 1, 9
```

CODE EXPLAINED

- → `public int Add(int a, int b)` — modifier, return type, name, parameters
- → `return` — hands back a value and exits
- → `=> expr` — one-expression body
- → `void` — no return value
- → `static` — call on the type: `Calculator.Average`
- → Tuple return — several named values

SIGNATURE PARTS

PART	EXAMPLES
Access	`public, private, internal, protected`
Modifiers	`static, virtual, override, async, abstract`
Return type	`void, int, Task<T>, (int, int)`
Name	`PascalCase verb: GetTotal, Save`
Parameters	`(int id, string? name = null)`
Generic	`T Max<T>(T a, T b)`

STATIC VS INSTANCE

	STATIC	INSTANCE
Called on	Type	Object
Access to this	No	Yes
Uses fields	Static only	All
Typical use	Helpers, factories	Behavior of the object

⚠ COMMON MISTAKES

- Missing `return` on some path — error CS0161: not all code paths return a value
- Calling an instance method without an object — CS0120; make it `static` or create an instance
- Overlong methods doing many things — split into smaller, well-named methods

✓ TIPS & BEST PRACTICES

- Name methods with verbs: `CalculateTax`, `TryParse`, `LoadAsync`
- Mark helpers that don't use instance state as `static`
- Use expression bodies for one-liners

QUICK SUMMARY

- `ReturnType Name(params) { }` — method
- `void` — no result
- `=>` — expression-bodied
- `static` — no instance needed

Parameters & Arguments

named arguments · optional / default · ref · out · in · ref readonly · out var

PASSING DATA IN AND OUT

By default arguments are passed **by value** (a copy of the value or of the reference). **Named** and **optional** arguments make calls readable. The modifiers `ref`, `out`, `in` and `ref readonly` pass **by reference**: `ref` reads and writes the caller's variable, `out` must be assigned by the method, and `in` / `ref readonly` pass large structs without copying.

KEY FACTS

- Optional parameters must come after required ones
- Default values must be compile-time constants
- `out var x` declares the variable inline
- `out _` discards a value you don't need

PARAMETER KINDS

```
void Greet(string name, string greet = "Hello", bool shout = false)
    => Console.WriteLine($"{greet}, {name}{(shout ? "!" : "")}");
Greet("Ada");                          // Hello, Ada
Greet("Ada", "Hi");                    // Hi, Ada
Greet("Ada", shout: true);             // Hello, Ada!  (named, skips one)

void Double(ref int x) => x *= 2;
int n = 5; Double(ref n);              // n = 10

bool TryDivide(int a, int b, out int result)
{
    result = b == 0 ? 0 : a / b;       // must assign
    return b != 0;
}
if (TryDivide(10, 3, out var q)) Console.WriteLine(q); // 3

double Length(in Vector3 v) => Math.Sqrt(Vector3.Dot(v, v));
```

CODE EXPLAINED

- → `greet = "Hello"` — optional with default
- → `shout: true` — named argument
- → `ref` — caller must pass `ref n`; method can change it
- → `out` — method must assign before returning
- → `out var q` — inline declaration
- → `in` — read-only reference, avoids copying big structs

PARAMETER MODIFIERS

MODIFIER	CALLER WRITES	METHOD MAY
(none)	`M(x)`	Use a copy
`ref`	`M(ref x)`	Read & write caller var
`out`	`M(out x)`	Must write; can't read first
`in`	`M(x)` or `M(in x)`	Read only
`ref readonly (C# 12)`	`M(ref x)` / `M(in x)`	Read only; needs a variable
`params`	`M(1, 2, 3)`	Receive a collection

RULES

RULE	EXAMPLE
Required before optional	`(int a, int b = 0)`
Named after positional	`M(1, b: 2)`
Defaults are constants	`= null, = 0, = "x"`
Use default for structs	`TimeSpan t = default`

⚠ COMMON MISTAKES

- Expecting a method to replace the caller's object without `ref` — only the copy of the reference changes
- Changing a public optional default — callers compiled earlier keep the old value
- Using `ref` just to return extra values — return a tuple or record instead

✓ TIPS & BEST PRACTICES

- Use named arguments for boolean flags: `Save(overwrite: true)`
- Follow the `TryX(input, out result)` pattern for operations that can fail
- Use `in` only for large structs; for small ones copying is cheaper

QUICK SUMMARY

- Optional + named — readable calls
- `ref` — read/write caller's variable
- `out` — extra output, must assign
- `in` / `ref readonly` — no-copy read-only

params & Variadic Methods

params T[] · params Span<T> · params IEnumerable<T> · zero arguments · passing an array

VARIABLE NUMBERS OF ARGUMENTS

A `params` parameter lets callers pass **zero or more** arguments as a comma-separated list, or a collection directly. Before C# 13 it had to be an array; **C# 13 params collections** also allow `Span<T>`, `ReadOnlySpan<T>`, `IEnumerable<T>`, `List<T>` and other collection types, and span versions avoid allocating.

KEY FACTS

- Only **one** `params` parameter, and it must be **last**
- Zero arguments → an empty collection, never `null`
- Passing `null` explicitly gives `null` (for arrays)
- `ReadOnlySpan<T>` overloads win when both exist

PARAMS IN ACTION

```
static int Sum(params int[] nums) => nums.Sum();

Console.WriteLine(Sum());              // 0  (empty array)
Console.WriteLine(Sum(1, 2, 3));       // 6
Console.WriteLine(Sum([4, 5]));        // 9  (pass a collection)

// C# 13: params collections
static int Max(params ReadOnlySpan<int> xs)   // no heap allocation
{
    int m = int.MinValue;
    foreach (var x in xs) m = Math.Max(m, x);
    return m;
}
static void LogAll(string prefix, params IEnumerable<string> items)
    => Console.WriteLine($"{prefix}: {string.Join(", ", items)}");

LogAll("Tags", "c#", "dotnet");          // Tags: c#, dotnet
```

CODE EXPLAINED

- → `params int[]` — classic variadic array
- → `Sum()` — empty array, safe to loop
- → `Sum([4, 5])` — pass an existing collection
- → `params ReadOnlySpan<int>` — stack-allocated args
- → `params IEnumerable<string>` — accept LINQ results
- → Regular parameters come before `params`

ALLOWED PARAMS TYPES (C# 13)

TYPE	ALLOCATES?
`T[]`	Yes
`Span<T>` / `ReadOnlySpan<T>`	No (stack)
`IEnumerable<T>` / `IReadOnlyList<T>`	Yes
`List<T>` / `ICollection<T>`	Yes
`Custom with [CollectionBuilder]`	Depends

BCL EXAMPLES

METHOD	CALL
`string.Join`	`string.Join(", ", a, b, c)`
`Path.Combine`	`Path.Combine("a", "b", "c")`
`Console.WriteLine`	`Console.WriteLine("{0}{1}", x, y)`
`string.Concat`	`string.Concat("a", "b", "c", "d")`

⚠ COMMON MISTAKES

- Putting `params` before other parameters — CS0231; it must be last
- Calling `M(null)` on a `params string[]` and expecting an array with one null — you get a `null` array
- Mutating the `params` array and expecting the caller to see changes — callers usually pass literals

✓ TIPS & BEST PRACTICES

- Use `params ReadOnlySpan<T>` for hot utility methods
- Add a non-params overload for the common 1–2 argument case if allocation matters
- Accept `params IEnumerable<T>` when callers often pass queries

QUICK SUMMARY

- `params T[] xs` — zero or more args
- Must be the last parameter
- C# 13 — spans and collections allowed
- Callers can pass a collection directly

Method Overloading

SAME NAME, DIFFERENT SIGNATURES

Overloading means several methods share a name but differ in the **number, types or ref-kinds** of their parameters. The compiler picks the best match at compile time using **overload resolution**: exact matches beat implicit conversions, and if two candidates are equally good the call is **ambiguous**.

KEY FACTS

- Return type alone can't distinguish overloads
- Parameter names don't count either
- `ref` / `out` / `in` do count (but not `ref` vs `out`)
- C# 13 `[OverloadResolutionPriority]` lets library authors steer resolution

OVERLOADS & RESOLUTION

```
static void Print(int x)     => Console.WriteLine($"int {x}");
static void Print(long x)    => Console.WriteLine($"long {x}");
static void Print(double x)  => Console.WriteLine($"double {x}");
static void Print(string s)  => Console.WriteLine($"string {s}");
static void Print(object o)  => Console.WriteLine($"object {o}");

Print(5);          // int 5        (exact)
Print(5L);         // long 5
Print((short)5);   // int 5        (short → int is the best widening)
Print(5f);         // double 5     (float → double)
Print("hi");       // string hi
Print(DateTime.Now); // object …   (only match)

static int Area(int side) => side * side;
static int Area(int w, int h) => w * h;     // by count

// static double Area(int s) => …;   // ✗ CS0111: return type only

static void F(int a, long b) { }
static void F(long a, int b) { }
// F(1, 2);                              // ✗ CS0121: ambiguous
```

CODE EXPLAINED

→ Exact match — `int` overload for `5`

→ Best widening — `short` prefers `int` over `long`

→ `object` — fallback that accepts anything

→ Different parameter counts — always distinct

→ Return type only — not an overload

→ Two equally good candidates — ambiguity error

VALID OVERLOAD DIFFERENCES

DIFFER BY	EXAMPLE
Parameter count	`Area(int)` / `Area(int, int)`
Parameter types	`Parse(string)` / `Parse(char[])`
Ref-kind	`M(int)` / `M(ref int)`
Generic arity	`M()` / `M<T>()`
NOT return type	✗
NOT param names	✗

OVERLOADS VS OPTIONAL PARAMS

USE	WHEN
Overloads	Different types or behavior
Optional params	Same logic, sensible defaults
Overloads	Public library APIs (versioning-safe)
`params`	Variable count of same type

⚠ COMMON MISTAKES

- Overloads that do different things — same name should mean same intent
- Mixing overloads and optional parameters so calls become ambiguous
- Passing `null` to overloads with two reference types — CS0121; cast: `Print((string?)null)`

✓ TIPS & BEST PRACTICES

- Have simpler overloads call the most complete one
- Add a `ReadOnlySpan<T>` overload for performance-sensitive callers
- Prefer distinct names (`FromSeconds`, `FromMinutes`) when types alone are confusing

QUICK SUMMARY

- Overloads — same name, different parameters
- Resolution — exact > widening > object
- Return type doesn't count
- CS0121 — ambiguous call

Local Functions & Scope

LOCAL FUNCTIONS AND SCOPE

A **local function** is a method declared inside another method; it's visible only there and keeps helper logic close to where it's used. **Scope** in C# is the block `{ }` where a name is declared: locals disappear at the end of their block. A `static` local function can't capture the outer method's variables, which prevents accidental dependencies.

KEY FACTS

- Local functions can be called before their declaration
- Non-static local functions can capture outer locals
- A local can't shadow another local in an enclosing scope
- Fields can be shadowed by locals; use `this.x` to reach the field

LOCAL FUNCTIONS

```
int SumOfSquares(int[] nums)
{
    int total = 0;
    foreach (var n in nums) total += Square(n);
    return total;

    static int Square(int x) => x * x;      // static: no captures
}

IEnumerable<int> Evens(int max)
{
    ArgumentOutOfRangeException.ThrowIfNegative(max); // runs eagerly
    return Iterate();

    IEnumerable<int> Iterate()              // captures max
    {
        for (int i = 0; i <= max; i += 2) yield return i;
    }
}

if (true) { int inner = 1; }
// Console.WriteLine(inner);                // ✗ CS0103: out of scope
```

CODE EXPLAINED

- → Local function — declared after `return`, still callable
- → `static` — compiler error if it touches outer variables
- → Iterator helper — validates eagerly, yields lazily
- → Captures — `Iterate` reads `max` from the outer method
- → Block scope — `inner` exists only inside its braces

SCOPE LEVELS

SCOPE	LIFETIME / VISIBILITY
`Block { }`	Until the closing brace
`Method`	Parameters + locals of the method
`Loop variable`	Inside the loop only
`Pattern variable`	Where the match is definitely true

LOCAL FUNCTION VS LAMBDA

	LOCAL FN	LAMBDA
`Recursion`	Easy	Awkward
`Generic`	Yes	No
`yield / iterators`	Yes	No
`Allocation`	Usually none	Delegate (+ closure)

⚠ COMMON MISTAKES

- Declaring the same name in a nested block — CS0136: name conflicts with an outer local
- Using a local function where a private method would be reused elsewhere — move it out
- Letting local functions capture lots of state — make them `static` and pass parameters

✓ TIPS & BEST PRACTICES

- Use the eager-validation + local iterator pattern for `yield` methods
- Prefer `static` local functions by default
- Keep variables in the narrowest block that needs them

QUICK SUMMARY

- Local function — helper inside a method
- `static` local — no captures
- Block scope — names end at `}`
- No shadowing of enclosing locals

Delegates

METHODS AS VALUES

A **delegate** is a type-safe reference to a method: a variable you can store, pass and invoke. The BCL provides generic delegates for almost every need — **`Func<…, TResult>`** returns a value, **`Action<…>`** returns `void`, **`Predicate<T>`** returns `bool`. Delegates can point to static methods, instance methods or lambdas.

KEY FACTS

- `Func<int, string>` = takes int, returns string (last type is the result)
- Method group: `list.ForEach(Console.WriteLine)`
- Delegates are **multicast**: `+=` adds targets
- Invoking a null delegate throws — use `?.Invoke`

DELEGATES

```
delegate int MathOp(int a, int b);          // custom delegate type

static int Add(int a, int b) => a + b;
MathOp op = Add;                            // method group
Console.WriteLine(op(2, 3));                // 5

Func<int, int, int> mul = (a, b) => a * b;
Func<string, int> len = s => s.Length;
Action<string> log = msg => Console.WriteLine($"[log] {msg}");
Predicate<int> isEven = n => n % 2 == 0;

Console.WriteLine(mul(4, 5));               // 20
log("started");                             // [log] started
Console.WriteLine(isEven(7));               // False

Action notify = () => Console.Write("A");
notify += () => Console.Write("B");         // multicast
notify();                                   // AB
Func<int>? maybe = null;
int? r = maybe?.Invoke();                   // null, no exception
```

CODE EXPLAINED

- → `delegate` — declare your own signature (rarely needed)
- → Method group — assign a method by name
- → `Func<T1, T2, TResult>` — last type parameter is the return
- → `Action<T>` — returns void
- → `+=` — chain several methods
- → `?.Invoke()` — safe call when possibly null

BUILT-IN DELEGATES

TYPE	SIGNATURE
`Action`	`() → void`
`Action<T1, T2>`	`(T1, T2) → void`
`Func<TResult>`	`() → TResult`
`Func<T, TResult>`	`(T) → TResult`
`Predicate<T>`	`(T) → bool`
`Comparison<T>`	`(T, T) → int`

WHERE YOU MEET THEM

API	DELEGATE
`list.Where(x => …)`	Func<T, bool>
`list.ForEach(…)`	Action<T>
`list.Sort(…)`	Comparison<T>
`Task.Run(…)`	Action / Func<Task>
`button.Click += …`	EventHandler

⚠ COMMON MISTAKES

- Declaring custom delegate types when `Func` / `Action` already fit
- Calling a delegate field without a null check — `NullReferenceException`
- Expecting a multicast `Func` to return all results — you only get the **last** target's value

✓ TIPS & BEST PRACTICES

- Accept `Func<…>` parameters to make methods configurable
- Pass method groups (`Select(int.Parse)`) when signatures already match
- Use `Comparison<T>` lambdas for one-off sorts

QUICK SUMMARY

- Delegate — reference to a method
- `Func` — returns a value; `Action` — void
- `Predicate<T>` — bool test
- `+=` — multicast

Lambda Expressions

x => · statement lambdas · natural type · default parameters · static lambdas · parameter modifiers (C# 14) · _ discards

ANONYMOUS FUNCTIONS

A **lambda** `(params) => body` creates an anonymous function that converts to a delegate or expression tree. **Expression lambdas** return a single expression; **statement lambdas** use braces. Since C# 10 lambdas have a **natural type**, so `var` works; C# 12 added default parameter values and C# 14 allows `ref`/`out`/`in` modifiers without writing parameter types.

KEY FACTS

- Parentheses optional for one untyped parameter: `x => x * 2`
- Parameter types are usually inferred
- `static` lambdas can't capture locals
- Use `_` for unused parameters

LAMBDA FORMS

```
Func<int, int> twice = x => x * 2;                  // expression lambda
Func<int, int, int> add = (a, b) => a + b;
Action hi = () => Console.WriteLine("hi");          // no parameters

Func<string, bool> isValid = s =>                   // statement lambda
{
    if (string.IsNullOrWhiteSpace(s)) return false;
    return s.Length <= 20;
};

var parse = (string s) => int.Parse(s);             // natural type (C# 10)
var pick = object (bool b) => b ? 1 : "one";        // explicit return type
var inc = (int x, int by = 1) => x + by;            // default param (C# 12)
Console.WriteLine(inc(5));                          // 6

Func<int, int> sq = static x => x * x;              // no captures allowed
EventHandler h = (_, _) => Console.WriteLine("clicked"); // discards

delegate bool TryParser<T>(string s, out T value);
TryParser<int> tp = (s, out v) => int.TryParse(s, out v); // C# 14
```

CODE EXPLAINED

- → `x => x * 2` — single expression is the return value
- → `{ … return …; }` — statement lambda
- → `var` + typed params — compiler infers `Func<string,int>`
- → `object (bool b) =>` — declare the return type
- → `by = 1` — default parameter value
- → `(s, out v)` — modifiers without types (C# 14)

SYNTAX VARIANTS

FORM	EXAMPLE
One param	`x => x + 1`
Many params	`(x, y) => x + y`
Typed params	`(int x, int y) => x + y`
No params	`() => DateTime.Now`
Async	`async () => await LoadAsync()`
Attribute	`[Pure] (int x) => x`

WITH LINQ & COLLECTIONS

CALL	RESULT
`nums.Where(n => n > 2)`	`filter`
`nums.Select(n => n * n)`	`map`
`names.OrderBy(n => n.Length)`	`sort key`
`list.RemoveAll(x => x < 0)`	`predicate`
`dict.GetValueOrDefault(k)`	`(no lambda)`

⚠ COMMON MISTAKES

- Using `async void` lambdas for `Action` parameters — exceptions crash the process; prefer `Func<Task>`
- Big statement lambdas inside LINQ — extract a named method
- Capturing a loop variable in a `for` loop — see closures; copy it to a local first

✓ TIPS & BEST PRACTICES

- Mark lambdas `static` in hot paths to avoid closure allocations
- Use method groups (`Select(int.Parse)`) when no extra logic is needed
- Name complex predicates: `Func<Order, bool> isLate = …`

QUICK SUMMARY

- `x => expr` — expression lambda
- `x => { … }` — statement lambda
- Natural type — `var f = (int x) => …`
- Defaults (C# 12), modifiers (C# 14)

Closures & Captured Variables

captured variables · lifetime extension · loop capture gotcha · static lambda prevents capture · closure allocations

CAPTURING VARIABLES

When a lambda or local function uses a variable from its surrounding method, it **captures the variable itself**, not a copy of its value. The compiler moves captured variables into a hidden class so they **outlive** the method. Later changes are visible to the lambda, which is powerful — and the source of the classic `for`-loop bug.

KEY FACTS

- Captured = shared variable, not a snapshot
- `foreach` creates a fresh variable each iteration (C# 5+)
- `for` uses **one** variable for all iterations
- Captures allocate a closure object

CLOSURES

```
int counter = 0;
Action increment = () => counter++;       // captures counter
increment(); increment();
Console.WriteLine(counter);               // 2

Func<int> MakeCounter()
{
    int count = 0;                        // lives on after return
    return () => ++count;
}
var next = MakeCounter();
Console.WriteLine($"{next()} {next()}");   // 1 2
var actions = new List<Action>();
for (int i = 0; i < 3; i++)
    actions.Add(() => Console.Write(i));  // all share one i
actions.ForEach(a => a());                // 333  ✗
for (int i = 0; i < 3; i++)
{
    int copy = i;                         // new variable each pass
    actions.Add(() => Console.Write(copy));
}                                         // 012  ✓

Func<int, int> sq = static x => x * x;    // static: can't capture
```

CODE EXPLAINED

→ Capture — lambda shares `counter` with the method

→ `MakeCounter` — local outlives the call

→ `for` capture — every lambda sees the final `i` (3)

→ `int copy = i` — fresh variable per iteration

→ `static` lambda — compile error if it captures

WHAT GETS CAPTURED

VARIABLE	FRESH PER ITERATION?
`foreach (var x in xs)`	Yes
`for (int i = …)`	No — shared
`Local declared inside loop body`	Yes
`Parameters / outer locals`	One shared instance
`this (instance members)`	Captures the whole object

COSTS & RISKS

ISSUE	EFFECT
`Closure allocation`	Heap object per capture scope
`Lifetime extension`	Objects kept alive by long-lived lambdas
`Event handlers capturing this`	Memory leaks if not removed
`Shared mutable state`	Race conditions across threads

⚠ COMMON MISTAKES

- Capturing a `for` loop counter and running the lambdas later — they all see the last value
- Capturing large objects in long-lived callbacks — they can't be garbage-collected
- Mutating captured variables from parallel tasks — data races; use `Interlocked` or locks

✓ TIPS & BEST PRACTICES

- Use `static` lambdas to guarantee no hidden captures
- Copy to a local inside the loop when deferring work
- Pass state explicitly (e.g. `Task.Factory.StartNew(state => …, state)`) in hot code

QUICK SUMMARY

- Closures capture variables, not values
- Captured locals outlive the method
- `for` counter is shared; copy it
- `static` — forbid captures

Events

PUBLISH AND SUBSCRIBE

An **event** lets a class notify subscribers that something happened, without knowing who they are. It's declared with the `event` keyword on a delegate type — usually `EventHandler` or `EventHandler<TEventArgs>`. Outside code can only **subscribe** (+=) and **unsubscribe** (−=); only the declaring class can raise the event.

KEY FACTS

- Standard signature: `(object? sender, TEventArgs e)`
- Raise with `Handler?.Invoke(this, args)`
- Subscribers keep the **publisher's** reference to them alive
- Name events with verbs: `Changed`, `Completed`, `Closing`

DECLARING & USING EVENTS

```
public sealed class PriceChangedEventArgs(decimal old, decimal @new) : EventArgs
{ public decimal Old => old; public decimal New => @new; }

public class Stock
{
    private decimal _price;
    public event EventHandler<PriceChangedEventArgs>? PriceChanged;

    public void Update(decimal price)
    {
        var old = _price; _price = price;
        PriceChanged?.Invoke(this, new(old, price));   // raise
    }
}

var msft = new Stock();
void OnChange(object? s, PriceChangedEventArgs e) =>
    Console.WriteLine($"{e.Old} → {e.New}");
msft.PriceChanged += OnChange;      // subscribe
msft.Update(410);                   // 0 → 410
msft.PriceChanged -= OnChange;      // unsubscribe
```

CODE EXPLAINED

- → `EventArgs` subclass — carries event data
- → `event EventHandler<T>?` — nullable: no subscribers yet
- → `?.Invoke(this, …)` — raise safely
- → `+=` / `-=` — subscribe / unsubscribe
- → Named handler — needed to unsubscribe later

EVENT VS PLAIN DELEGATE FIELD

CAPABILITY (OUTSIDE)	EVENT	DELEGATE
`+= / -=`	✔	✔
`= (replace all)`	✖	✔
`Invoke`	✖	✔
`Set to null`	✖	✔

FRAMEWORK EVENTS

EVENT	WHERE
`PropertyChanged`	INotifyPropertyChanged (MVVM)
`Click`	WinForms / WPF / MAUI buttons
`AppDomain.UnhandledException`	Last-chance errors
`Console.CancelKeyPress`	Ctrl+C
`FileSystemWatcher.Changed`	File changes

⚠ COMMON MISTAKES

- Subscribing with a lambda and later trying `-=` with another lambda — it's a different delegate
- Forgetting to unsubscribe from long-lived publishers — the subscriber can't be garbage-collected
- Raising events without a null check — throws when nobody subscribed

✓ TIPS & BEST PRACTICES

- Follow `(object? sender, TEventArgs e)` so handlers are reusable
- Unsubscribe in `Dispose()` for objects with shorter lives than the publisher
- Consider `IObservable<T>` or channels for complex streams of events

QUICK SUMMARY

- `event EventHandler<T>` — declare
- `?.Invoke(this, args)` — raise
- `+=` / `-=` — subscribe / unsubscribe
- Unsubscribe to avoid leaks

Recursion

METHODS THAT CALL THEMSELVES

A **recursive** method solves a problem by calling itself on a smaller input until it reaches a **base case** that returns directly. Recursion fits naturally with trees, nested data and divide-and-conquer algorithms. Each call uses stack space, and C# **does not guarantee tail-call optimization**, so very deep recursion can crash the process.

KEY FACTS

- Every recursive method needs a reachable base case
- Default stack ≈ 1 MB (main thread)
- `StackOverflowException` can't be caught — the process ends
- Memoization caches results of repeated calls

RECURSION EXAMPLES

```
static long Factorial(int n) =>
    n <= 1 ? 1 : n * Factorial(n - 1);        // base case: n <= 1
Console.WriteLine(Factorial(5));              // 120

static long Fib(int n, Dictionary<int, long>? memo = null)
{
    memo ??= [];
    if (n < 2) return n;                      // base cases 0, 1
    if (memo.TryGetValue(n, out var hit)) return hit;
    return memo[n] = Fib(n - 1, memo) + Fib(n - 2, memo);
}
Console.WriteLine(Fib(50));                   // 12586269025

static int Depth(Node? node) =>               // tree recursion
    node is null ? 0 : 1 + Math.Max(Depth(node.Left), Depth(node.Right));
```

CODE EXPLAINED

- → Base case — stops the recursion
- → Recursive case — smaller input each call
- → Memo dictionary — turns $O(2^n)$ into $O(n)$
- → Tree depth — recursion mirrors the structure

WHEN TO USE

PROBLEM	RECURSION?
Trees, file systems, JSON	Natural fit
Divide & conquer (merge sort)	Natural fit
Simple counting loops	No — use a loop
Very deep inputs (100k+)	No — iterate with a stack
Overlapping subproblems	Yes + memoization

CHECKLIST

CHECK	WHY
Base case exists	Otherwise infinite recursion
Input shrinks	Guarantees termination
Depth bounded	Avoid stack overflow
Results cached?	Avoid exponential time

⚠ COMMON MISTAKES

- Missing or unreachable base case — infinite recursion and a fatal `StackOverflowException`
- Naive recursive Fibonacci — exponential time; memoize or iterate
- Relying on tail-call optimization — the JIT may or may not do it

✓ TIPS & BEST PRACTICES

- Write the base case first
- Convert deep recursion to a loop with `Stack<T>` or `Queue<T>`
- Use local functions for recursive helpers with extra parameters

QUICK SUMMARY

- Base case + smaller recursive case
- No guaranteed tail calls in C#
- Memoize overlapping subproblems
- Explicit stack for very deep data

Iterators & yield

yield return · yield break · IEnumerable<T> · deferred execution · infinite sequences · IAsyncEnumerable<T>

LAZY SEQUENCES WITH YIELD

A method returning `IEnumerable<T>` can use **`yield return`** to produce items one at a time. The compiler turns it into a state machine: **nothing runs until someone enumerates**, and each `MoveNext` resumes where the last `yield` left off. `yield break` ends the sequence. This is how LINQ operators are built.

KEY FACTS

- Return type must be `IEnumerable<T>`, `IEnumerator<T>` or `IAsyncEnumerable<T>`
- Execution is **deferred** until enumeration
- Re-enumerating runs the method again
- `yield` can't appear in a `catch` block

ITERATORS

```
static IEnumerable<int> CountTo(int max)
{
    for (int i = 1; i <= max; i++)
    { Console.Write($"[{i}]"); yield return i; }   // runs when pulled
}

var seq = CountTo(3);                    // nothing printed yet
foreach (var n in seq) Console.Write(n);  // [1]1[2]2[3]3

static IEnumerable<long> Fibonacci()      // infinite sequence
{
    long a = 0, b = 1;
    while (true) { yield return a; (a, b) = (b, a + b); }
}
Console.WriteLine(string.Join(",", Fibonacci().Take(8)));   // 0,1,1,2,3,5,8,13

static IEnumerable<string> NonEmpty(IEnumerable<string> lines)
{
    foreach (var l in lines)
    {
        if (l == "END") yield break;      // stop early
        if (l.Length > 0) yield return l;
    }
}
```

CODE EXPLAINED

- → `yield return` — hand out one item, pause
- → Deferred — calling `CountTo` runs nothing
- → Interleaving — prints happen as each item is pulled
- → Infinite iterator — safe with `Take`
- → `yield break` — end the sequence

BEHAVIOR

ACTION	EFFECT
`Call the method`	Creates the iterator only
`foreach / ToList()`	Runs the body
`Take(n)`	Stops after n items
`Enumerate twice`	Body runs twice

ITERATOR VS LIST

ITERATOR (YIELD)	RETURN A LIST
`Lazy, streams items`	Eager, all in memory
`Re-runs on each loop`	Computed once
`Validation is deferred`	Validation immediate

⚠ COMMON MISTAKES

- Expecting argument validation to run at call time — it runs on first `MoveNext`; validate in a wrapper
- Enumerating an expensive iterator several times — cache with `ToList()`
- Disposing a resource before enumeration finishes — keep `using` inside the iterator

✓ TIPS & BEST PRACTICES

- Use the wrapper + local iterator pattern for eager checks
- Stream large files with `yield return` per line
- Use `IAsyncEnumerable<T>` for paged API results

QUICK SUMMARY

- `yield return` — produce lazily
- `yield break` — end sequence
- Deferred execution — runs on enumeration
- `IAsyncEnumerable<T>` — async streams

Sheet 060

Extension Methods & Members

this T · static class · extension blocks (C# 14) · extension properties · static extension members · extension indexers (C# 15 preview)

ADDING MEMBERS TO EXISTING TYPES

Extension methods let you add methods to types you don't own: a `static` method in a `static` class whose first parameter has the `this` modifier. **C# 14** introduces **extension blocks** (`extension(T receiver) { … }`) that also allow **extension properties** and **static extension members**. **C# 15 (preview)** adds extension **indexers**.

KEY FACTS

- Extensions need their namespace imported with `using`
- Instance members always win over extensions
- They can't access private members of the type
- LINQ is built entirely from extension methods

CLASSIC & C# 14 EXTENSION BLOCKS

```
public static class Ext
{
    public static bool IsBlank(this string? s) =>   // classic (C# 3+)
        string.IsNullOrWhiteSpace(s);

    extension<T>(IEnumerable<T> src)                // C# 14 block
    {
        public bool IsEmpty => !src.Any();          // extension property
        public IEnumerable<T> SkipFirst() => src.Skip(1);
    }

    extension(string)                               // static extension
    {
        public static string Twice(string s) => s + s;
    }
}

Console.WriteLine("  ".IsBlank());              // True
int[] nums = [1, 2, 3];
Console.WriteLine(nums.IsEmpty);                // False
Console.WriteLine(string.Twice("ab"));          // abab
```

CODE EXPLAINED

- → `this string? s` — the type being extended
- → `extension<T>(IEnumerable<T> src)` — block for a receiver
- → Extension property — `nums.IsEmpty`, no parentheses
- → `extension(string)` — static members appear on the type
- → `string.Twice(…)` — called like a built-in static

WHAT YOU CAN ADD

MEMBER	SINCE
`Instance methods (this T)`	C# 3
`Properties`	C# 14
`Static methods & properties`	C# 14
`Indexers`	C# 15 preview
`Fields / state`	Never

C# 15 PREVIEW: EXTENSION INDEXER

```
extension(IEnumerable<int> seq)
{
    public int this[int i] => seq.ElementAt(i);
}
var third = Enumerable.Range(1, 10)[2];   // 3
```

⚠ COMMON MISTAKES

- Forgetting the `using` for the extension's namespace — the method seems to vanish
- Defining an extension with the same signature as an instance method — it's never called
- Putting heavy logic or hidden I/O in extension properties — they look like cheap fields

✓ TIPS & BEST PRACTICES

- Group extensions by receiver type: `StringExtensions`, `EnumerableExtensions`
- Extend interfaces (`IEnumerable<T>`) to reach many types at once
- Classic `this` methods and extension blocks can coexist; migration isn't required

QUICK SUMMARY

- `static M(this T x)` — classic extension method
- `extension(T x) { … }` — C# 14 block
- Extension properties & static members — C# 14
- Extension indexers — C# 15 preview

Arrays — Basics

FIXED-SIZE SEQUENCES

An **array** `T[]` stores a fixed number of elements of one type in contiguous memory, accessed by zero-based index. Its length is set at creation and can't change. C# supports **multidimensional** arrays `T[,]` (one rectangular block) and **jagged** arrays `T[][]` (arrays of arrays with different lengths).

KEY FACTS

- Arrays are reference types; elements start at `default(T)`
- Out-of-range index → `IndexOutOfRangeException`
- `Length` = total elements; `GetLength(d)` per dimension
- Use `List<T>` when the size changes

CREATING & USING ARRAYS

```
int[] a = new int[3];                  // [0, 0, 0]
int[] b = { 1, 2, 3 };                 // initializer
int[] c = [4, 5, 6];                   // collection expression (C# 12)
string[] names = new string[2];        // [null, null]

a[0] = 10;
Console.WriteLine(b[^1]);              // 3 (last)
Console.WriteLine(c.Length);           // 3

int[,] grid = new int[2, 3];           // 2 rows × 3 cols
grid[1, 2] = 7;
Console.WriteLine(grid.GetLength(1));  // 3

int[][] jagged = [[1], [2, 3], [4, 5, 6]];
Console.WriteLine(jagged[2][1]);       // 5

Array.Resize(ref a, 5);                // new array: [10, 0, 0, 0, 0]
int[] empty = [];                      // Array.Empty<int>()
```

CODE EXPLAINED

- → `new int[3]` — fixed size, zeroed
- → `{ … }` / `[ … ]` — initialize with values
- → `b[^1]` — index from the end
- → `int[,]` — rectangular, indexed `[r, c]`
- → `int[][]` — jagged, indexed `[r][c]`
- → `Array.Resize` — allocates a new array and copies

ARRAY KINDS

KIND	DECLARE	ACCESS
Single	`int[] a = new int[5]`	`a[i]`
Rectangular	`int[,] g = new int[3, 4]`	`g[r, c]`
3-D	`int[,,] cube`	`cube[x, y, z]`
Jagged	`int[][] j = new int[3][]`	`j[r][c]`

PROPERTIES

MEMBER	MEANING
`Length`	Total element count
`Rank`	Number of dimensions
`GetLength(d)`	Size of dimension d
`a[^1]` / `a[1..3]`	Index / range (copy)
`AsSpan()`	Zero-copy view

⚠ COMMON MISTAKES

- Using `<= arr.Length` in a loop — the last valid index is `Length - 1`
- Expecting `Array.Resize` to keep references — other variables still point to the old array
- Comparing arrays with `==` — compares references; use `SequenceEqual`

✓ TIPS & BEST PRACTICES

- Prefer jagged arrays over `[,]` for performance and flexibility
- Return `[]` (or `Array.Empty<T>()`) instead of `null`
- Use `ReadOnlySpan<T>` parameters to accept arrays without copying

QUICK SUMMARY

- `T[]` — fixed-size, zero-based
- `T[,]` — rectangular; `T[][]` — jagged
- `Length` / `GetLength(d)` — sizes
- `[…]` — collection expression initializer

Arrays — Methods

STATIC ARRAY HELPERS

The `System.Array` class provides static methods to **sort**, **search**, **fill**, **copy** and **reverse** arrays in place. Most mirror LINQ operations but work directly on the array without allocating. `BinarySearch` is fast (O(log n)) but requires the array to be **sorted** first.

KEY FACTS

- `Sort`, `Reverse`, `Fill`, `Clear` modify in place
- `IndexOf` returns -1 when missing
- `BinarySearch` returns a negative number when missing
- `Array.Sort` is unstable (introsort)

ARRAY METHODS

```
int[] n = [5, 3, 8, 1, 9];

Array.Sort(n);                          // [1, 3, 5, 8, 9]
Console.WriteLine(Array.BinarySearch(n, 8));   // 3
Console.WriteLine(Array.BinarySearch(n, 4));   // -3 (~index to insert)
Console.WriteLine(Array.IndexOf(n, 9));        // 4
Array.Reverse(n);                       // [9, 8, 5, 3, 1]

int[] z = new int[4];
Array.Fill(z, 7);                       // [7, 7, 7, 7]
Array.Clear(z, 1, 2);                   // [7, 0, 0, 7]
Array.Copy(n, z, 2);                    // z = [9, 8, 0, 7]

string[] w = ["pear", "fig", "apple"];
Array.Sort(w, (x, y) => x.Length.CompareTo(y.Length)); // [fig, pear, apple]
int idx = Array.FindIndex(w, s => s.StartsWith('a'));  // 2
bool any = Array.Exists(w, s => s.Length > 4);         // True
int[] lens = Array.ConvertAll(w, s => s.Length);       // [3, 4, 5]
```

CODE EXPLAINED

- → `Sort` — ascending in place
- → `BinarySearch` — needs sorted input; `~result` = insert point
- → `Reverse` / `Fill` / `Clear` — in-place edits
- → `Copy(src, dest, count)` — copy elements
- → Comparison lambda — custom sort order
- → `FindIndex` / `Exists` / `ConvertAll` — predicate helpers

ARRAY METHODS

METHOD	RETURNS / EFFECT
`Array.Sort(a [, cmp])`	Sorts in place
`Array.BinarySearch(a, x)`	Index or negative
`Array.IndexOf(a, x)`	Index or -1
`Array.Find(a, pred)`	First match or default
`Array.FindAll(a, pred)`	New array of matches
`Array.TrueForAll(a, pred)`	bool

ARRAY VS LINQ

IN PLACE (ARRAY)	NEW SEQUENCE (LINQ)
`Array.Sort(a)`	`a.Order()`
`Array.Reverse(a)`	`a.Reverse()`
`Array.Find(a, p)`	`a.FirstOrDefault(p)`
`Array.ConvertAll(a, f)`	`a.Select(f).ToArray()`
`Array.Exists(a, p)`	`a.Any(p)`

⚠ COMMON MISTAKES

- Calling `BinarySearch` on unsorted data — returns wrong answers without any error
- Expecting `Array.Sort` to keep equal items in order — it's unstable; use LINQ `OrderBy` for stability
- Confusing `Array.Copy` with assignment — `b = a` copies the reference, not the elements

✓ TIPS & BEST PRACTICES

- Use `~Array.BinarySearch(...)` to find the insertion point
- Use `a.AsSpan().Sort()` / `.Reverse()` for span-based work
- Sort parallel arrays together with `Array.Sort(keys, items)`

QUICK SUMMARY

- `Array.Sort` / `Reverse` — in place
- `BinarySearch` — sorted only
- `Fill` / `Clear` / `Copy` — bulk edits
- `Find` / `Exists` / `ConvertAll` — predicates

Collection Expressions

[1, 2, 3] · .. spread · [] empty · target typing · with(…) arguments (C# 15 preview) · CollectionBuilder

ONE SYNTAX FOR EVERY COLLECTION

Collection expressions (C# 12) create arrays, lists, spans, sets and immutable collections with the same `[a, b, c]` syntax; the **target type** decides what gets built. The **spread** element `..xs` inlines another collection. **C# 15 (preview)** adds a leading `with(...)` element that passes constructor arguments such as capacity or a comparer.

KEY FACTS

- No natural type: `var x = [1, 2];` is an error
- `[]` is the empty collection of the target type
- Spans may be stack-allocated — no heap use
- Custom types opt in with `[CollectionBuilder]`

COLLECTION EXPRESSIONS

```
int[] arr = [1, 2, 3];
List<string> names = ["Ada", "Bob"];
Span<int> span = [10, 20, 30];              // can live on the stack
HashSet<char> vowels = ['a', 'e', 'i'];
ImmutableArray<int> imm = [1, 2];
IEnumerable<int> seq = [1, 2, 3];           // compiler picks a type

int[] more = [0, .. arr, 4];                // [0, 1, 2, 3, 4]
List<string> all = [.. names, .. GetOthers()];// combine sources
int[] none = [];                            // empty

static int Sum(ReadOnlySpan<int> xs)
    { int s = 0; foreach (var x in xs) s += x; return s; }
Console.WriteLine(Sum([1, 2, 3]));          // 6 — no array allocated

// C# 15 preview: constructor arguments
List<string> big = [with(capacity: 1000), .. names];
HashSet<string> set = [with(StringComparer.OrdinalIgnoreCase), "A", "a"];
Console.WriteLine(set.Count);               // 1
```

CODE EXPLAINED

- → Target type — array, list, span, set, immutable…
- → `IEnumerable<int>` target — compiler chooses an efficient type
- → `..arr` — spread the elements of another collection
- → `[]` — empty collection
- → Span argument — elements stay on the stack
- → `with(...)` — must be the first element (C# 15 preview)

BEFORE VS AFTER

OLD	COLLECTION EXPRESSION
`new int[] { 1, 2 }`	`[1, 2]`
`new List<int> { 1, 2 }`	`[1, 2]`
`Array.Empty<int>()`	`[]`
`a.Concat(b).ToArray()`	`[.. a, .. b]`
`new List<T>(cap)`	`[with(capacity: cap)]`

CUSTOM TYPE SUPPORT

```
[CollectionBuilder(typeof(Bag), nameof(Bag.Create))]
public class Bag<T> : IEnumerable<T> { … }
public static class Bag
{
    public static Bag<T> Create<T>(ReadOnlySpan<T> xs)
        => new(xs);
}
Bag<int> b = [1, 2, 3];
```

⚠ COMMON MISTAKES

- `var list = [1, 2, 3];` — CS9176; declare the target type
- Assuming `IEnumerable<int> x = [...]` gives a `List` you can cast and mutate — the type is unspecified
- Using `with(...)` anywhere but first, or without `LangVersion` preview

✓ TIPS & BEST PRACTICES

- Use `[]` to return empty collections — clearer than `new()`
- Prefer `ReadOnlySpan<T>` parameters so callers' `[...]` avoid allocation
- Use spreads to combine lists without LINQ `Concat`

QUICK SUMMARY

- `[a, b]` — build any collection type
- `..xs` — spread
- `[]` — empty
- `with(...)` — ctor args (C# 15 preview)

List<T> — Basics

THE GO-TO GROWABLE LIST

`List<T>` is a resizable array: it keeps elements in order, allows duplicates, and gives O(1) access by index. It grows automatically (doubling its internal **capacity**) as you add items. It's the default collection for most code; inserting or removing near the start is O(n) because elements shift.

KEY FACTS

- `Count` = items stored; `Capacity` = room before resize
- Index past `Count - 1` → `ArgumentOutOfRangeException`
- `Remove(x)` removes the **first** match only
- In `System.Collections.Generic`

LIST<T> BASICS

```
List<string> fruits = ["apple", "banana"];   // or new List<string>()
fruits.Add("cherry");                        // [apple, banana, cherry]
fruits.Insert(0, "kiwi");                    // [kiwi, apple, banana, cherry]
fruits.AddRange(["fig", "date"]);            // append many

Console.WriteLine(fruits[1]);                // apple
Console.WriteLine(fruits.Count);             // 6
fruits[0] = "lime";                          // replace

bool removed = fruits.Remove("banana");      // True
fruits.RemoveAt(0);                          // remove by index
fruits.RemoveRange(0, 2);                    // remove 2 starting at 0
Console.WriteLine(fruits.Contains("fig"));   // True
Console.WriteLine(fruits.IndexOf("date"));   // 1

var big = new List<int>(capacity: 10_000);   // pre-size
fruits.Clear();                              // Count = 0
```

CODE EXPLAINED

- → `Add` — append to the end (amortized O(1))
- → `Insert(i, x)` — shift later items right
- → `AddRange` — append a collection
- → `list[i]` — get or set by index
- → `Remove` / `RemoveAt` / `RemoveRange` — delete
- → `capacity:` — avoid repeated growth

PERFORMANCE

OPERATION	COST
`list[i] get / set`	O(1)
`Add`	O(1) amortized
`Insert / RemoveAt (front)`	O(n)
`Contains / IndexOf / Remove(x)`	O(n)
`Sort`	O(n log n)

CORE MEMBERS

MEMBER	USE
`Count / Capacity`	Size / allocated room
`Add / AddRange`	Append
`Insert / InsertRange`	Add at index
`Remove / RemoveAt / RemoveRange`	Delete
`Contains / IndexOf`	Search
`Clear / TrimExcess`	Empty / shrink

⚠ COMMON MISTAKES

- Removing items in a forward `for` loop — elements shift and get skipped; use `RemoveAll` or loop backwards
- Using `Contains` on large lists in a loop — O(n²); use a `HashSet<T>` for lookups
- Exposing `List<T>` from public APIs — callers can mutate it; return `IReadOnlyList<T>`

✓ TIPS & BEST PRACTICES

- Pre-size with `capacity:` when you know the count
- Use `Queue<T>` / `LinkedList<T>` for frequent front inserts
- Return `IReadOnlyList<T>` and accept `IEnumerable<T>`

QUICK SUMMARY

- `List<T>` — growable ordered list
- `Add` / `Insert` / `Remove` — edit
- `list[i]` — O(1) index access
- `Count` vs `Capacity`

List<T> — Methods

LIST<T> METHODS WITH PREDICATES

Beyond basic add/remove, `List<T>` has built-in methods that take **predicates** and **converters**: find items, remove all matches, sort with custom rules, and convert to another type. They run **in place** or return a new list without needing LINQ, and some (`RemoveAll`, `Sort`) are faster than LINQ equivalents.

KEY FACTS

- `Find` returns `default(T)` when nothing matches
- `FindIndex` returns -1 when nothing matches
- `Sort()` uses `IComparable<T>` ; overloads accept rules
- `CollectionsMarshal.AsSpan` exposes the backing array

PREDICATE METHODS

```
List<int> n = [7, 2, 9, 4, 6, 1];

int firstBig = n.Find(x => x > 5);            // 7
List<int> evens = n.FindAll(x => x % 2 == 0);  // [2, 4, 6]
int idx = n.FindIndex(x => x > 8);            // 2
int last = n.FindLast(x => x < 5);            // 1
bool any = n.Exists(x => x > 8);              // True
bool all = n.TrueForAll(x => x > 0);          // True

int removed = n.RemoveAll(x => x < 3);        // 2 → [7, 9, 4, 6]
n.Sort();                                     // [4, 6, 7, 9]
n.Sort((a, b) => b.CompareTo(a));             // [9, 7, 6, 4] descending
n.Reverse();                                  // [4, 6, 7, 9]

List<string> s = n.ConvertAll(x => $"#{x}");  // ["#4", "#6", …]
IReadOnlyList<int> ro = n.AsReadOnly();       // read-only wrapper
n.ForEach(x => Console.Write(x));             // 4679

Span<int> span = CollectionsMarshal.AsSpan(n); // fast, no copy
span[0] = 100;                                // n[0] = 100
```

CODE EXPLAINED

- → `Find` / `FindLast` — first / last match
- → `FindAll` — new list of matches
- → `RemoveAll` — returns number removed
- → `Sort(Comparison)` — custom order
- → `ConvertAll` — map to a new `List<TOut>`
- → `AsSpan` — don't add/remove while using it

METHOD REFERENCE

METHOD	RETURNS
`Find(pred)`	T or default
`FindAll(pred)`	List<T>
`FindIndex / FindLastIndex`	int or -1
`Exists / TrueForAll`	bool
`RemoveAll(pred)`	int removed
`ConvertAll(f)`	List<TOut>

LIST METHOD VS LINQ

LIST<T>	LINQ
`Find(p)`	`FirstOrDefault(p)`
`FindAll(p)`	`Where(p).ToList()`
`Exists(p)`	`Any(p)`
`TrueForAll(p)`	`All(p)`
`ConvertAll(f)`	`Select(f).ToList()`

⚠ COMMON MISTAKES

- Treating `Find` returning `0` as "not found" for `List<int>` — 0 may be a real value; use `FindIndex`
- Adding to the list while holding a span from `AsSpan` — the span may point to a stale array
- Using `ForEach` with async lambdas — it doesn't await them

✓ TIPS & BEST PRACTICES

- Prefer `RemoveAll` over manual removal loops
- Sort by key with `list.Sort((a, b) => a.Age.CompareTo(b.Age))`
- Use LINQ when chaining several steps; list methods for single in-place ops

QUICK SUMMARY

- `Find` / `FindAll` / `FindIndex` — search
- `RemoveAll` — bulk delete
- `Sort(cmp)` — custom ordering
- `ConvertAll` — map to new list

Dictionary — Basics

KEY-VALUE LOOKUPS

`Dictionary<TKey, TValue>` maps unique **keys** to **values** using a hash table, giving near **O(1)** lookups, inserts and removals. Keys must be unique and non-null and should have stable `GetHashCode`/`Equals`. Use `TryGetValue` to read safely — the indexer throws when a key is missing.

KEY FACTS

- `dict[key] = v` adds **or** overwrites
- `Add(key, v)` throws if the key exists
- `dict[missing]` throws `KeyNotFoundException`
- Enumeration order isn't guaranteed

DICTIONARY BASICS

```
var ages = new Dictionary<string, int>
{
    ["Ada"] = 36,                          // index initializer
    ["Bob"] = 25,
};
ages.Add("Cy", 41);                        // throws if "Cy" exists
ages["Bob"] = 26;                          // overwrite

Console.WriteLine(ages["Ada"]);            // 36
// Console.WriteLine(ages["Zed"]);         // KeyNotFoundException

if (ages.TryGetValue("Cy", out int cy))    // safe read
    Console.WriteLine(cy);                 // 41
Console.WriteLine(ages.ContainsKey("Bob")); // True
Console.WriteLine(ages.Count);             // 3

ages.Remove("Bob");                        // True
ages.Remove("Cy", out int old);            // removed value → 41

foreach (var (name, age) in ages)          // KeyValuePair deconstruct
    Console.WriteLine($"{name}: {age}");
```

CODE EXPLAINED

- → `["Ada"] = 36` — index initializer
- → `Add` — strict insert
- → `dict[k] = v` — upsert
- → `TryGetValue` — one lookup, no exception
- → `Remove(k, out v)` — remove and return the value
- → `var (k, v)` — deconstruct during iteration

CORE MEMBERS

MEMBER	RETURNS / EFFECT
`dict[k]`	Value or KeyNotFoundException
`dict[k] = v`	Add or overwrite
`Add(k, v)`	ArgumentException on duplicate
`TryGetValue(k, out v)`	bool
`ContainsKey / ContainsValue`	bool (value search is O(n))
`Keys / Values`	Live views of the dictionary

GOOD KEYS

KEY TYPE	NOTES
`string`	Choose a comparer for case
`int / long / Guid`	Ideal
`enum`	Fine
`record / record struct`	Value equality built in
`Mutable class`	Avoid — hash can change

⚠ COMMON MISTAKES

- `if (d.ContainsKey(k)) x = d[k];` — two lookups; use `TryGetValue`
- Using a mutable object as a key and then changing it — the entry becomes unreachable
- Relying on insertion order — use `OrderedDictionary<K,V>` (.NET 9) or `SortedDictionary`

✓ TIPS & BEST PRACTICES

- Pass `StringComparer.OrdinalIgnoreCase` for case-insensitive keys
- Pre-size: `new Dictionary<K,V>(capacity)` for large loads
- Use `IReadOnlyDictionary<K,V>` in public APIs

QUICK SUMMARY

- `Dictionary<K,V>` — O(1) key lookup
- `TryGetValue` — safe read
- `dict[k] = v` — upsert
- Keys unique, non-null, stable hash

Sheet 067

Dictionary — Advanced Patterns

GetValueOrDefault · TryAdd · counting / grouping pattern · custom comparers · SortedDictionary · OrderedDictionary<K,V> · GetValueRefOrAddDefault

DICTIONARY PATTERNS

Real code uses dictionaries for **counting**, **grouping** and **caching**. Helpers like `GetValueOrDefault` and `TryAdd` remove boilerplate, and `CollectionsMarshal.GetValueRefOrAddDefault` updates a value with a **single hash lookup**. Choose `SortedDictionary` for key order and **`OrderedDictionary<K,V>`** (.NET 9) for insertion order with index access.

KEY FACTS

- `GetValueOrDefault` is an extension on `IReadOnlyDictionary`
- `TryAdd` returns false instead of throwing
- `ToDictionary` throws on duplicate keys
- Read-heavy, never-changing maps → `FrozenDictionary`

COUNTING, GROUPING, FAST UPDATES

```
string[] words = ["a", "b", "a", "c", "a"];

var counts = new Dictionary<string, int>();                // counting
foreach (var w in words)
    counts[w] = counts.GetValueOrDefault(w) + 1;           // a:3 b:1 c:1

foreach (var w in words)                                   // one lookup
    CollectionsMarshal.GetValueRefOrAddDefault(counts, w, out _)++;

var byLen = new Dictionary<int, List<string>>();           // grouping
foreach (var w in new[] { "hi", "yo", "hey" })
{
    if (!byLen.TryGetValue(w.Length, out var list))
        byLen[w.Length] = list = [];
    list.Add(w);                                           // 2:[hi, yo] 3:[hey]
}

bool added = counts.TryAdd("a", 99);                       // False
var ordered = new OrderedDictionary<string, int> { ["x"] = 1, ["y"] = 2 };
Console.WriteLine(ordered.GetAt(1));                       // [y, 2]
```

CODE EXPLAINED

- → `GetValueOrDefault(k) + 1` — simple counter
- → `GetValueRefOrAddDefault(...)++` — update by ref, single lookup
- → TryGetValue + add — classic grouping
- → `TryAdd` — insert only if absent
- → `OrderedDictionary.GetAt(i)` — index access (.NET 9)

DICTIONARY TYPES

TYPE	ORDER	LOOKUP
`Dictionary<K,V>`	Unspecified	O(1)
`SortedDictionary<K,V>`	By key	O(log n)
`OrderedDictionary<K,V>`	Insertion	O(1)
`ConcurrentDictionary<K,V>`	Unspecified	O(1), thread-safe
`FrozenDictionary<K,V>`	Unspecified	Fastest reads

BUILD FROM LINQ

CALL	RESULT
`ToDictionary(x => x.Id)`	Id → item
`ToDictionary(x => x.Id, x => x.Name)`	Id → name
`GroupBy(k).ToDictionary(…)`	Key → list
`CountBy(x => x.Kind)`	Kind → count (.NET 9)
`ToLookup(x => x.Kind)`	Read-only multi-map

⚠ COMMON MISTAKES

- Using `ToDictionary` when keys may repeat — throws; use `GroupBy` or `ToLookup`
- Modifying a dictionary while enumerating it — throws; iterate `Keys.ToList()`
- Sharing a plain `Dictionary` across threads with writes — corrupts state; use `ConcurrentDictionary`

✓ TIPS & BEST PRACTICES

- Use `CountBy` / `AggregateBy` (.NET 9) instead of manual counting loops
- Use `FrozenDictionary` for lookup tables built once at startup
- Use `TryAdd` for "first one wins" semantics

QUICK SUMMARY

- `GetValueOrDefault` — counting
- TryGetValue + add — grouping
- `GetValueRefOrAddDefault` — single-lookup updates
- Sorted / Ordered / Frozen variants

Sets

UNIQUE ITEMS

`HashSet<T>` stores **unique** values with O(1) `Add`, `Remove` and `Contains`, making it ideal for de-duplication and membership tests. It also supports mathematical **set operations** — union, intersection, difference — either in place or as tests. `SortedSet<T>` keeps items ordered and supports ranges.

KEY FACTS

- `Add` returns `false` for duplicates (no exception)
- No index access; order is unspecified
- Set operations like `UnionWith` modify the set in place
- Pass a comparer for case-insensitive strings

HASHSET IN ACTION

```
var tags = new HashSet<string>(StringComparer.OrdinalIgnoreCase)
    { "c#", "dotnet" };
Console.WriteLine(tags.Add("C#"));          // False (duplicate, ignoring case)
Console.WriteLine(tags.Contains("DOTNET")); // True

int[] nums = [3, 1, 3, 2, 1];
var unique = new HashSet<int>(nums);        // {3, 1, 2}

var a = new HashSet<int> { 1, 2, 3, 4 };
var b = new HashSet<int> { 3, 4, 5 };
a.IntersectWith(b);                         // a = {3, 4}
a.UnionWith([9]);                           // a = {3, 4, 9}
a.ExceptWith(b);                            // a = {9}
a.SymmetricExceptWith([9, 10]);             // a = {10}

var small = new HashSet<int> { 3 };
Console.WriteLine(small.IsSubsetOf(b));     // True
Console.WriteLine(b.Overlaps([5, 6]));      // True

var ranks = new SortedSet<int> { 50, 10, 30 };   // {10, 30, 50}
Console.WriteLine(ranks.Min);               // 10
var mid = ranks.GetViewBetween(20, 40);     // {30}
```

CODE EXPLAINED

- → Comparer — defines what counts as equal
- → `Add` — false if already present
- → `new HashSet<T>(source)` — de-duplicate
- → `IntersectWith` / `UnionWith` / `ExceptWith` — in place
- → `IsSubsetOf` / `Overlaps` — tests
- → `SortedSet` — ordered, `Min` / `Max`, ranges

SET OPERATIONS

METHOD	MATH	KEEPS
`UnionWith(x)`	A ∪ B	All items
`IntersectWith(x)`	A ∩ B	Items in both
`ExceptWith(x)`	A − B	Items only in A
`SymmetricExceptWith(x)`	A △ B	Items in exactly one

SET TESTS

METHOD	TRUE WHEN
`IsSubsetOf(x)`	Every item of A is in B
`IsSupersetOf(x)`	A contains all of B
`Overlaps(x)`	Share at least one item
`SetEquals(x)`	Same items, any order

⚠ COMMON MISTAKES

- Using `List.Contains` for repeated membership checks — O(n) each; a `HashSet` is O(1)
- Expecting `UnionWith` to return a new set — it mutates; use LINQ `Union` for a new sequence
- Storing mutable objects whose hash changes after insertion — they become unfindable

✓ TIPS & BEST PRACTICES

- Use `seen.Add(x)` as a combined "check and insert" in loops
- Use LINQ `Distinct` / `DistinctBy` for one-off de-duplication
- Use `FrozenSet<T>` for large, read-only lookup sets

QUICK SUMMARY

- `HashSet<T>` — unique, O(1) membership
- `UnionWith` / `IntersectWith` / `ExceptWith`
- `IsSubsetOf` / `Overlaps` — tests
- `SortedSet<T>` — ordered set

Sheet 069

Stacks, Queues & Linked Lists

Stack<T> · Push / Pop / Peek · Queue<T> · Enqueue / Dequeue · PriorityQueue<T,P> · LinkedList<T>

ORDERED ACCESS COLLECTIONS

Stack<T> is last-in, first-out (LIFO): push and pop at the top. **Queue<T>** is first-in, first-out (FIFO). **PriorityQueue<TElement, TPriority>** always dequeues the item with the **lowest** priority value. **LinkedList<T>** is a doubly linked list with O(1) insert and remove when you hold a node.

KEY FACTS

- `Pop` / `Dequeue` on empty throws — use `TryPop` / `TryDequeue`
- `Peek` looks without removing
- `PriorityQueue` isn't stable for equal priorities
- None of these are thread-safe (see Concurrent)

STACK, QUEUE, PRIORITYQUEUE, LINKEDLIST

```
var undo = new Stack<string>();
undo.Push("type A"); undo.Push("type B");
Console.WriteLine(undo.Pop());              // type B (last in)
Console.WriteLine(undo.Peek());             // type A (still there)

var jobs = new Queue<int>([1, 2]);
jobs.Enqueue(3);
Console.WriteLine(jobs.Dequeue());          // 1 (first in)
while (jobs.TryDequeue(out int j)) Console.Write(j);   // 23

var pq = new PriorityQueue<string, int>();
pq.Enqueue("low", 5); pq.Enqueue("urgent", 1); pq.Enqueue("mid", 3);
Console.WriteLine(pq.Dequeue());            // urgent (smallest priority)

var list = new LinkedList<string>(["a", "c"]);
LinkedListNode<string> a = list.First!;
list.AddAfter(a, "b");                      // a ⇄ b ⇄ c
list.AddFirst("start");
list.Remove(a);                             // O(1) with the node
Console.WriteLine(string.Join(",", list));  // start,b,c
```

CODE EXPLAINED

- → `Push` / `Pop` / `Peek` — stack top
- → `Enqueue` / `Dequeue` — queue ends
- → `TryDequeue` — loop until empty, no exceptions
- → `PriorityQueue` — element + priority; min first
- → `LinkedListNode` — handle for O(1) inserts
- → `AddFirst` / `AddAfter` / `Remove(node)`

CHOOSING

TYPE	ORDER	USE FOR
`Stack<T>`	LIFO	Undo, DFS, parsing
`Queue<T>`	FIFO	Work queues, BFS
`PriorityQueue<E,P>`	Lowest P first	Scheduling, Dijkstra
`LinkedList<T>`	Insertion	Frequent middle inserts

SAFE MEMBERS

MEMBER	RETURNS
`TryPop(out x) / TryPeek`	bool
`TryDequeue(out x)`	bool
`pq.TryDequeue(out e, out p)`	bool + priority
`Count`	Items stored
`Contains(x)`	O(n) search

⚠ COMMON MISTAKES

- Calling `Pop` / `Dequeue` without checking `Count` — throws `InvalidOperationException` when empty
- Expecting `PriorityQueue` to return the highest number first — it's a min-queue; negate or use a custom comparer
- Using `LinkedList<T>` for general lists — poor cache locality makes it slower than `List<T>`

✓ TIPS & BEST PRACTICES

- Use `PriorityQueue` with `Comparer<int>.Create((a, b) => b - a)` for max-first
- Use `Stack<T>` to turn recursion into iteration
- For producers/consumers across threads use `Channel<T>`

QUICK SUMMARY

- `Stack<T>` — LIFO
- `Queue<T>` — FIFO
- `PriorityQueue<E,P>` — min priority first
- `LinkedList<T>` — O(1) node insert/remove

Tuples & Deconstruction

(int, string) · named elements · ValueTuple · Deconstruct method · discards _ · tuple equality

LIGHTWEIGHT GROUPS OF VALUES

Tuples `(int, string)` bundle a few values without declaring a type; they're `System.ValueTuple` structs with optional element **names**. **Deconstruction** unpacks tuples — and any type with a `Deconstruct` method, including records — into separate variables. Tuples compare with == element by element.

KEY FACTS

- Unnamed elements are `Item1`, `Item2`, …
- Names are compile-time only (not in reflection)
- Tuples are mutable value types
- Use `_` to discard parts you don't need

TUPLES & DECONSTRUCTION

```
(int, string) t = (1, "one");
Console.WriteLine(t.Item2);                    // one

var p = (Name: "Ada", Age: 36);                // named elements
Console.WriteLine(p.Name);                     // Ada

(int Min, int Max) Bounds(int[] xs) => (xs.Min(), xs.Max());
var (lo, hi) = Bounds([4, 9, 2]);              // lo = 2, hi = 9
var (_, max) = Bounds([1, 5]);                 // discard min

int a = 1, b = 2;
(a, b) = (b, a);                               // swap

Console.WriteLine((1, "x") == (1, "x"));       // True

public class Point(int x, int y)
{
    public void Deconstruct(out int x0, out int y0) => (x0, y0) = (x, y);
}
var (x, y) = new Point(3, 4);                  // x = 3, y = 4

foreach (var (key, value) in dict) { }         // KeyValuePair deconstructs
```

CODE EXPLAINED

- → `(int, string)` — tuple type
- → `Name:` — element names for readability
- → Tuple return — multiple values from a method
- → `var (a, b) = …` — deconstruct
- → `(a, b) = (b, a)` — swap without a temp
- → `Deconstruct` — make your types deconstructible

TUPLE VS RECORD VS OUT

OPTION	BEST WHEN
`Tuple`	Private/internal, 2–3 values
`record`	Public API, named concept
`out params`	Try-pattern methods
`Anonymous type`	LINQ projections only

TUPLE FEATURES

FEATURE	EXAMPLE
`Inferred names`	`var t = (p.Name, p.Age); t.Age`
`Equality`	`(1, 2) == (1, 2)`
`Nested`	`((1, 2), 3)`
`As dictionary key`	`Dictionary<(int, int), T>`
`Pattern match`	`(a, b) switch { (0, _) => … }`

⚠ COMMON MISTAKES

- Returning big unnamed tuples from public APIs — `Item3` means nothing; use a record
- Expecting tuple names to survive serialization — JSON shows `Item1`, `Item2`
- Mixing up `System.Tuple<T>` (class, read-only) with value tuples `(T1, T2)`

✓ TIPS & BEST PRACTICES

- Name tuple elements in return types: `(int Count, decimal Total)`
- Use tuples as composite dictionary keys
- Add `Deconstruct` to small types for pattern matching

QUICK SUMMARY

- `(T1, T2)` — value tuple
- `var (a, b) = x` — deconstruction
- `_` — discard
- `Deconstruct` — custom deconstruction

Immutable & Frozen Collections

COLLECTIONS THAT NEVER CHANGE

Immutable collections (`System.Collections.Immutable`) never change; "modifying" methods return a **new** collection that shares structure with the old one. They're safe to share across threads. **Frozen collections** (`System.Collections.Frozen`, .NET 8) are built once and optimized for the **fastest possible reads**. `IReadOnlyList<T>` is only a read-only *view*.

KEY FACTS

- Use **builders** for many changes, then `ToImmutable()`
- `ImmutableArray<T>` is a struct — fast reads, slow adds
- Frozen creation is slower; lookups are faster
- `AsReadOnly()` wraps; the original can still change

IMMUTABLE & FROZEN

```
ImmutableList<int> v1 = [1, 2, 3];
ImmutableList<int> v2 = v1.Add(4);           // new list
Console.WriteLine($"{v1.Count} {v2.Count}");  // 3 4

var d1 = ImmutableDictionary<string, int>.Empty.Add("a", 1);
var d2 = d1.SetItem("a", 2);                 // d1 unchanged

var builder = ImmutableArray.CreateBuilder<int>();
for (int i = 0; i < 1000; i++) builder.Add(i);
ImmutableArray<int> arr = builder.ToImmutable();   // one allocation

FrozenDictionary<string, int> codes = new Dictionary<string, int>
    { ["US"] = 1, ["UK"] = 44, ["FR"] = 33 }.ToFrozenDictionary();
FrozenSet<string> banned = new[] { "root", "admin" }.ToFrozenSet();
Console.WriteLine(codes["UK"]);              // 44

var list = new List<int> { 1 };
IReadOnlyList<int> view = list.AsReadOnly();
list.Add(2);                                 // view.Count is now 2!
```

CODE EXPLAINED

- → `Add` on immutable — returns a new instance
- → `SetItem` — replace a value, old map untouched
- → Builder — batch edits, then freeze
- → `ToFrozenDictionary()` — build once, read fast
- → `ToFrozenSet()` — fast membership tests
- → `AsReadOnly()` — a view, not a snapshot

WHICH TYPE?

TYPE	STRENGTH
`ImmutableArray<T>`	Fast reads, rare changes
`ImmutableList<T>`	Frequent persistent edits
`ImmutableDictionary<K,V>`	Shared, versioned maps
`FrozenDictionary<K,V>`	Static lookup tables
`FrozenSet<T>`	Static allow/deny lists
`IReadOnlyList<T>`	Read-only API surface

READ-ONLY VS IMMUTABLE

	READ-ONLY VIEW	IMMUTABLE
`Caller can modify`	No	No
`Owner can modify`	Yes	No
`Thread-safe reads`	Only if no writes	Always
`Example`	list.AsReadOnly()	ImmutableList<T>

⚠ COMMON MISTAKES

- Calling `list.Add(x);` on an immutable list and ignoring the result — nothing changes
- Building a large immutable collection with repeated `Add` — use a builder
- Assuming `IReadOnlyList<T>` can't change — the underlying list still can

✓ TIPS & BEST PRACTICES

- Store configuration lookups in `FrozenDictionary` at startup
- Expose `IReadOnlyList<T>` but store `ImmutableArray<T>` for true safety
- Collection expressions work with immutable types: `ImmutableArray<int> a = [1, 2];`

QUICK SUMMARY

- Immutable — changes return new instances
- Builders — efficient bulk construction
- Frozen — fastest reads, built once
- Read-only views ≠ immutable

Concurrent Collections

THREAD-SAFE COLLECTIONS

`System.Collections.Concurrent` provides collections that many threads can use at once without external locks. **`ConcurrentDictionary`** offers atomic helpers like `GetOrAdd` and `AddOrUpdate`. **`ConcurrentQueue/ConcurrentStack/ConcurrentBag`** support lock-free producer/consumer work, and **`BlockingCollection`** adds blocking and bounding.

KEY FACTS

- Individual operations are atomic; sequences of them aren't
- `GetOrAdd` factories may run more than once
- `ConcurrentBag` is unordered, best when the same thread adds and takes
- For async pipelines prefer `Channel<T>`

CONCURRENT COLLECTIONS

```
var cache = new ConcurrentDictionary<string, int>();
int len = cache.GetOrAdd("hello", key => key.Length);      // 5
cache.AddOrUpdate("hits", 1, (key, old) => old + 1);       // add or increment
cache.TryUpdate("hits", 10, comparisonValue: 2);           // CAS-style update

var lazy = new ConcurrentDictionary<string, Lazy<Data>>();
Data d = lazy.GetOrAdd("k", k => new Lazy<Data>(() => Load(k))).Value;

var queue = new ConcurrentQueue<int>();
Parallel.For(0, 100, i => queue.Enqueue(i));
while (queue.TryDequeue(out int item)) Process(item);

using var work = new BlockingCollection<string>(boundedCapacity: 10);
var consumer = Task.Run(() =>
    { foreach (var job in work.GetConsumingEnumerable()) Handle(job); });
work.Add("job1"); work.Add("job2");
work.CompleteAdding();                    // consumer loop ends
await consumer;
```

CODE EXPLAINED

- → `GetOrAdd` — return existing or add new atomically
- → `AddOrUpdate` — insert or transform the old value
- → `TryUpdate(k, new, expected)` — compare-and-swap
- → `Lazy<T>` values — guarantee the factory runs once
- → `TryDequeue` — drain safely
- → `GetConsumingEnumerable` + `CompleteAdding` — producer/consumer

TYPES

TYPE	ORDER	USE
`ConcurrentDictionary<K,V>`	—	Shared caches, counters
`ConcurrentQueue<T>`	FIFO	Work items
`ConcurrentStack<T>`	LIFO	Object pools
`ConcurrentBag<T>`	None	Same-thread add/take
`BlockingCollection<T>`	Wraps one	Bounded, blocking sync
`Channel<T>`	FIFO	Async producer/consumer

ATOMIC VS NOT

CODE	ATOMIC?
`d.GetOrAdd(k, f)`	Yes (f may run twice)
`d.AddOrUpdate(k, a, u)`	Yes
`if (!d.ContainsKey(k)) d[k] = v`	No — race
`d[k] = d[k] + 1`	No — lost updates
`d.Count`	Takes all locks; slow

⚠ COMMON MISTAKES

- Check-then-act sequences (`ContainsKey` then add) — another thread can slip in; use `GetOrAdd` / `TryAdd`
- Expensive or side-effecting `GetOrAdd` factories — they can run more than once; wrap in `Lazy<T>`
- Polling `Count` in hot loops — it's expensive; track counts separately

✓ TIPS & BEST PRACTICES

- Use `AddOrUpdate` for counters, or `Interlocked` on a field
- Use `Channel<T>` instead of `BlockingCollection` in async code
- Prefer immutable data + swapping references for read-mostly state

QUICK SUMMARY

- `ConcurrentDictionary` — `GetOrAdd` / `AddOrUpdate`
- `ConcurrentQueue` / `Stack` / `Bag`
- `BlockingCollection` — bounded blocking pipeline
- Single operations are atomic, sequences aren't

Collection Interfaces

PROGRAMMING TO INTERFACES

Collections implement a hierarchy of interfaces. Choosing the right one for **parameters** and **return types** makes APIs flexible and honest: accept the **most general** type you need (often `IEnumerable<T>`), and return a **read-only** interface that describes what callers may do (`IReadOnlyList<T>`, `IReadOnlyDictionary<K,V>`).

KEY FACTS

- `IEnumerable<T>` = iterate only (may be lazy)
- `IReadOnlyCollection<T>` adds `Count`
- `IReadOnlyList<T>` adds an indexer
- `ICollection<T>` / `IList<T>` add mutation

CHOOSING PARAMETER & RETURN TYPES

```
// Accept the most general type you need
public decimal Total(IEnumerable<Order> orders) => orders.Sum(o => o.Amount);

// Need Count or index? Ask for it explicitly
public Order Middle(IReadOnlyList<Order> orders) => orders[orders.Count / 2];

// Return read-only interfaces, store concrete types
public class Cart
{
    private readonly List<Item> _items = [];
    public IReadOnlyList<Item> Items => _items;          // callers can't Add
    public void Add(Item i) => _items.Add(i);
}

private readonly Dictionary<string, int> _stock = [];
public IReadOnlyDictionary<string, int> Stock => _stock;

IEnumerable<int> q = Enumerable.Range(1, 3);   // lazy
List<int> l = [1, 2, 3];
ICollection<int> c = l;  c.Add(4);             // mutation via interface
IReadOnlySet<int> s = new HashSet<int>(l);     // .NET 5+
```

CODE EXPLAINED

- → `IEnumerable<T>` — works with arrays, lists, LINQ
- → `IReadOnlyList<T>` — you need `Count` + indexer
- → Private `List<T>`, public `IReadOnlyList<T>`
- → `IReadOnlyDictionary` — expose lookups safely
- → `ICollection<T>` — add/remove without indexing
- → `IReadOnlySet<T>` — membership without mutation

INTERFACES & MEMBERS

INTERFACE	ADDS
`IEnumerable<T>`	GetEnumerator
`IReadOnlyCollection<T>`	Count
`IReadOnlyList<T>`	this[int] (get)
`ICollection<T>`	Count, Add, Remove, Clear, Contains
`IList<T>`	this[int] (get/set), Insert, IndexOf
`IDictionary<K,V>` / `ISet<T>`	Map / set operations

API GUIDELINES

POSITION	PREFER
`Parameter, iterate once`	IEnumerable<T>
`Parameter, need Count/index`	IReadOnlyList<T>
`Return, caller reads`	IReadOnlyList<T>
`Return, caller owns copy`	List<T> / T[]
`Property exposing state`	IReadOnly* interface

⚠ COMMON MISTAKES

- Taking `List<T>` parameters when you only iterate — callers with arrays must copy
- Enumerating an `IEnumerable<T>` parameter several times — it may be a lazy query; materialize once
- Returning `IEnumerable<T>` over internal state and later mutating it — callers see changes mid-loop

✓ TIPS & BEST PRACTICES

- Accept `IEnumerable<T>`, return `IReadOnlyList<T>`
- Use `ReadOnlySpan<T>` parameters for hot, synchronous paths
- Return `[]` instead of `null` for empty results

QUICK SUMMARY

- `IEnumerable<T>` — iterate
- `IReadOnlyList<T>` — count + index, read-only
- `ICollection<T>` / `IList<T>` — mutable
- Accept general, return specific read-only

Sorting & Searching

ORDERING AND FINDING

Sorting needs a definition of order. Types define their **natural order** by implementing `IComparable<T>`; separate **comparers** (`IComparer<T>`, `Comparison<T>`, `StringComparer`) supply alternative orders. `List.Sort`/`Array.Sort` sort **in place** and are unstable; LINQ `OrderBy` returns a new **stable** sequence. Sorted data enables O(log n) `BinarySearch`.

KEY FACTS

- `CompareTo` returns < 0, 0 or > 0
- `Comparer<T>.Create(lambda)` builds a comparer
- `Order()` / `OrderDescending()` (.NET 7) sort by natural order
- `BinarySearch` returns `~insertIndex` when missing

SORTING & SEARCHING

```
public record Employee(string Name, int Age) : IComparable<Employee>
{
    public int CompareTo(Employee? o) => o is null ? 1 : Age.CompareTo(o.Age);
}

List<Employee> staff = [new("Cy", 41), new("Ada", 36), new("Bob", 36)];

staff.Sort();                                        // by Age (unstable)
staff.Sort((a, b) => string.Compare(a.Name, b.Name, StringComparison.Ordinal));
var byName = Comparer<Employee>.Create((a, b) => a.Name.CompareTo(b.Name));
staff.Sort(byName);

var stable = staff.OrderBy(e => e.Age).ThenBy(e => e.Name).ToList(); // stable
int[] nums = [5, 2, 8];
var asc = nums.Order();                              // 2, 5, 8

var names = new List<string> { "bob", "Ada", "cy" };
names.Sort(StringComparer.OrdinalIgnoreCase);        // Ada, bob, cy
int i = names.BinarySearch("BOB", StringComparer.OrdinalIgnoreCase); // 1
int miss = names.BinarySearch("ben", StringComparer.OrdinalIgnoreCase);
Console.WriteLine(~miss);                            // 1 (insert position)
```

CODE EXPLAINED

- → `IComparable<T>` — the type's default order
- → Lambda `Comparison<T>` — one-off order
- → `Comparer<T>.Create` — reusable comparer object
- → `OrderBy().ThenBy()` — multi-key, stable
- → `StringComparer` — culture/case rules for text
- → `~miss` — where a missing item would go

SORTING APIS

API	IN PLACE?	STABLE?
`list.Sort() / Array.Sort()`	Yes	No
`span.Sort()`	Yes	No
`OrderBy / ThenBy`	No (new)	Yes
`Order() / OrderDescending()`	No (new)	Yes
`SortedSet / SortedDictionary`	Kept sorted	—

SEARCHING

METHOD	COST
`IndexOf / Contains`	O(n)
`BinarySearch (sorted)`	O(log n)
`HashSet.Contains`	O(1)
`Dictionary.TryGetValue`	O(1)
`First(pred) (LINQ)`	O(n)

⚠ COMMON MISTAKES

- Writing `a.Age - b.Age` comparers — can overflow for large values; use `CompareTo`
- Sorting with one comparer and searching with another — `BinarySearch` gives wrong answers
- Expecting `List.Sort` to keep ties in original order — use `OrderBy` for stability

✓ TIPS & BEST PRACTICES

- Implement `IComparable<T>` only when a type has one obvious order
- Chain keys with `ThenBy` instead of complex comparison lambdas
- Use `StringComparer.Ordinal` for deterministic, culture-free sorting

QUICK SUMMARY

- `IComparable<T>` — natural order
- `IComparer<T>` / `Comparer<T>.Create` — custom order
- `Sort` in place vs `OrderBy` stable
- `BinarySearch` — sorted data only

Classes & Objects

BLUEPRINTS AND INSTANCES

A **class** is a reference type that bundles **data** (fields, properties) and **behavior** (methods). Creating an **object** with `new` allocates it on the heap and runs a constructor. Each object has its own copy of instance fields; `this` refers to the current object. **Object initializers** set properties right after construction.

KEY FACTS

- Default class accessibility is `internal`; members default to `private`
- Variables hold **references**; two variables can share one object
- `new()` (target-typed) infers the class from the declaration
- Unreferenced objects are garbage-collected

A COMPLETE SMALL CLASS

```
public class BankAccount
{
    private decimal _balance;                       // field (state)
    public string Owner { get; }                    // read-only property
    public string Id { get; init; } = Guid.NewGuid().ToString("N")[..8];

    public BankAccount(string owner) => Owner = owner;   // constructor

    public decimal Balance => _balance;             // computed property

    public void Deposit(decimal amount)             // method (behavior)
    {
        ArgumentOutOfRangeException.ThrowIfNegativeOrZero(amount);
        this._balance += amount;                    // this = current object
    }
}

var acct = new BankAccount("Ada");                  // create
acct.Deposit(100m);
BankAccount other = new("Bob") { Id = "custom" };   // target-typed new
Console.WriteLine(acct.Balance);                    // 100
```

CODE EXPLAINED

- → `private decimal _balance` — hidden state
- → `{ get; }` — set only in the constructor
- → `init` — settable in an object initializer
- → Constructor — runs on `new`
- → `this` — optional qualifier for members
- → `new("Bob") { Id = … }` — constructor + initializer

MEMBER KINDS

MEMBER	EXAMPLE
Field	`private int _count;`
Property	`public string Name { get; set; }`
Method	`public void Save() { }`
Constructor	`public Person(string n) { }`
Event	`public event EventHandler? Changed;`
Nested type	`public enum Kind { A, B }`

CREATING OBJECTS

SYNTAX	NOTES
`new Person("Ada")`	Classic
`Person p = new("Ada")`	Target-typed (C# 9)
`new Person { Age = 3 }`	Object initializer
`Activator.CreateInstance<T>()`	Reflection
`Person.Create(…)`	Static factory method

⚠ COMMON MISTAKES

- Making fields `public` — exposes internals; use properties
- Assuming `var b = a;` copies an object — both variables point to the same instance
- Doing heavy work or I/O in constructors — use a factory or an async init method

✓ TIPS & BEST PRACTICES

- Keep classes small and focused on one responsibility
- Override `ToString()` for easier debugging and logging
- Prefer `record` for data-only types with value equality

QUICK SUMMARY

- `class` — reference type blueprint
- `new` — create an instance
- Fields hold state, methods hold behavior
- Object initializers — set properties inline

Constructors

INITIALIZING OBJECTS

A **constructor** has the class's name and no return type; it runs when an object is created. Classes can have several (overloads) that **chain** with `: this(...)`, and derived classes call a base constructor with `: base(...)`. **Primary constructors** (C# 12) put parameters on the class itself, and **`required`** members force callers to set properties in an initializer.

KEY FACTS

- No constructor written → a public parameterless one is generated
- Order: field initializers → base ctor → ctor body
- Static constructor runs once, before first use
- Primary ctor params are captured, **not** properties (classes)

CONSTRUCTOR FORMS

```
public class Person
{
    public string Name { get; }
    public int Age { get; }
    public Person(string name, int age)
    {
        ArgumentNullException.ThrowIfNull(name);
        (Name, Age) = (name, age);
    }
    public Person(string name) : this(name, 0) { }     // chain to main ctor
}
public class Employee : Person
{
    public Employee(string name, string title) : base(name) => Title = title;
    public string Title { get; }
}
public class Service(ILogger<Service> log)             // primary ctor (C# 12)
{
    public void Run() => log.LogInformation("Running");
}
public class Options { public required string Url { get; init; } }
var o = new Options { Url = "https://x" };          // required: must set
```

CODE EXPLAINED

- → Main constructor — validate and assign
- → `: this(...)` — reuse another constructor
- → `: base(...)` — run the parent's constructor first
- → Primary ctor — parameters usable in all members
- → `required` — compile error if caller omits it

CONSTRUCTOR KINDS

KIND	EXAMPLE
Parameterized	`public Point(int x, int y)`
Chained	`: this(0, 0)`
Base call	`: base(name)`
Static	`static Config() { … }`

PRIMARY CONSTRUCTORS

TYPE	PARAMETERS BECOME
`record`	Public init properties
`record struct`	Public set properties
`class / struct`	Captured variables only

⚠ COMMON MISTAKES

- Calling virtual methods in a constructor — the derived part isn't initialized yet
- Expecting `new Service(...).logger` to work — primary ctor params on classes aren't members
- Forgetting `[SetsRequiredMembers]` on a ctor that sets required properties — callers still must set them

✓ TIPS & BEST PRACTICES

- Validate arguments in one main constructor; chain the others to it
- Use primary constructors for dependency injection classes
- Use `required` + `init` instead of long constructors for DTOs

QUICK SUMMARY

- Constructor — same name as class, no return type
- `: this()` / `: base()` — chaining
- Primary constructor — `class C(int x)`
- `required` — must be initialized

Access Modifiers

public · private · protected · internal · protected internal · private protected · file

WHO CAN SEE WHAT

Access modifiers control where a type or member can be used. Keeping members as private as possible is the core of **encapsulation**: you can change internals without breaking callers. Types default to `internal` (visible in the same assembly) and members default to `private`.

KEY FACTS

- Top-level types: only `public`, `internal` or `file`
- Interface members default to `public`
- `file` (C# 11) limits a type to one source file
- `InternalsVisibleTo` exposes internals to test assemblies

MODIFIERS IN CONTEXT

```
public class Account                        // visible to everyone
{
    private decimal _balance;               // this class only
    protected string Owner = "";            // this + derived classes
    internal int AuditId;                   // same assembly
    protected internal bool Flag;           // derived OR same assembly
    private protected int Secret;           // derived AND same assembly

    public decimal Balance => _balance;     // public read
    public string Name { get; private set; } = "";   // public get, private set
}

internal class Helper { }                   // default for top-level types

file class GeneratedOnly { }                // only in this .cs file

// In the .csproj — let tests see internals:
// <InternalsVisibleTo Include="MyApp.Tests" />
```

CODE EXPLAINED

- → `private` — tightest; default for members
- → `protected` — subclasses anywhere
- → `internal` — anything in the same project
- → `protected internal` — union of both
- → `private protected` — intersection of both
- → `private set` — different access per accessor

VISIBILITY MATRIX

MODIFIER	CLASS	DERIVED (SAME ASM)	SAME ASM	DERIVED (OTHER ASM)	ANYONE
`public`	✔	✔	✔	✔	✔
`protected internal`	✔	✔	✔	✔	✖
`internal`	✔	✔	✔	✖	✖
`protected`	✔	✔	✖	✔	✖
`private protected`	✔	✔	✖	✖	✖
`private`	✔	✖	✖	✖	✖
`file`	Same file only				

⚠ COMMON MISTAKES

- Making everything `public` — every member becomes part of your API and hard to change
- Using `protected` fields — subclasses can break invariants; expose `protected` methods or properties
- Mixing up `protected internal` (OR) and `private protected` (AND)

✓ TIPS & BEST PRACTICES

- Start `private`; widen only when needed
- Use `internal` + `InternalsVisibleTo` for testable internals
- Use `file` for helper types in source generators

QUICK SUMMARY

- `public` / `internal` — assembly boundary
- `protected` — subclasses
- `private` — default for members
- `file` — one source file (C# 11)

Properties

get / set · auto-properties · init · required · field keyword (C# 14) · computed properties · expression-bodied

ENCAPSULATED DATA

Properties look like fields to callers but run code through `get` and `set` accessors. **Auto-properties** generate a hidden backing field. `init` allows setting only during initialization, `required` forces callers to set it, and **C# 14's `field` keyword** lets you add logic to one accessor without declaring the backing field yourself.

KEY FACTS

- Accessors can have different access: `{ get; private set; }`
- `init` + `required` = immutable, must-set properties
- Computed properties have only a getter
- Use `@field` if you have a member literally named `field`

PROPERTY FORMS

```
public class Product
{
    public string Name { get; set; } = "";              // auto-property
    public int Id { get; init; }                         // initializer only
    public required string Sku { get; init; }           // caller must set
    public decimal Price { get; private set; }          // read-only to callers

    public string Code                                   // C# 14 field keyword
    {
        get;
        set => field = value?.Trim().ToUpperInvariant() ?? "";
    }

    private int _qty;                                    // classic backing field
    public int Quantity { get => _qty; set => _qty = Math.Max(0, value); }

    public decimal Total => Price * Quantity;           // computed
}

var p = new Product { Sku = "A1", Id = 7, Code = " ab " };   // Code = "AB"
// p.Id = 8;                                          // ✗ init-only
```

CODE EXPLAINED

- → `{ get; set; }` — compiler-generated field
- → `init` — only during construction/initializer
- → `required` — enforced at compile time
- → `private set` — only the class changes it
- → `field` — the auto-generated backing field (C# 14)
- → `=> expr` — computed, read-only

ACCESSOR COMBINATIONS

DECLARATION	CALLER CAN
`{ get; set; }`	Read, write
`{ get; }`	Read (set in ctor)
`{ get; init; }`	Read; set in initializer
`{ get; private set; }`	Read
`required … { get; init; }`	Must set once
`=> expression`	Read computed value

PROPERTY VS FIELD

	PROPERTY	FIELD
Validation	Yes	No
Data binding / JSON	Yes	Not by default
Interface member	Yes	No
Change later w/o breaking	Yes	No

⚠ COMMON MISTAKES

- Expensive work in getters — callers expect properties to be cheap; use a method
- Recursive property: `set => Name = value;` inside `Name` — stack overflow; use `field`
- Public setters on collection properties — callers can replace the whole list

✓ TIPS & BEST PRACTICES

- Default to `{ get; init; }` for data classes
- Use `field` to add validation without a separate backing field
- Expose collections as `IReadOnlyList<T>` with no setter

QUICK SUMMARY

- Auto-property — `{ get; set; }`
- `init` / `required` — controlled initialization
- `field` — C# 14 backing-field keyword
- Computed — `=> expression`

Static Members & Classes

TYPE-LEVEL MEMBERS

Static members belong to the **type**, not to any object: one copy shared by all instances, accessed as `TypeName.Member`. A **static class** can contain only static members and can't be instantiated — perfect for helpers like `Math`. A **static constructor** initializes static state once, before first use.

KEY FACTS

- Static methods can't use `this` or instance members
- `const` is implicitly static
- Static state lives for the whole app lifetime
- Static fields are shared across threads

STATIC IN PRACTICE

```
public class Order
{
    private static int _nextId = 1;                    // shared counter
    public static readonly TimeSpan Timeout = TimeSpan.FromSeconds(30);
    public const int MaxItems = 100;

    public int Id { get; } = Interlocked.Increment(ref _nextId); // per instance

    static Order() => Console.WriteLine("Order type initialized");  // once

    public static Order Create() => new();             // factory
}

var a = Order.Create();   // "Order type initialized" printed first
var b = Order.Create();
Console.WriteLine($"{a.Id} {b.Id} {Order.MaxItems}");  // 2 3 100

public static class Temperature                        // static class
{
    public static double ToFahrenheit(double c) => c * 9 / 5 + 32;
}
Console.WriteLine(Temperature.ToFahrenheit(100));      // 212
```

CODE EXPLAINED

- → `static int _nextId` — one value for the whole type
- → `static readonly` — runtime constant
- → `Interlocked.Increment` — thread-safe counter
- → Static constructor — runs before first access
- → `static class` — helpers only, no instances

CONST VS STATIC READONLY

	CONST	STATIC READONLY
`Value known`	Compile time	Run time
`Types`	Primitives, string, enum	Any
`Changing it`	Recompile all callers	Just the owner
`Example`	const int Max = 10	static readonly DateTime Start

USE STATIC FOR

GOOD FIT	AVOID FOR
`Pure helper functions`	Per-request state
`Factories`	Things you'll mock in tests
`Caches of immutable data`	Mutable global state
`Constants`	Dependencies (use DI)

⚠ COMMON MISTAKES

- Storing per-user or per-request data in static fields — every request shares it
- Unsynchronized writes to static fields from multiple threads — race conditions
- Hiding dependencies behind static calls — hard to test; inject interfaces instead

✓ TIPS & BEST PRACTICES

- Mark helper methods `static` when they don't use instance state
- Prefer DI singletons over hand-written static singletons in apps
- Use `using static System.Math;` to call `Sqrt` without the prefix

QUICK SUMMARY

- `static` — belongs to the type
- `static class` — no instances
- Static constructor — one-time init
- `const` vs `static readonly`

Indexers & Partial Types

this[int i] · multi-parameter indexers · partial class · partial methods · partial properties · partial constructors / events (C# 14)

INDEXERS AND PARTIAL TYPES

An **indexer** `this […]` lets objects be accessed with `[]` like arrays or dictionaries. **Partial types** split one class across several files — commonly one written by you and one by a **source generator** or designer. Partial **methods**, **properties** (C# 13) and **constructors/events** (C# 14) let one part declare a member and another implement it.

KEY FACTS

- Indexers can be overloaded by parameter type
- All partial parts must use `partial` and the same namespace
- Partial methods with access modifiers must be implemented
- Source generators rely heavily on partial members

INDEXERS & PARTIAL MEMBERS

```
public class Matrix(int rows, int cols)
{
    private readonly double[] _data = new double[rows * cols];

    public double this[int r, int c]                  // 2-parameter indexer
    {
        get => _data[r * cols + c];
        set => _data[r * cols + c] = value;
    }
}
var m = new Matrix(2, 2);  m[1, 0] = 3.5;

public partial class Customer                         // Customer.cs (yours)
{
    public partial string Name { get; set; }          // declaring part
}
public partial class Customer                         // Customer.g.cs (generated)
{
    private string _name = "";
    public partial string Name { get => _name; set => _name = value; }
}
```

CODE EXPLAINED

- → `this[int r, int c]` — multi-parameter indexer
- → `get` / `set` — like a property
- → `partial class` — one type, several files
- → Partial property — declared in one part, implemented in another (C# 13)

PARTIAL MEMBERS

MEMBER	SINCE	NOTES
`partial class / struct / interface`	C# 2	Split across files
`partial property / indexer`	C# 13	Declaring + implementing part
`partial constructor / event`	C# 14	Same pattern

INDEXERS IN THE BCL

TYPE	INDEXER
`List<T>`	this[int]
`Dictionary<K,V>`	this[K]
`string`	this[int] (get only)

⚠ COMMON MISTAKES

- Throwing a different exception type than BCL indexers — use `ArgumentOutOfRangeException` / `KeyNotFoundException`
- Editing generated partial files — changes are overwritten; put code in your own part
- Declaring a partial method with a return value but no implementation — compile error

✓ TIPS & BEST PRACTICES

- Use indexers only when `[]` is natural for the type
- Return `ref` from indexers on large structs to avoid copies
- Name generated files `*.g.cs` so tools treat them specially

QUICK SUMMARY

- `this[…]` — indexer
- `partial class` — split across files
- Partial methods / properties / ctors / events
- Used by source generators

Inheritance

: Base · base. · virtual · override · sealed · new (hiding) · protected members

REUSING AND EXTENDING CLASSES

A class can **inherit** from one base class with `: Base`, gaining its members. Methods marked **`virtual`** can be **overridden** in derived classes with `override`; `base.Method()` calls the parent's version. **`sealed`** stops further inheritance. C# supports **single** class inheritance plus any number of interfaces.

KEY FACTS

- All classes ultimately derive from `object`
- Constructors aren't inherited; chain with `: base(...)`
- `new` on a member **hides** rather than overrides
- Structs and records structs can't inherit classes

INHERITANCE CHAIN

```
public class Animal(string name)
{
    public string Name { get; } = name;
    public virtual string Speak() => "...";              // overridable
    public override string ToString() => $"{Name} says {Speak()}";
}

public class Dog(string name) : Animal(name)
{
    public override string Speak() => "Woof";
}

public sealed class Puppy(string name) : Dog(name)       // can't be inherited
{
    public override string Speak() => base.Speak() + "!"; // call parent
}

Animal a = new Puppy("Rex");
Console.WriteLine(a);              // Rex says Woof!
public class Cat(string n) : Animal(n)
{ public new string Speak() => "Meow"; }   // hides, doesn't override
Animal c = new Cat("Tom");
Console.WriteLine(c.Speak());      // ... (Animal's version)
```

CODE EXPLAINED

- → `virtual` — permits overriding
- → `: Animal(name)` — inherit and pass ctor args
- → `override` — replace the behavior
- → `base.Speak()` — call the parent implementation
- → `sealed` — end of the chain
- → `new` — hiding; called by static type

KEYWORDS

KEYWORD	ON	MEANING
`virtual`	Base member	Can be overridden
`override`	Derived member	Replaces base behavior
`abstract`	Base member	Must be overridden
`sealed`	Class / override	No further inheritance/override
`new`	Derived member	Hides base member

INHERITED FROM OBJECT

MEMBER	OVERRIDE?
`ToString()`	Often
`Equals(object?)`	For value equality
`GetHashCode()`	With Equals

⚠ COMMON MISTAKES

- Forgetting `virtual` on the base method — `override` fails with CS0506
- Using `new` when you meant `override` — behavior depends on the variable's type
- Deep inheritance hierarchies — fragile; prefer composition and interfaces

✓ TIPS & BEST PRACTICES

- Seal classes that aren't designed for inheritance
- Favor composition ("has-a") over inheritance ("is-a") for reuse
- Call `base` in overrides when extending, not replacing, behavior

QUICK SUMMARY

- `: Base` — single inheritance
- `virtual` / `override` — polymorphic methods
- `base.` — parent access
- `sealed` — stop inheritance

Polymorphism & Casting

ONE INTERFACE, MANY FORMS

Polymorphism lets code work with a base type or interface while each object supplies its own behavior. A call to a `virtual` member is dispatched at **run time** to the object's actual type. **Upcasting** (derived → base) is implicit; **downcasting** (base → derived) needs a check with `is` / `as` or an explicit cast.

KEY FACTS

- Overridden members dispatch on the runtime type
- Hidden (`new`) members dispatch on the compile-time type
- Failed explicit cast → `InvalidCastException`
- Covariant return types allowed since C# 9

POLYMORPHISM IN ACTION

```
abstract class Shape
{
    public abstract double Area();
    public virtual Shape Clone() => (Shape)MemberwiseClone();
}
class Circle(double r) : Shape
{
    public override double Area() => Math.PI * r * r;
    public override Circle Clone() => new(r);         // covariant return (C# 9)
}
class Square(double s) : Shape { public override double Area() => s * s; }

List<Shape> shapes = [new Circle(1), new Square(2)];   // upcast: implicit
foreach (var s in shapes)
    Console.WriteLine($"{s.GetType().Name}: {s.Area():F2}");

Shape sh = shapes[0];
if (sh is Circle c) Console.WriteLine("circle!");      // safe downcast
Square? sq = sh as Square;                             // null — not a square
// var bad = (Square)sh;                               // InvalidCastException

double total = shapes.Sum(s => s.Area());             // works for any Shape
```

CODE EXPLAINED

- → `abstract Area()` — each shape must implement
- → Covariant return — override returns a more specific type
- → `List<Shape>` — holds any subclass
- → Virtual dispatch — the right `Area()` runs
- → `is Circle c` — test + downcast
- → `as` — null on failure; cast — exception

CASTING OPTIONS

CODE	ON FAILURE
`Base b = derived;`	— (always works)
`(Derived)b`	InvalidCastException
`b as Derived`	null
`b is Derived d`	false

WHICH METHOD RUNS?

BASE MEMBER	DERIVED	VIA BASE REF
`virtual`	override	Derived version
`virtual`	new	Base version
`non-virtual`	new	Base version
`abstract`	override	Derived version

⚠ COMMON MISTAKES

- Long `if (x is A) … else if (x is B)` chains — move the behavior into a virtual method
- Blind downcasts `(Derived)b` — use `is` patterns
- Expecting `new` members to be polymorphic — only `override` participates in dispatch

✓ TIPS & BEST PRACTICES

- Program against base types or interfaces in method signatures
- Use switch expressions on types for closed sets you don't control
- Combine `abstract` base classes with `sealed` leaf classes

QUICK SUMMARY

- Virtual dispatch — runtime type decides
- Upcast implicit; downcast checked
- `is` / `as` — safe casts
- Covariant returns — C# 9

Abstract Classes

PARTIAL BLUEPRINTS

An **abstract class** can't be instantiated; it exists to be inherited. It can mix **implemented** members (shared code and state) with **abstract** members that every concrete subclass must override. Use it when related types share real implementation; use an interface when they only share a contract.

KEY FACTS

- Abstract members have no body and are implicitly virtual
- A class with any abstract member must be `abstract`
- Abstract classes can have constructors, fields and state
- A class inherits only one abstract class

TEMPLATE METHOD PATTERN

```
public abstract class Report
{
    protected Report(string title) => Title = title;
    public string Title { get; }
    public string Render() =>                        // template method
        $"== {Title} ==\n{Body()}\n{Footer()}";
    protected abstract string Body();                // must override
    protected virtual string Footer() => "-- end --";   // may override
}
public sealed class SalesReport(decimal total) : Report("Sales")
{
    protected override string Body() => $"Total: {total:C}";
}
Report r = new SalesReport(1234m);
Console.WriteLine(r.Render());
```

CODE EXPLAINED

- → `abstract class` — can't `new` it directly
- → `protected` ctor — only subclasses construct it
- → `Render` — shared algorithm with fixed steps
- → `abstract Body()` — each subclass fills it in
- → `virtual Footer()` — default that can be replaced
- → Use through the base type

ABSTRACT CLASS VS INTERFACE

	ABSTRACT CLASS	INTERFACE
`Instance fields / state`	Yes	No
`Constructors`	Yes	No
`Implemented members`	Yes	Default methods (C# 8)
`Multiple inheritance`	No (one base)	Yes (many)
`Access modifiers`	Any	Mostly public
`Best for`	Shared code in a family	Capabilities / contracts

MEMBER MODIFIERS

MODIFIER	SUBCLASS MUST…
`abstract`	Override
`virtual`	Optionally override
`(none)`	Use as-is
`sealed override`	Stop further overrides

⚠ COMMON MISTAKES

- Making the constructor `public` — misleading; make it `protected`
- Putting unrelated helpers in an abstract base "for reuse" — use composition instead
- Adding a new abstract member to a published base — breaks every existing subclass

✓ TIPS & BEST PRACTICES

- Combine an interface (contract) with an abstract base (default implementation)
- Keep abstract bases small and focused
- Seal concrete subclasses unless they're designed to be extended

QUICK SUMMARY

- `abstract class` — no direct instances
- `abstract` member — must override
- `virtual` member — may override
- Template method — base controls the flow

Interfaces

CONTRACTS BETWEEN TYPES

An **interface** declares members that implementing types promise to provide. A class or struct can implement **many** interfaces, which is how C# models capabilities like `IDisposable`, `IComparable<T>` or `IEnumerable<T>`. Modern interfaces can also contain **default implementations** (C# 8) and **static abstract** members (C# 11) for generic math.

KEY FACTS

- Names start with `I` by convention
- Members are public by default
- Explicit implementation hides a member unless cast to the interface
- Interfaces can't hold instance fields

INTERFACES IN PRACTICE

```
public interface IShape
{
    double Area();                                    // contract
    string Describe() => $"{GetType().Name}: {Area():F1}";  // default (C# 8)
}
public interface INamed { string Name { get; } }

public class Circle(double r) : IShape, INamed         // implement two
{
    public string Name => "circle";
    public double Area() => Math.PI * r * r;
}

IShape s = new Circle(2);
Console.WriteLine(s.Describe());                      // Circle: 12.6

public class Log : IDisposable
{
    void IDisposable.Dispose() => Console.WriteLine("closed"); // explicit
}
((IDisposable)new Log()).Dispose();                   // closed
```

CODE EXPLAINED

- → Member without body — implementer must supply it
- → Default method — available through the interface
- → `: IShape, INamed` — multiple interfaces
- → Explicit implementation — `IDisposable.Dispose()`
- → Cast to the interface to call explicit members

INTERFACES YOU'LL IMPLEMENT

INTERFACE	GIVES YOU
`IDisposable / IAsyncDisposable`	using / await using
`IEnumerable<T>`	foreach, LINQ
`IComparable<T>`	Sort()
`IEquatable<T>`	Fast typed equality
`IFormattable / ISpanFormattable`	Format strings
`INumber<T> (generic math)`	Math on any numeric T

INTERFACE FEATURES

FEATURE	C#
`Methods, properties, events, indexers`	1
`Default implementations`	8
`Static members`	8
`Static abstract / virtual`	11
`Private / protected helpers`	8

⚠ COMMON MISTAKES

- Fat interfaces with dozens of members — split them (Interface Segregation)
- Relying on default methods through a class reference — they're only visible via the interface type
- Creating an interface for every class "just in case" — add one when you need a second implementation or a test seam

✓ TIPS & BEST PRACTICES

- Depend on interfaces in constructors to enable DI and mocking
- Use explicit implementation to avoid name clashes between interfaces
- Keep interfaces small: `IReader`, `IWriter` rather than `IFileManager`

QUICK SUMMARY

- `interface` — contract, many per type
- Default methods — C# 8
- Explicit implementation — `I.Member`
- `static abstract` — generic math (C# 11)

Structs

LIGHTWEIGHT VALUE TYPES

A **struct** is a **value type**: it's copied on assignment and usually lives inline (on the stack or inside another object), avoiding heap allocations. Structs suit small, immutable data like points, money or IDs. `readonly struct` guarantees immutability, and `ref struct` (like Span<T>) must stay on the stack.

KEY FACTS

- Guideline: ≤ 16 bytes, immutable, value-like
- Structs can't inherit (but can implement interfaces)
- `default(MyStruct)` zeroes every field
- Parameterless constructors allowed since C# 10

STRUCT KINDS

```
public readonly struct Point(double x, double y)       // immutable
{
    public double X { get; } = x;
    public double Y { get; } = y;
    public double Length => Math.Sqrt(X * X + Y * Y);
    public Point With(double x) => new(x, Y);
}

var p1 = new Point(3, 4);
var p2 = p1;                          // full copy
Console.WriteLine(p1.Length);         // 5
Point zero = default;                 // (0, 0)

public record struct Temp(double C);   // value equality + ToString
Console.WriteLine(new Temp(20) == new Temp(20));   // True

public ref struct Parser(ReadOnlySpan<char> input)   // stack-only
{
    private ReadOnlySpan<char> _rest = input;        // can hold a span
    public char Next() { var c = _rest[0]; _rest = _rest[1..]; return c; }
}
```

CODE EXPLAINED

- → `readonly struct` — all fields immutable
- → Assignment — copies every field
- → `default` — all zeros, no constructor runs
- → `record struct` — generated equality and printing
- → `ref struct` — can hold spans; never boxed or captured

STRUCT FLAVORS

KIND	USE FOR
`struct`	Small mutable values (rare)
`readonly struct`	Immutable values (preferred)
`record struct`	Data with value equality
`readonly record struct`	Immutable data records
`ref struct`	Span-based parsers, buffers

STRUCT VS CLASS

	STRUCT	CLASS
`Kind`	Value	Reference
`Allocation`	Inline / stack	Heap
`Assignment`	Copies data	Copies reference
`Null`	Only as T?	Yes
`Inheritance`	No	Yes

⚠ COMMON MISTAKES

- Large mutable structs — copies are expensive and edits apply to copies
- Mutating a struct through a property getter (`list[0].X = 1`) — CS1612; you'd change a copy
- Casting structs to interfaces in hot loops — boxing allocates every time

✓ TIPS & BEST PRACTICES

- Default to `readonly struct` or `readonly record struct`
- Pass large structs with `in` to avoid copying
- Measure before converting classes to structs for performance

QUICK SUMMARY

- `struct` — value type, copied
- `readonly struct` — immutable
- `record struct` — value equality
- `ref struct` — stack-only

Records

record · positional records · with expressions · value equality · record struct · generated ToString · record inheritance

DATA-CENTRIC TYPES

A **record** (C# 9) is a class (or `record struct`, C# 10) designed for data. The compiler generates **value equality**, a readable `ToString()`, **deconstruction** and support for **`with` expressions** that copy an object with some properties changed. Positional syntax declares the whole type in one line.

KEY FACTS

- `record` = `record class` — reference type
- Positional record properties are `init`-only
- `with` creates a shallow copy
- Records can inherit from other records only

RECORDS

```
public record Person(string Name, int Age);            // positional record

var a = new Person("Ada", 36);
var b = new Person("Ada", 36);
Console.WriteLine(a == b);                  // True  (value equality)
Console.WriteLine(ReferenceEquals(a, b));  // False
Console.WriteLine(a);                       // Person { Name = Ada, Age = 36 }

var older = a with { Age = 37 };            // non-destructive copy
var (name, age) = older;                    // deconstruct → "Ada", 37

public record Product(string Sku, decimal Price)
{
    public decimal PriceWithTax => Price * 1.2m;      // extra members
    public string Sku { get; } = Sku.ToUpperInvariant(); // customize a property
}

public abstract record Shape;
public record Circle(double R) : Shape;               // record inheritance

public readonly record struct Money(decimal Amount, string Currency);
```

CODE EXPLAINED

- → Positional — properties, ctor and Deconstruct in one line
- → `==` — compares all members
- → `ToString()` — prints members
- → `with { … }` — copy with changes
- → Body — add computed or customized members
- → `record struct` — value-type record

COMPILER GENERATES

MEMBER	BEHAVIOR
`Equals / == / !=`	Compare all fields
`GetHashCode()`	Based on all fields
`ToString()`	Type { Prop = value, … }
`Deconstruct()`	Positional records
`<Clone>$ + with`	Shallow copy
`EqualityContract`	Type-aware equality

RECORD KINDS

DECLARATION	PROPERTIES
`record R(int X)`	class; X is init
`record struct R(int X)`	struct; X is get/set
`readonly record struct R(int X)`	struct; X is init
`record R { public int X { get; init; } }`	nominal form

⚠ COMMON MISTAKES

- Records holding `List<T>` — equality compares the list **reference**, not its contents
- Using records as EF Core entities that change identity-wise — entities need reference identity
- Expecting `with` to deep-copy — nested objects are shared

✓ TIPS & BEST PRACTICES

- Use records for DTOs, messages, events and value objects
- Use `readonly record struct` for small values like coordinates
- Use `ImmutableArray<T>` or override `Equals` when records hold collections

QUICK SUMMARY

- `record R(…)` — data type with value equality
- `with` — non-destructive mutation
- Generated `ToString`, `Deconstruct`
- `record struct` — value-type variant

Equality & Object Members

Equals override · GetHashCode · HashCode.Combine · IEquatable<T> · operator == pairing · ToString override

CUSTOM EQUALITY

By default, classes compare by **reference** and structs by **field values via reflection** (slow). To give a class value equality, override `Equals(object?)` and `GetHashCode()`, implement `IEquatable<T>`, and keep ==/!= consistent. Equal objects **must** return equal hash codes, or dictionaries and sets break.

KEY FACTS

- `HashCode.Combine(a, b, …)` builds good hash codes
- Hash codes should not change while an object is in a set
- Records generate all of this for you
- Override `ToString()` for debugging and logs

IMPLEMENTING EQUALITY

```
public sealed class Isbn : IEquatable<Isbn>
{
    public string Value { get; }
    public Isbn(string value) => Value = value.Replace("-", "");

    public bool Equals(Isbn? other) =>
        other is not null && Value == other.Value;      // typed, no boxing

    public override bool Equals(object? obj) => Equals(obj as Isbn);
    public override int GetHashCode() => Value.GetHashCode();

    public static bool operator ==(Isbn? a, Isbn? b) => Equals(a, b);
    public static bool operator !=(Isbn? a, Isbn? b) => !(a == b);

    public override string ToString() => $"ISBN {Value}";
}

var x = new Isbn("978-0-13-468599-1");
var y = new Isbn("9780134685991");
Console.WriteLine(x == y);                              // True
Console.WriteLine(new HashSet<Isbn> { x }.Contains(y)); // True
int h = HashCode.Combine(order.Id, order.Date);      // multi-field hash
```

CODE EXPLAINED

- → `IEquatable<T>.Equals(T?)` — fast typed comparison
- → `Equals(object?)` — forwards to the typed version
- → `GetHashCode` — must agree with `Equals`
- → `==` / `!=` — overload together, null-safe
- → Works in `HashSet` / `Dictionary` because hash + equals match
- → `HashCode.Combine` — combine several fields

EQUALITY CONTRACT

RULE	MEANING
Reflexive	x.Equals(x) is true
Symmetric	x.Equals(y) == y.Equals(x)
Transitive	x = y and y = z → x = z
Null	x.Equals(null) is false
Hash	Equal objects → equal hashes

OBJECT MEMBERS

MEMBER	DEFAULT
`Equals(object?)`	Reference (class)
`GetHashCode()`	Identity-based
`ToString()`	Type name

⚠ COMMON MISTAKES

- Overriding `Equals` but not `GetHashCode` — CS0659 warning; sets and dictionaries misbehave
- Basing the hash on mutable fields — the object gets "lost" inside a `HashSet` after changes
- Implementing `==` but leaving `Equals` as reference equality — inconsistent results

✓ TIPS & BEST PRACTICES

- Prefer `record` and skip all of this boilerplate
- Seal classes with custom equality to avoid subclass surprises
- Use `IEqualityComparer<T>` when equality depends on context

QUICK SUMMARY

- Override `Equals` + `GetHashCode` together
- Implement `IEquatable<T>`
- `HashCode.Combine` — hashing helper
- Records — equality generated

Generics — Basics

TYPE PARAMETERS

Generics let you write classes, interfaces and methods with **type parameters** (T) that callers fill in, giving type safety and reuse without casting or boxing. `List<T>`, `Dictionary<TKey, TValue>` and `Func<T, TResult>` are generic. The compiler usually **infers** type arguments for generic methods.

KEY FACTS

- Name type params `T` or `TSomething`
- `default(T)` / `default` gives null or zero
- Value-type instantiations get specialized code — no boxing
- C# 14: `nameof(List<>)` with an unbound generic

GENERIC TYPES & METHODS

```
public class Box<T>(T value)
{
    public T Value { get; } = value;
    public Box<TOut> Map<TOut>(Func<T, TOut> f) => new(f(Value));
}

var b = new Box<int>(21);
Box<string> s = b.Map(x => $"#{x * 2}");     // TOut inferred as string
Console.WriteLine(s.Value);                  // #42

static T FirstOrFallback<T>(IEnumerable<T> xs, T fallback)
{
    foreach (var x in xs) return x;
    return fallback;
}
Console.WriteLine(FirstOrFallback([3, 4], 0));          // 3 (T = int)
Console.WriteLine(FirstOrFallback<string>([], "none")); // none

static void Swap<T>(ref T a, ref T b) => (a, b) = (b, a);

Console.WriteLine(nameof(Dictionary<,>));    // Dictionary (C# 14)
```

CODE EXPLAINED

- → `Box<T>` — generic class
- → `Map<TOut>` — generic method on a generic type
- → Inference — `TOut` comes from the lambda's return
- → `<string>` — explicit when inference can't decide
- → `Swap<T>(ref T, ref T)` — works for any type
- → `nameof(Dictionary<,>)` — unbound generic name (C# 14)

COMMON GENERIC TYPES

TYPE	PARAMETERS
`List<T>`	Element type
`Dictionary<TKey, TValue>`	Key, value
`Func<T, TResult>`	Input, output
`Task<TResult>`	Async result
`Nullable<T>` / `T?`	Value type
`IEnumerable<T>`	Sequence element

TERMINOLOGY

TERM	EXAMPLE
Type parameter	`T in List<T>`
Type argument	`int in List<int>`
Open / unbound	`List<>`
Closed / constructed	`List<int>`
Generic method	`Max<T>(T a, T b)`

⚠ COMMON MISTAKES

- Using `object` parameters instead of generics — loses type safety and boxes value types
- Comparing `T` values with `==` without a constraint — not allowed; use `EqualityComparer<T>.Default`
- Creating `new T()` without the `new()` constraint — compile error

✓ TIPS & BEST PRACTICES

- Let inference work; only write type arguments when needed
- Prefer generic methods over generic classes when only one method needs `T`
- Use `EqualityComparer<T>.Default.Equals(a, b)` for generic equality

QUICK SUMMARY

- `class C<T>` — generic type
- `M<T>(…)` — generic method
- Type inference — usually automatic
- `default(T)` — zero or null

Generic Constraints

where T : class · struct · new() · notnull · unmanaged · base / interface constraints · allows ref struct

RESTRICTING TYPE PARAMETERS

Constraints (`where T : …`) tell the compiler what a type argument must support, which unlocks members you can use on T — calling interface methods, `new T()`, comparing, or treating it as a reference or value type. Callers get a compile error if their type doesn't satisfy the constraint.

KEY FACTS

- Multiple constraints are comma-separated
- Order: `class` / `struct` first, `new()` last
- Each type parameter gets its own `where` clause
- `allows ref struct` (C# 13) is an anti-constraint

CONSTRAINTS IN USE

```
static T Max<T>(T a, T b) where T : IComparable<T>
    => a.CompareTo(b) >= 0 ? a : b;             // CompareTo now allowed
Console.WriteLine(Max(3, 7));                   // 7
static T Create<T>() where T : new() => new T(); // parameterless ctor
public class Repo<TEntity, TKey>
    where TEntity : class, IEntity<TKey>        // ref type + interface
    where TKey : notnull                        // non-nullable key
{
    private readonly Dictionary<TKey, TEntity> _items = [];
    public void Add(TEntity e) => _items[e.Id] = e;
}

static T Sum<T>(IEnumerable<T> xs) where T : INumber<T>   // generic math
{
    T total = T.Zero;
    foreach (var x in xs) total += x;
    return total;
}
Console.WriteLine(Sum([1.5, 2.5]));             // 4
static int Len<T>(T s) where T : allows ref struct => 0;  // spans OK (C# 13)
```

CODE EXPLAINED

- → `IComparable<T>` — enables `CompareTo`
- → `new()` — enables `new T()`
- → `class, IEntity<TKey>` — combined constraints
- → `notnull` — suitable as dictionary key
- → `INumber<T>` — `T.Zero`, `+` on any number type
- → `allows ref struct` — accept `Span<T>` as `T`

CONSTRAINT REFERENCE

CONSTRAINT	T MUST BE
`class` / `class?`	Reference type (non-null / nullable)
`struct`	Non-nullable value type
`notnull`	Non-nullable type
`unmanaged`	Struct with no references
`new()`	Has public parameterless ctor
`BaseClass` / `IInterface`	Derives / implements

TYPICAL USES

GOAL	CONSTRAINT
`Compare` / `sort`	IComparable<T>
`Arithmetic`	INumber<T>
`Dictionary key`	notnull
`Factory`	new()
`Interop` / `pointers`	unmanaged

⚠ COMMON MISTAKES

- Over-constraining — every constraint limits who can use your API
- Expecting `where T : struct` to allow `int?` — nullable value types don't satisfy `struct`
- Writing `new()` before other constraints — it must be last

✓ TIPS & BEST PRACTICES

- Constrain to interfaces, not concrete classes, when possible
- Use generic math interfaces (`INumber<T>`) instead of overloads per numeric type
- Use `where T : Enum` for type-safe enum helpers

QUICK SUMMARY

- `where T : …` — constraint clause
- `class` / `struct` / `notnull` / `unmanaged`
- `new()` — create instances
- `INumber<T>` — generic math

Covariance & Contravariance

VARIANCE IN GENERICS

Covariance (`out T`) lets you use a more **derived** type than declared: an `IEnumerable<string>` can be used as an `IEnumerable<object>`. **Contravariance** (`in T`) goes the other way: an `Action<object>` can act as an `Action<string>`. Variance applies only to **interfaces and delegates** with reference-type arguments.

KEY FACTS

- `out T` — T only appears in output positions
- `in T` — T only appears in input positions
- Classes like `List<T>` are invariant
- Value types don't participate (`IEnumerable<int>` ≠ `IEnumerable<object>`)

COVARIANCE & CONTRAVARIANCE

```
IEnumerable<string> words = ["a", "b"];
IEnumerable<object> objs = words;                // ✓ covariant: out T
// List<object> list = new List<string>();       // ✗ List<T> is invariant

Func<string> getName = () => "Ada";
Func<object> getObj = getName;                   // ✓ Func<out TResult>

Action<object> logAny = o => Console.WriteLine(o);
Action<string> logText = logAny;                 // ✓ contravariant: in T

var byText = Comparer<object>.Create((a, b) => $"{a}".CompareTo($"{b}"));
IComparer<string> forStrings = byText;           // ✓ IComparer<in T>

public interface IProducer<out T> { T Produce(); }        // output only
public interface IConsumer<in T> { void Consume(T item); } // input only

object[] arr = new string[2];                    // array covariance (legacy)
// arr[0] = 42;          // ArrayTypeMismatchException at run time
```

CODE EXPLAINED

→ `IEnumerable<out T>` — sequence of derived fits base

→ `List<T>` — invariant because it both reads and writes

→ `Func<out TResult>` — returns can be widened

→ `Action<in T>` — handler for base works for derived

→ Declare `out` / `in` on your own interfaces

→ Array covariance — compiles but can throw

VARIANT BCL TYPES

TYPE	VARIANCE
`IEnumerable<out T>`	Covariant
`IReadOnlyList<out T>`	Covariant
`Func<in T, out TResult>`	Both
`Action<in T>`	Contravariant
`IComparer<in T>` / `IEqualityComparer<in T>`	Contravariant
`List<T>`, `IList<T>`, `Dictionary`	Invariant

MEMORY AID

KEYWORD	DIRECTION	EXAMPLE
`out`	Derived → Base	IEnumerable<Cat> → IEnumerable<Animal>
`in`	Base → Derived	Action<Animal> → Action<Cat>
(none)	Exact only	List<Cat> ≠ List<Animal>

⚠ COMMON MISTAKES

- Expecting `List<Cat>` to convert to `List<Animal>` — invariant; use `IEnumerable<Animal>` or `IReadOnlyList<Animal>`
- Relying on array covariance — writes can throw `ArrayTypeMismatchException`
- Expecting variance with value types — `IEnumerable<int>` isn't `IEnumerable<object>`

✓ TIPS & BEST PRACTICES

- Accept `IEnumerable<Base>` parameters so callers can pass derived lists
- Mark your own read-only interfaces `out T`
- Use `Cast<T>()` / `OfType<T>()` when conversion isn't automatic

QUICK SUMMARY

- `out T` — covariance (derived → base)
- `in T` — contravariance (base → derived)
- Interfaces and delegates only
- Arrays are covariant but unsafe

Reading Files

READING FILES

The static **File** class reads a whole file in one call — as a string, an array of lines or bytes. For large files use **File.ReadLines**, which streams lines lazily, or a **StreamReader** for full control. Every method has an Async version for apps that must stay responsive. Text defaults to **UTF-8**.

KEY FACTS

- Missing file → `FileNotFoundException`
- `ReadAllLines` loads everything; `ReadLines` streams
- Relative paths resolve from the current directory
- Pass an `Encoding` for non-UTF-8 files

READING TEXT

```
string path = "notes.txt";
if (!File.Exists(path)) return;

string all = File.ReadAllText(path);              // whole file
string[] lines = File.ReadAllLines(path);         // array of lines
byte[] raw = File.ReadAllBytes("logo.png");       // binary

foreach (string line in File.ReadLines("big.log")) // lazy, low memory
    if (line.Contains("ERROR")) Console.WriteLine(line);

int errors = File.ReadLines("big.log").Count(l => l.StartsWith("ERR"));

using var reader = new StreamReader(path);        // manual control
while (reader.ReadLine() is { } line)
    Console.WriteLine(line);

string text = await File.ReadAllTextAsync(path);  // async
await foreach (var l in File.ReadLinesAsync(path)) { }  // .NET 7+
```

CODE EXPLAINED

- → `File.Exists` — check before reading (still handle errors)
- → `ReadAllText` / `ReadAllLines` / `ReadAllBytes` — load fully
- → `ReadLines` — `IEnumerable<string>`, one line at a time
- → LINQ on `ReadLines` — streams without loading the file
- → `ReadLine() is { } line` — loop until null
- → `ReadAllTextAsync` / `ReadLinesAsync` — non-blocking

READ METHODS

METHOD	RETURNS	MEMORY
`File.ReadAllText`	string	Whole file
`File.ReadAllLines`	string[]	Whole file
`File.ReadLines`	IEnumerable<string>	One line
`File.ReadAllBytes`	byte[]	Whole file
`StreamReader`	Chunks / lines	Buffer

EXCEPTIONS TO EXPECT

EXCEPTION	CAUSE
`FileNotFoundException`	Path doesn't exist
`DirectoryNotFoundException`	Folder missing
`UnauthorizedAccessException`	No permission
`IOException`	File locked / in use

⚠ COMMON MISTAKES

- Using `ReadAllLines` on multi-GB logs — runs out of memory; use `ReadLines`
- Relying only on `File.Exists` — the file can disappear before you read it; still catch `IOException`
- Assuming the working directory is the exe folder — use `AppContext.BaseDirectory`

✓ TIPS & BEST PRACTICES

- Use `Path.Combine(AppContext.BaseDirectory, "data.json")` for app files
- Use async APIs in web and UI apps
- Pass `Encoding.Latin1` etc. when reading legacy files

QUICK SUMMARY

- `File.ReadAllText` — whole file
- `File.ReadLines` — streaming lines
- `StreamReader` — fine-grained reads
- `…Async` — non-blocking versions

Writing & Appending Files

WRITING FILES

`File.WriteAllText` / `WriteAllLines` **create or overwrite** a file in one call, and `AppendAllText` adds to the end. For many writes use a **`StreamWriter`**, which buffers output; dispose it (or `Flush`) so data reaches disk. **`FileStream`** with a `FileMode` gives precise control over create/append/overwrite behavior.

KEY FACTS

- Write methods overwrite existing files
- The target **directory** must already exist
- Data may sit in a buffer until `Dispose` / `Flush`
- Default encoding is UTF-8 without BOM

WRITING TEXT

```
File.WriteAllText("out.txt", "Hello\n");              // create/overwrite
File.AppendAllText("out.txt", "More\n");              // append
File.WriteAllLines("list.txt", ["a", "b", "c"]);      // one per line
File.WriteAllBytes("data.bin", [0x01, 0x02]);

using (var w = new StreamWriter("log.txt", append: true))
{
    w.WriteLine($"{DateTime.Now:o} started");
    w.WriteLine("…");
}                                                     // flushed + closed

await File.WriteAllTextAsync("out.txt", json);        // async

// Safe replace: write temp, then move over the original
string tmp = path + ".tmp";
await File.WriteAllTextAsync(tmp, newContent);
File.Move(tmp, path, overwrite: true);
```

CODE EXPLAINED

- → `WriteAllText` — replaces the whole file
- → `AppendAllText` — creates if missing, else appends
- → `WriteAllLines` — any `IEnumerable<string>`
- → `StreamWriter(path, append: true)` — buffered writes
- → `using` — guarantees flush and close
- → Temp + `File.Move` — avoid half-written files

FILEMODE

MODE	IF EXISTS	IF MISSING
`Create`	Overwrite	Create
`CreateNew`	IOException	Create
`Open`	Open	FileNotFoundException
`OpenOrCreate`	Open	Create
`Append`	Seek to end	Create

WRITE METHODS

METHOD	EFFECT
`File.WriteAllText`	Overwrite with string
`File.AppendAllLines`	Append lines
`File.Copy(a, b, true)`	Copy, overwrite
`File.Delete(path)`	No error if missing
`Directory.CreateDirectory`	Make folders first

⚠ COMMON MISTAKES

- Writing to a folder that doesn't exist — `DirectoryNotFoundException`; call `Directory.CreateDirectory` first
- Forgetting to dispose a `StreamWriter` — the end of the file is missing
- Overwriting important files in place — a crash leaves them corrupt; write to a temp file and move

✓ TIPS & BEST PRACTICES

- Use `File.AppendAllText` for quick logs; `ILogger` for real logging
- Pass `new UTF8Encoding(true)` if a tool needs a BOM
- Use `FileOptions.Asynchronous` for heavy async file I/O

QUICK SUMMARY

- `WriteAllText` — overwrite
- `AppendAllText` — append
- `StreamWriter` — buffered, dispose it
- Temp file + `File.Move` — atomic replace

Paths & Directories

PATHS AND DIRECTORIES

Path builds and takes apart path strings without touching the disk, handling the right separator for each OS. **Directory** and **File** perform actions (create, list, move, delete), while **FileInfo** / **DirectoryInfo** are object versions that cache metadata like size and timestamps.

KEY FACTS

- `/` works on Windows too; `Path` handles separators
- `Path.Combine` drops earlier parts if a later one is rooted
- `EnumerateFiles` streams; `GetFiles` builds an array
- `Directory.CreateDirectory` is a no-op if it exists

PATH & DIRECTORY

```
string dir  = Path.Combine("data", "2026", "reports");   // data/2026/reports
string file = Path.Join(dir, "q3.csv");

Console.WriteLine(Path.GetFileName(file));               // q3.csv
Console.WriteLine(Path.GetFileNameWithoutExtension(file)); // q3
Console.WriteLine(Path.GetExtension(file));              // .csv
Console.WriteLine(Path.GetDirectoryName(file));          // data/2026/reports
Console.WriteLine(Path.ChangeExtension(file, ".bak"));   // …/q3.bak
string full = Path.GetFullPath(file);                    // absolute path

Directory.CreateDirectory(dir);                          // creates all levels
var all = SearchOption.AllDirectories;
foreach (var f in Directory.EnumerateFiles("src", "*.cs", all))
    Console.WriteLine(f);

var info = new FileInfo(file);
if (info.Exists) Console.WriteLine($"{info.Length} bytes");

string home = Environment.GetFolderPath(Environment.SpecialFolder.UserProfile);
string temp = Path.GetTempPath();
```

CODE EXPLAINED

- → `Path.Combine` / `Join` — build paths safely
- → `GetFileName` / `GetExtension` — split a path
- → `GetFullPath` — resolve relative to the current directory
- → `CreateDirectory` — makes parent folders too
- → `EnumerateFiles(dir, pattern, AllDirectories)` — recursive search
- → `FileInfo` — size, dates, attributes

FILE & DIRECTORY ACTIONS

CALL	DOES
`File.Copy / Move / Delete`	Single file
`Directory.Move / Delete(d, true)`	Folder (recursive)
`Directory.Exists(d)`	Check folder
`Directory.GetDirectories(d)`	Subfolders
`File.GetLastWriteTimeUtc(f)`	Timestamp

USEFUL LOCATIONS

API	GIVES
`AppContext.BaseDirectory`	App's install folder
`Directory.GetCurrentDirectory()`	Working directory
`Path.GetTempPath()`	Temp folder
`SpecialFolder.ApplicationData`	Per-user app data
`SpecialFolder.UserProfile`	Home folder

⚠ COMMON MISTAKES

- Concatenating paths with `+ "\\" +` — breaks on Linux/macOS; use `Path.Combine`
- `Path.Combine("a", "/b")` returns `/b` — a rooted part resets the path
- Using `GetFiles` on huge trees — loads everything; use `EnumerateFiles`

✓ TIPS & BEST PRACTICES

- Use `Path.Join` when parts may start with separators
- Use `EnumerationOptions { IgnoreInaccessible = true }` for robust scans
- Watch folders with `FileSystemWatcher`

QUICK SUMMARY

- `Path` — string-only path helpers
- `Directory` / `File` — disk actions
- `FileInfo` / `DirectoryInfo` — metadata
- `EnumerateFiles` — streaming search

JSON — Serialization

SYSTEM.TEXT.JSON SERIALIZATION

JsonSerializer converts objects to JSON strings (`Serialize`) and back (`Deserialize<T>`). Behavior is set with **JsonSerializerOptions** — naming policy, indentation, null handling — and with **attributes** on properties. For speed and trimming/AOT support, a **source-generated** `JsonSerializerContext` does the work at compile time.

KEY FACTS

- Default naming is **as declared** (PascalCase)
- `JsonSerializerDefaults.Web` = camelCase + case-insensitive
- Records and `init` properties are supported
- Reuse one options instance — creating them is costly

ROUND TRIP

```
public record Person(string Name, int Age, string? Email = null);

var opts = new JsonSerializerOptions(JsonSerializerDefaults.Web)
{
    WriteIndented = true,
    DefaultIgnoreCondition = JsonIgnoreCondition.WhenWritingNull
};

string json = JsonSerializer.Serialize(new Person("Ada", 36), opts);
// { "name": "Ada", "age": 36 }

Person? p = JsonSerializer.Deserialize<Person>(json, opts);

await using var fs = File.OpenRead("people.json");          // from a file
var people = await JsonSerializer.DeserializeAsync<List<Person>>(fs, opts);

[JsonSerializable(typeof(Person))]                          // source gen
internal partial class AppJson : JsonSerializerContext { }
string fast = JsonSerializer.Serialize(p!, AppJson.Default.Person);
```

CODE EXPLAINED

- → `JsonSerializerDefaults.Web` — camelCase, case-insensitive
- → `WriteIndented` — pretty-print
- → `WhenWritingNull` — skip null properties
- → `Deserialize<T>` — JSON → object (may be null)
- → `DeserializeAsync` — stream from a file or HTTP body
- → `JsonSerializerContext` — compile-time, AOT-friendly

ATTRIBUTES

ATTRIBUTE	EFFECT
`[JsonPropertyName("id")]`	Rename in JSON
`[JsonIgnore]`	Skip property
`[JsonRequired]`	Fail if missing
`[JsonConverter(typeof(JsonStringEnumConverter))]`	Enum as text

COMMON OPTIONS

OPTION	PURPOSE
`PropertyNamingPolicy`	CamelCase, SnakeCaseLower…
`PropertyNameCaseInsensitive`	Lenient reading
`AllowTrailingCommas`	Lenient input
`ReferenceHandler.IgnoreCycles`	Break object cycles

⚠ COMMON MISTAKES

- Deserializing camelCase JSON with default options — properties stay empty; use Web defaults
- Creating new `JsonSerializerOptions` per call — slow; cache a static instance
- Serializing EF entities with cycles — throws; project to DTOs or ignore cycles

✓ TIPS & BEST PRACTICES

- Use source generation for Native AOT and trimmed apps
- Use records as DTOs for clean JSON contracts
- Use `JsonSerializer.Serialize(obj, opts)` with `JsonNamingPolicy.SnakeCaseLower` for snake_case APIs

QUICK SUMMARY

- `Serialize` / `Deserialize<T>` — core API
- `JsonSerializerOptions` — behavior
- Attributes — per-property control
- `JsonSerializerContext` — source generation

JSON — DOM & Readers

JsonDocument · JsonElement · JsonNode · JsonObject · Utf8JsonReader · Newtonsoft.Json differences

WORKING WITH RAW JSON

When you don't have (or want) a class for the data, read JSON as a tree. **`JsonDocument`** is a fast, **read-only** DOM over pooled memory (dispose it). **`JsonNode`** / `JsonObject` / `JsonArray` form a **mutable** DOM you can edit and write back. **`Utf8JsonReader`** is a forward-only reader for maximum performance.

KEY FACTS

- `JsonDocument` must be disposed
- `GetProperty` throws if missing; `TryGetProperty` doesn't
- `JsonNode` indexers return `null` when missing
- All live in `System.Text.Json(.Nodes)`

JSONDOCUMENT & JSONNODE

```
string json = """{ "name": "Ada", "tags": ["c#", "math"], "age": 36 }""";

using JsonDocument doc = JsonDocument.Parse(json);          // read-only
JsonElement root = doc.RootElement;
string name = root.GetProperty("name").GetString()!;       // Ada
int age = root.GetProperty("age").GetInt32();               // 36
foreach (var t in root.GetProperty("tags").EnumerateArray())
    Console.Write(t.GetString() + " ");                     // c# math
bool hasEmail = root.TryGetProperty("email", out _);        // False

JsonNode node = JsonNode.Parse(json)!;                      // mutable
node["age"] = 37;                                           // edit
node["tags"]!.AsArray().Add("ai");                          // append
node["address"] = new JsonObject { ["city"] = "London" };   // add object
Console.WriteLine(node["address"]?["city"]);                // London
string updated = node.ToJsonString();
```

CODE EXPLAINED

- → `"""…"""` — raw literal, no escaping needed
- → `JsonDocument.Parse` — fast immutable tree
- → `GetProperty().GetInt32()` — typed accessors
- → `EnumerateArray()` — iterate JSON arrays
- → `JsonNode.Parse` — editable tree
- → `ToJsonString()` — serialize back

WHICH API?

API	MUTABLE	BEST FOR
`JsonSerializer`	—	Typed objects
`JsonDocument`	No	Fast reads, big payloads
`JsonNode / JsonObject`	Yes	Patch & build JSON
`Utf8JsonReader`	No	Streaming, max speed
`Utf8JsonWriter`	—	Write JSON manually

NEWTONSOFT.JSON EQUIVALENTS

NEWTONSOFT	SYSTEM.TEXT.JSON
`JsonConvert.SerializeObject`	`JsonSerializer.Serialize`
`JObject.Parse`	`JsonNode.Parse`
`[JsonProperty("x")]`	`[JsonPropertyName("x")]`
`JToken`	`JsonNode / JsonElement`

⚠ COMMON MISTAKES

- Forgetting to dispose `JsonDocument` — pooled buffers leak
- Using a `JsonElement` after its document is disposed — `ObjectDisposedException`
- Calling `GetString()` on a number — throws `InvalidOperationException`; check `ValueKind`

✓ TIPS & BEST PRACTICES

- Use `root.Clone()` to keep a `JsonElement` beyond the document's life
- Use `JsonNode` for small config edits; typed classes for real data
- Check `element.ValueKind` before reading unknown data

QUICK SUMMARY

- `JsonDocument` — read-only DOM
- `JsonNode` — mutable DOM
- `Utf8JsonReader` — streaming reader
- `TryGetProperty` — safe lookup

XML

READING AND WRITING XML

LINQ to XML (`System.Xml.Linq`) is the modern XML API: `XDocument` and `XElement` load, query and build XML with LINQ. **Explicit casts** read element/attribute values safely, and **functional construction** builds trees in one expression. `XmlSerializer` maps whole classes to XML when you need a typed round trip.

KEY FACTS

- `Element("x")` returns `null` if missing
- `(int?)el.Attribute("y")` casts safely
- `Descendants` searches all levels; `Elements` only children
- Namespaces use `XNamespace ns = "…"; ns + "name"`

LINQ TO XML

```
var doc = XDocument.Parse("""
<books>
  <book id="1" year="2008"><title>C# in Depth</title></book>
  <book id="2"><title>CLR via C#</title></book>
</books>
""");

foreach (var b in doc.Descendants("book"))
{
    string title = (string?)b.Element("title") ?? "";
    int? year = (int?)b.Attribute("year");           // null if missing
    Console.WriteLine($"{b.Attribute("id")?.Value}: {title} {year}");
}

var recent = doc.Descendants("book")
    .Where(b => (int?)b.Attribute("year") > 2000)
    .Select(b => (string)b.Element("title")!);

var xml = new XElement("order", new XAttribute("id", 42),
    new XElement("item", "Pen"), new XElement("qty", 3));
xml.Save("order.xml");   // <order id="42"><item>Pen</item><qty>3</qty></order>
```

CODE EXPLAINED

- → `XDocument.Parse` / `Load` — from string / file
- → `Descendants("book")` — every `<book>` at any depth
- → `(string?)Element(…)` — value or null
- → `(int?)Attribute(…)` — typed attribute value
- → LINQ — filter and project XML
- → `new XElement(…)` — build XML declaratively

NAVIGATION

MEMBER	RETURNS
`Root`	Top element
`Elements("x")`	Direct children
`Descendants("x")`	All levels
`Element("x")`	First child or null
`Attribute("a")`	XAttribute or null
`Parent / Ancestors()`	Upward navigation

XMLSERIALIZER

```
var ser = new XmlSerializer(typeof(Order));
using var w = File.Create("o.xml");
ser.Serialize(w, order);
using var r = File.OpenRead("o.xml");
var back = (Order)ser.Deserialize(r)!;
```

⚠ COMMON MISTAKES

- Using `.Value` on a possibly missing element — `NullReferenceException`; use casts
- Forgetting namespaces — `Descendants("book")` finds nothing in namespaced XML
- Using the old `XmlDocument` API for new code — LINQ to XML is simpler

✓ TIPS & BEST PRACTICES

- Use `XDocument.Load(path, LoadOptions.PreserveWhitespace)` to keep formatting
- Use `XmlReader` for very large files
- Prefer JSON for new APIs; use XML when a format requires it

QUICK SUMMARY

- `XDocument` / `XElement` — LINQ to XML
- `Descendants` / `Elements` — query
- Casts — safe value reads
- `XmlSerializer` — typed round trip

Streams & Binary Files

BYTE STREAMS

Stream is the abstract base for reading and writing sequences of bytes: files (`FileStream`), memory (`MemoryStream`), network (`NetworkStream`) and wrappers like `GZipStream` or `BufferedStream`. Streams can be **chained**, and `BinaryReader`/`BinaryWriter` read and write primitive types in binary form.

KEY FACTS

- Check `CanRead`, `CanWrite`, `CanSeek` before use
- `Position` / `Seek` work only on seekable streams
- `Read` may return fewer bytes than requested
- Always dispose streams (`using`)

STREAMS IN PRACTICE

```
using var ms = new MemoryStream();
using (var bw = new BinaryWriter(ms, Encoding.UTF8, leaveOpen: true))
{
    bw.Write(42);              // 4 bytes
    bw.Write(3.14);            // 8 bytes
    bw.Write("hi");            // length-prefixed string
}
ms.Position = 0;               // rewind
using var br = new BinaryReader(ms);
int i = br.ReadInt32(); double d = br.ReadDouble();   // same order as written
Console.WriteLine($"{i} {d} {br.ReadString()}");      // 42 3.14 hi

await using var src = File.OpenRead("big.log");
await using var dst = File.Create("big.log.gz");
await using var gz = new GZipStream(dst, CompressionLevel.Optimal);
await src.CopyToAsync(gz);                        // compress while copying

byte[] buffer = new byte[4096];
int n = await src.ReadAsync(buffer);              // may be < 4096
await src.ReadExactlyAsync(buffer.AsMemory(0, 16)); // exactly 16 or throw
```

CODE EXPLAINED

- → `MemoryStream` — bytes in memory
- → `BinaryWriter` — write primitives as bytes
- → `leaveOpen: true` — don't close the inner stream
- → `Position = 0` — rewind before reading
- → Chaining — file → gzip → file
- → `ReadExactlyAsync` — loop until the buffer is full (.NET 7)

STREAM TYPES

TYPE	BACKS ONTO
`FileStream`	Disk file
`MemoryStream`	byte[] in memory
`NetworkStream`	Socket
`GZipStream / BrotliStream`	Compression wrapper
`BufferedStream`	Adds buffering
`CryptoStream`	Encryption wrapper

CORE MEMBERS

MEMBER	USE
`Read / ReadAsync`	Fill a buffer
`Write / WriteAsync`	Write bytes
`CopyTo / CopyToAsync`	Pipe to another stream
`Position / Seek / Length`	Random access
`Flush / FlushAsync`	Push buffered data

⚠ COMMON MISTAKES

- Assuming `Read` fills the buffer — it returns how many bytes it read; loop or use `ReadExactly`
- Reading a `MemoryStream` right after writing — `Position` is at the end; rewind first
- Disposing a wrapper that closes the stream you still need — pass `leaveOpen: true`

✓ TIPS & BEST PRACTICES

- Use `CopyToAsync` to move data between streams
- Use `ms.ToArray()` to get the written bytes
- Use `System.IO.Pipelines` for high-throughput network parsing

QUICK SUMMARY

- `Stream` — byte I/O abstraction
- `MemoryStream` / `FileStream` — common backings
- `BinaryReader` / `Writer` — primitives
- Chain wrappers like `GZipStream`

Namespaces & using

ORGANIZING CODE WITH NAMESPACES

Namespaces group related types and prevent name clashes (`MyApp.Data.Customer` vs `MyApp.Web.Customer`). A **`using` directive** imports a namespace so you can write short type names. C# 10 added **global usings** (once per project) and **file-scoped namespaces**; C# 12 lets **aliases** name any type, including tuples.

KEY FACTS

- Namespace ≈ folder path by convention
- `ImplicitUsings` adds System, IO, Linq, Tasks… for you
- `global::` starts lookup at the root namespace
- Namespaces don't control access — modifiers do

USING FORMS

```
// GlobalUsings.cs — applies to every file in the project
global using System.Text.Json;
global using MyApp.Domain;

// Customer.cs
using System.Collections.Concurrent;        // regular import
using static System.Math;                   // import static members
using Json = System.Text.Json.JsonSerializer;   // alias a type
using Point = (int X, int Y);               // alias a tuple (C# 12)

namespace MyApp.Services;                   // file-scoped (C# 10)

public class Geometry
{
    public double Dist(Point a, Point b) =>
        Sqrt(Pow(a.X - b.X, 2) + Pow(a.Y - b.Y, 2));  // Math via using static
    public string Save(Point p) => Json.Serialize(p);
    public global::System.DateTime Now => global::System.DateTime.Now;
}
```

CODE EXPLAINED

- → `global using` — one import for the whole project
- → `using X;` — short names for X's types
- → `using static` — call `Sqrt` without `Math.`
- → Alias — rename a type or tuple shape
- → `namespace X;` — whole file in X
- → `global::` — avoid ambiguity with local names

IMPLICIT USINGS (CONSOLE)

NAMESPACE	CONTAINS
`System`	Console, Math, DateTime
`System.Collections.Generic`	List, Dictionary
`System.IO`	File, Path, Stream
`System.Linq`	LINQ operators
`System.Net.Http`	HttpClient
`System.Threading.Tasks`	Task, async

PROJECT-LEVEL USINGS

```
<ItemGroup>
  <Using Include="MyApp.Domain" />
  <Using Include="System.Math" Static="true" />
  <Using Remove="System.Net.Http" />
</ItemGroup>
```

⚠ COMMON MISTAKES

- Naming a class the same as its namespace (`MyApp.Order.Order`) — causes confusing lookups
- Scattering `global using` across many files — keep them in one `GlobalUsings.cs`
- Hiding which namespace a type comes from with too many `using static` imports

✓ TIPS & BEST PRACTICES

- Match namespaces to folder structure
- Use aliases to resolve clashes like `Task` (yours vs System's)
- Use file-scoped namespaces in all new code

QUICK SUMMARY

- `namespace X;` — file-scoped
- `using` / `using static` / alias
- `global using` — project-wide
- `ImplicitUsings` — common namespaces auto-imported

NuGet Packages

PACKAGE MANAGEMENT

NuGet is .NET's package manager; nuget.org hosts hundreds of thousands of libraries. Packages are declared as `<PackageReference>` items in the project file and restored automatically on build. **Central Package Management** keeps every version in one `Directory.Packages.props` file so all projects agree.

KEY FACTS

- Restore happens automatically on build/run/test
- Packages cache in `~/.nuget/packages`
- Transitive dependencies come along automatically
- Lock files make restores reproducible

COMMANDS & FILES

```
dotnet add package Serilog                       # latest stable
dotnet add package Serilog --version 4.2.0       # exact version
dotnet add package Npgsql --prerelease           # include previews
dotnet remove package Serilog
dotnet list package --outdated                   # find upgrades
dotnet list package --vulnerable                 # security advisories
dotnet pack -c Release                           # build a .nupkg
dotnet nuget push bin/Release/*.nupkg -k $KEY -s nuget.org
```

CODE EXPLAINED

- → `add package` — writes a `PackageReference`
- → `--version` — pin a specific version
- → `--outdated` / `--vulnerable` — maintenance checks
- → `pack` — create a package from your library
- → `nuget push` — publish to a feed

DIRECTORY.PACKAGES.PROPS

```
<Project>
  <PropertyGroup>
    <ManagePackageVersionsCentrally>true</ManagePackageVersionsCentrally>
  </PropertyGroup>
  <ItemGroup>
    <PackageVersion Include="Serilog" Version="4.2.0" />
    <PackageVersion Include="xunit.v3" Version="1.0.0" />
  </ItemGroup>
</Project>
<!-- each .csproj then uses: <PackageReference Include="Serilog" /> -->
```

FILES INVOLVED

FILE	ROLE
`*.csproj`	PackageReference items
`Directory.Packages.props`	Central versions
`nuget.config`	Package sources / feeds
`packages.lock.json`	Locked dependency graph

VERSION SYNTAX

SYNTAX	MEANING
`1.2.3`	≥ 1.2.3 (lowest match)
`[1.2.3]`	Exactly 1.2.3
`[1.0,2.0)`	≥ 1.0 and < 2.0
`1.*`	Latest 1.x (floating)
`(,2.0]`	≤ 2.0

⚠ COMMON MISTAKES

- Different versions of the same package across projects — use Central Package Management
- Floating versions (`*`) in production — builds change without code changes
- Ignoring `--vulnerable` warnings (NU1901–NU1904) — update or override the transitive package

✓ TIPS & BEST PRACTICES

- Set `RestorePackagesWithLockFile` to `true` and restore with `--locked-mode` in CI
- Add private feeds in a repo-level `nuget.config`
- Check a package's downloads, license and last release before adopting it

QUICK SUMMARY

- `dotnet add package` — add dependency
- `PackageReference` — in the .csproj
- `Directory.Packages.props` — central versions
- `dotnet pack` / `nuget push` — publish

Math & Random

MATH AND RANDOM NUMBERS

System.Math provides rounding, powers, roots, trigonometry and clamping for `double` (with `decimal` overloads for rounding). Rounding defaults to **banker's rounding** (to even). **Random.Shared** is a thread-safe generator for everyday use; **RandomNumberGenerator** produces cryptographically secure values for tokens and passwords.

KEY FACTS

- `Math.Round(2.5)` → 2 (to even)
- `Random.Next(min, max)` excludes `max`
- `new Random(seed)` gives repeatable sequences
- `MathF` has `float` versions

MATH & RANDOM

```
Console.WriteLine(Math.Round(2.5));                          // 2
Console.WriteLine(Math.Round(2.5, MidpointRounding.AwayFromZero)); // 3
Console.WriteLine(Math.Round(2.345m, 2));                    // 2.34
Console.WriteLine(Math.Floor(-2.5));                         // -3
Console.WriteLine(Math.Ceiling(2.1));                        // 3
Console.WriteLine(Math.Truncate(-2.7));                      // -2
Console.WriteLine(Math.Clamp(150, 0, 100));                  // 100
Console.WriteLine(Math.Pow(2, 8));                           // 256
Console.WriteLine(Math.Sqrt(81));                            // 9

int die = Random.Shared.Next(1, 7);                          // 1-6
double r = Random.Shared.NextDouble();                       // [0, 1)
string[] colors = ["red", "green", "blue"];
string[] picks = Random.Shared.GetItems(colors, 5);          // with repeats
Random.Shared.Shuffle(colors);                               // in place

var seeded = new Random(42);                                 // reproducible
int token = RandomNumberGenerator.GetInt32(100_000, 1_000_000); // secure
string hex = RandomNumberGenerator.GetHexString(16);         // .NET 8
```

CODE EXPLAINED

- → `Round` — banker's rounding by default
- → `AwayFromZero` — school-style rounding
- → `Floor` / `Ceiling` / `Truncate` — directional rounding
- → `Clamp` — limit to a range
- → `Random.Shared` — thread-safe shared instance
- → `RandomNumberGenerator` — cryptographic randomness

MORE MATH

MEMBER	EXAMPLE → RESULT
`Math.Abs`	`Math.Abs(-5) → 5`
`Math.Max / Min`	`Math.Max(3, 7) → 7`
`Math.Log10 / Log2`	`Math.Log2(8) → 3`
`Math.PI / Math.E`	`3.14159… / 2.71828…`
`Math.Sin(rad)`	`Math.Sin(Math.PI / 2) → 1`

MIDPOINTROUNDING

MODE	2.5 / -2.5
`ToEven (default)`	`2 / -2`
`AwayFromZero`	`3 / -3`
`ToZero`	`2 / -2`
`ToPositiveInfinity`	`3 / -2`
`ToNegativeInfinity`	`2 / -3`

⚠ COMMON MISTAKES

- Expecting `Math.Round(2.5)` to be 3 — default is to-even; pass `AwayFromZero`
- Creating `new Random()` in a tight loop — use `Random.Shared` instead
- Using `Random` for passwords or tokens — predictable; use `RandomNumberGenerator`

✓ TIPS & BEST PRACTICES

- Round `decimal` for money: `Math.Round(x, 2, MidpointRounding.AwayFromZero)`
- Seed `Random` in tests for repeatable results
- Use `int.Clamp` / `double.Clamp` generic math helpers too

QUICK SUMMARY

- `Math.Round` — banker's by default
- `Floor` / `Ceiling` / `Clamp`
- `Random.Shared` — everyday randomness
- `RandomNumberGenerator` — secure randomness

Dates & Times

DATES, TIMES AND DURATIONS

DateTime stores a date and time with a Kind (Local, Utc, Unspecified); **DateTimeOffset** also stores the **offset from UTC**, so it identifies an exact instant — prefer it for timestamps. **DateOnly** and **TimeOnly** (.NET 6) represent just a date or a time, **TimeSpan** is a duration, and **TimeProvider** (.NET 8) makes "now" testable.

KEY FACTS

- Store and compare times in **UTC**
- Subtracting dates gives a `TimeSpan`
- IANA zone IDs work on all OSes (.NET 6+)
- Formatting is culture-sensitive unless you specify

WORKING WITH TIME

```
DateTimeOffset now = DateTimeOffset.UtcNow;              // exact instant
DateTime local = DateTime.Now;                           // machine local
DateOnly birthday = new(1990, 7, 4);
TimeOnly opens = new(9, 30);

var due = now.AddDays(14).AddHours(-2);                  // arithmetic
TimeSpan left = due - now;                               // 13.22:00:00
Console.WriteLine(left.TotalHours);                      // 334
Console.WriteLine(TimeSpan.FromMinutes(90));             // 01:30:00

var parsed = DateOnly.ParseExact("2026-10-11", "yyyy-MM-dd");
Console.WriteLine(parsed.DayOfWeek);                     // Sunday
Console.WriteLine(now.ToString("O"));                    // ISO 8601 round-trip

var chicago = TimeZoneInfo.FindSystemTimeZoneById("America/Chicago");
DateTimeOffset there = TimeZoneInfo.ConvertTime(now, chicago);

public class Invoice(TimeProvider clock)                 // testable "now"
{
    public bool IsOverdue(DateTimeOffset due) => clock.GetUtcNow() > due;
}
```

CODE EXPLAINED

- → `DateTimeOffset.UtcNow` — best for timestamps
- → `DateOnly` / `TimeOnly` — no time / no date
- → `AddDays`, `AddHours` — return new values (immutable)
- → `due - now` — `TimeSpan` difference
- → `ParseExact` — strict format parsing
- → `TimeProvider` — inject time; fake it in tests

TYPES

TYPE	REPRESENTS
`DateTime`	Date + time (+ Kind)
`DateTimeOffset`	Instant + UTC offset
`DateOnly`	Calendar date
`TimeOnly`	Time of day
`TimeSpan`	Duration
`TimeZoneInfo`	Zone rules & DST

FORMAT STRINGS

FORMAT	EXAMPLE
`O`	`2026-10-11T14:05:00.0000000+00:00`
`yyyy-MM-dd`	`2026-10-11`
`HH:mm:ss`	`14:05:00`
`dddd, MMM d`	`Sunday, Oct 11`
`TimeSpan @"hh\:mm"`	`01:30`

⚠ COMMON MISTAKES

- Storing `DateTime.Now` in databases — local times are ambiguous across zones and DST; store UTC
- Mixing `Kind.Local` and `Kind.Utc` values in arithmetic — results are silently wrong
- Calling `DateTime.Now` directly in business logic — untestable; inject `TimeProvider`

✓ TIPS & BEST PRACTICES

- Use `DateTimeOffset` for anything that happened at a moment
- Use `DateOnly` for birthdays and due dates
- Use `FakeTimeProvider` (Microsoft.Extensions.TimeProvider.Testing) in tests

QUICK SUMMARY

- `DateTimeOffset` — exact instants
- `DateOnly` / `TimeOnly` — partial values
- `TimeSpan` — durations
- `TimeProvider` — testable clock

Environment & Processes

THE PROCESS AND ITS ENVIRONMENT

Environment exposes information about the running process and machine: environment variables, processor count, OS, exit codes. **Process** starts and controls other programs, optionally capturing their output. **Stopwatch** measures elapsed time precisely, and **OperatingSystem** / **RuntimeInformation** detect the platform.

KEY FACTS

- Env var lookup returns `null` if not set
- `Environment.Exit(code)` ends the process immediately
- Redirected output must be read or the child can hang
- `Stopwatch` uses a high-resolution timer

ENVIRONMENT, PROCESS, STOPWATCH

```
string? env = Environment.GetEnvironmentVariable("ASPNETCORE_ENVIRONMENT");
string home = Environment.GetEnvironmentVariable("HOME") ?? "/";
Console.WriteLine(Environment.ProcessorCount);       // e.g. 10
Console.WriteLine(Environment.MachineName);
Console.WriteLine(OperatingSystem.IsMacOS());        // True on a Mac
Console.WriteLine(RuntimeInformation.FrameworkDescription); // .NET 10.0.x

var psi = new ProcessStartInfo("git")
{
    ArgumentList = { "status", "--short" },
    RedirectStandardOutput = true,
    WorkingDirectory = "/repo"
};
using var proc = Process.Start(psi)!;
string output = await proc.StandardOutput.ReadToEndAsync();
await proc.WaitForExitAsync();
Console.WriteLine($"exit {proc.ExitCode}: {output}");

var sw = Stopwatch.StartNew();
DoWork();
Console.WriteLine($"{sw.ElapsedMilliseconds} ms");
long t0 = Stopwatch.GetTimestamp();                  // allocation-free
Console.WriteLine(Stopwatch.GetElapsedTime(t0));     // TimeSpan (.NET 7)
```

CODE EXPLAINED

- → `GetEnvironmentVariable` — `null` when unset
- → `OperatingSystem.IsX()` — platform checks
- → `ArgumentList` — one entry per arg, no quoting issues
- → `RedirectStandardOutput` — capture output
- → `WaitForExitAsync` — await the child process
- → `Stopwatch` — timing; `GetElapsedTime` avoids allocation

ENVIRONMENT MEMBERS

MEMBER	GIVES
`Environment.CurrentDirectory`	Working directory
`Environment.NewLine`	\n or \r\n
`Environment.ExitCode`	Set exit code
`Environment.GetCommandLineArgs()`	All args incl. exe
`Environment.Is64BitProcess`	bool

PLATFORM CHECKS

API	EXAMPLE
`OperatingSystem.IsWindows()`	Feature switches
`OperatingSystem.IsLinux()`	Paths, services
`RuntimeInformation.OSArchitecture`	X64 / Arm64
`RuntimeInformation.OSDescription`	OS version string

⚠ COMMON MISTAKES

- Building one argument string with manual quotes — use the `ArgumentList` overload
- Redirecting output but not reading it — the child blocks when its buffer fills
- Timing code with `DateTime.Now` — low resolution; use `Stopwatch`

✓ TIPS & BEST PRACTICES

- Read configuration through `IConfiguration`, not raw env vars, in apps
- Use CliWrap for complex process piping
- Use BenchmarkDotNet for serious performance measurement

QUICK SUMMARY

- `Environment` — env vars, machine info
- `Process.Start` — run other programs
- `Stopwatch` — precise timing
- `OperatingSystem.IsX()` — platform checks

Configuration

APP CONFIGURATION

`Microsoft.Extensions.Configuration` merges settings from several **providers** — `appsettings.json`, an environment-specific JSON file, environment variables, user secrets and command-line args — into one key/value view. Later providers override earlier ones. The **Options pattern** binds sections to typed classes injected as `IOptions<T>`.

KEY FACTS

- Keys are hierarchical: `Smtp:Host`
- Env vars use `__` for `:` → `Smtp__Host`
- User secrets are for **development** only
- Host builders set all of this up automatically

CONFIGURATION & OPTIONS

```
// appsettings.json
// { "Smtp": { "Host": "mail.local", "Port": 25 }, "Retries": 3 }

var builder = Host.CreateApplicationBuilder(args);    // loads json, env, args
IConfiguration config = builder.Configuration;
string? host = config["Smtp:Host"];                   // "mail.local"
int retries = config.GetValue("Retries", defaultValue: 1);   // 3
var smtp = config.GetSection("Smtp").Get<SmtpOptions>();     // bind object

builder.Services.AddOptions<SmtpOptions>()
    .Bind(config.GetSection("Smtp"))
    .ValidateDataAnnotations()
    .ValidateOnStart();                               // fail fast
public class SmtpOptions
{
    [Required] public string Host { get; set; } = "";
    [Range(1, 65535)] public int Port { get; set; } = 25;
}

public class Mailer(IOptions<SmtpOptions> opts)       // inject
{
    public string Target => $"{opts.Value.Host}:{opts.Value.Port}";
}
```

CODE EXPLAINED

- → `CreateApplicationBuilder` — default providers wired up
- → `config["A:B"]` — read a raw string
- → `GetValue<T>` — typed with default
- → `Get<T>()` — bind a section to a class
- → `AddOptions().Bind().Validate…()` — options with validation
- → `IOptions<T>` — inject settings

DEFAULT PROVIDER ORDER (LAST WINS)

#	SOURCE
1	appsettings.json
2	appsettings.{Environment}.json
3	User secrets (Development)
4	Environment variables
5	Command-line args

OPTIONS INTERFACES

INTERFACE	RELOADS?	LIFETIME
`IOptions<T>`	No	Singleton
`IOptionsSnapshot<T>`	Per request	Scoped
`IOptionsMonitor<T>`	Live + OnChange	Singleton

⚠ COMMON MISTAKES

- Committing passwords in `appsettings.json` — use user secrets locally and a vault in production
- Using `:` in environment variable names — not valid everywhere; use `__`
- Reading `IConfiguration` everywhere — bind typed options once and inject them

✓ TIPS & BEST PRACTICES

- `dotnet user-secrets set "Smtp:Password" "…"` for local secrets
- Use `ValidateOnStart()` so bad config fails at startup
- Use `IOptionsMonitor<T>` for settings that change at run time

QUICK SUMMARY

- Providers merge; later wins
- `Section:Key` — hierarchical keys
- Options pattern — typed `IOptions<T>`
- Validate on start

Generic Host & Dependency Injection

GENERIC HOST AND DEPENDENCY INJECTION

The **Generic Host** (`Host.CreateApplicationBuilder`) bundles configuration, logging and a **dependency injection** container. You **register** services with a lifetime, and the container **constructs** them, passing dependencies into constructors. Long-running work runs as an `IHostedService` / `BackgroundService`. ASP.NET Core uses the same system.

KEY FACTS

- Depend on interfaces; register implementations
- Constructor injection is the default style
- Scopes are created per web request (or manually)
- Keyed services (.NET 8) register several implementations by key

HOST, SERVICES, WORKER

```
var builder = Host.CreateApplicationBuilder(args);
builder.Services.AddSingleton<IClock, SystemClock>();      // one for app
builder.Services.AddScoped<IOrderRepo, SqlOrderRepo>();    // one per scope
builder.Services.AddTransient<IEmailSender, SmtpSender>(); // new every time
builder.Services.AddKeyedSingleton<ICache, RedisCache>("redis");
builder.Services.AddHostedService<CleanupWorker>();
using IHost host = builder.Build();
await host.RunAsync();                                     // until Ctrl+C

public class CleanupWorker(IServiceScopeFactory scopes) : BackgroundService
{
    protected override async Task ExecuteAsync(CancellationToken stop)
    {
        using var timer = new PeriodicTimer(TimeSpan.FromMinutes(5));
        while (await timer.WaitForNextTickAsync(stop))
        {
            using var scope = scopes.CreateScope();          // scoped deps
            var repo = scope.ServiceProvider.GetRequiredService<IOrderRepo>();
            await repo.PurgeOldAsync();
        }
    }
}
```

CODE EXPLAINED

- → `AddSingleton/Scoped/Transient<I, T>()` — register with lifetime
- → `AddKeyedSingleton` — resolve with `[FromKeyedServices("redis")]`
- → `AddHostedService` — background work
- → `RunAsync` — runs until shutdown signal
- → Primary ctor — dependencies injected
- → `CreateScope()` — use scoped services from a singleton

LIFETIMES

LIFETIME	INSTANCE PER	TYPICAL USE
`Singleton`	Application	Caches, clients, config
`Scoped`	Scope / request	DbContext, unit of work
`Transient`	Resolution	Lightweight stateless

RULES OF THUMB

RULE	WHY
`No scoped inside singleton`	Captive dependency bug
`Resolve via ctor, not service locator`	Explicit dependencies

⚠ COMMON MISTAKES

- Injecting a scoped service (e.g. `DbContext`) into a singleton — it lives forever and isn't thread-safe
- Calling `GetService` everywhere (service locator) — hides dependencies; use constructor injection
- Registering the same interface twice by accident — the last registration wins for single resolution

✓ TIPS & BEST PRACTICES

- Group registrations in extension methods: `services.AddBilling()`
- Use `TryAdd*` in libraries so apps can override registrations
- Use `PeriodicTimer` for async loops in background services

QUICK SUMMARY

- `Host.CreateApplicationBuilder` — host + DI + config
- Singleton / Scoped / Transient lifetimes
- Constructor injection
- `BackgroundService` — long-running work

xUnit

UNIT TESTING WITH XUNIT

xUnit is the most widely used .NET test framework. A test is a public method marked **[Fact]**; **[Theory]** runs the same test with several data sets. A **new instance** of the test class is created per test, so the constructor is your setup and `Dispose` your teardown. **xUnit v3** (package `xunit.v3`) builds tests as executables.

KEY FACTS

- v3 templates: `dotnet new install xunit.v3.templates`
- Run: `dotnet test` (or the test exe in v3)
- `Assert.Equal(expected, actual)` — expected first
- Async tests return `Task` / `ValueTask`

FACTS, THEORIES, FIXTURES

```
public class CalculatorTests(DbFixture db) : IClassFixture<DbFixture>
{
    private readonly Calculator _calc = new();      // fresh per test

    [Fact]
    public void Add_TwoNumbers_ReturnsSum()
    {
        int result = _calc.Add(2, 3);               // Act
        Assert.Equal(5, result);                    // Assert
    }

    [Theory]
    [InlineData(2, 2, 4)]
    [InlineData(-1, 1, 0)]
    public void Add_Cases(int a, int b, int expected) =>
        Assert.Equal(expected, _calc.Add(a, b));

    [Fact]
    public async Task Divide_ByZero_Throws() =>
        await Assert.ThrowsAsync<DivideByZeroException>(
            () => _calc.DivAsync(1, 0));
}
```

CODE EXPLAINED

- → `IClassFixture<T>` — one shared fixture for all tests in the class
- → Constructor — runs before **each** test
- → `[Fact]` — single test case
- → `[Theory]` + `[InlineData]` — parameterized test
- → `Assert.Equal(expected, actual)`
- → `ThrowsAsync<T>` — assert an async exception

COMMON ASSERTS

ASSERT	CHECKS
`Equal / NotEqual`	Values (collections too)
`True / False`	Condition
`Null / NotNull`	Reference
`Contains / DoesNotContain`	Item or substring
`Throws<T>(() => …)`	Exact exception type

RUNNING TESTS

COMMAND	DOES
`dotnet test`	Build + run all
`dotnet test --filter Name~Add`	Name contains "Add"
`dotnet test --filter Category=Slow`	By [Trait]
`dotnet watch test`	Re-run on save

⚠ COMMON MISTAKES

- Swapping `Assert.Equal` arguments — failure messages show expected/actual reversed
- Sharing state in static fields between tests — tests run in parallel across classes and interfere
- `async void` test methods — failures can be missed; return `Task`

✓ TIPS & BEST PRACTICES

- Name tests `Method_Scenario_Expected`
- Follow Arrange-Act-Assert with blank lines between
- Use `TestContext.Current.CancellationToken` (v3) in async tests

QUICK SUMMARY

- `[Fact]` — single test
- `[Theory]` + `[InlineData]` — data-driven
- Constructor / `Dispose` — setup / teardown
- `IClassFixture<T>` — shared expensive setup

NUnit & MSTest

NUNIT AND MSTEST

NUnit and **MSTest** are the other two mainstream .NET test frameworks. Concepts map closely to xUnit — test attributes, data-driven cases, setup/teardown — but the attribute names and assertion styles differ. NUnit is known for its fluent **constraint model** (`Assert.That(x, Is.EqualTo(5))`); MSTest is Microsoft's own, built into Visual Studio.

KEY FACTS

- Templates: `dotnet new nunit` / `dotnet new mstest`
- Both run with `dotnet test`
- Both support the **Microsoft.Testing.Platform** runner
- NUnit reuses one fixture instance for all tests in a class

SAME TEST, TWO FRAMEWORKS

```
[TestFixture]                                          // NUnit
public class CartTests
{
    private Cart _cart = null!;
    [SetUp] public void Init() => _cart = new Cart();      // before each
    [TestCase(1, 10.0, 10.0)]
    [TestCase(3, 2.5, 7.5)]
    public void Total_IsQtyTimesPrice(int qty, double price, double total)
    {
        _cart.Add("pen", qty, price);
        Assert.That(_cart.Total, Is.EqualTo(total).Within(0.001));
    }
}
[TestClass]                                            // MSTest
public class CartTestsMs
{
    [TestMethod, DataRow(1, 10.0, 10.0)]
    public void Total(int qty, double price, double total) =>
        Assert.AreEqual(total, Cart.Of("pen", qty, price).Total, 0.001);
}
```

CODE EXPLAINED

- → `[TestFixture]` / `[TestClass]` — test class
- → `[SetUp]` / `[TestInitialize]` — before each test
- → `[TestCase]` / `[DataRow]` — inline data
- → `Assert.That(x, Is.EqualTo(y))` — NUnit constraints
- → `.Within(0.001)` — tolerance for doubles
- → `Assert.AreEqual(expected, actual, delta)` — MSTest

ATTRIBUTE MAP

XUNIT	NUNIT	MSTEST
`[Fact]`	`[Test]`	`[TestMethod]`
`[Theory]`	`[TestCase]`	`[DataRow]`
`ctor`	`[SetUp]`	`[TestInitialize]`
`Dispose`	`[TearDown]`	`[TestCleanup]`
`IClassFixture`	`[OneTimeSetUp]`	`[ClassInitialize]`

NUNIT CONSTRAINTS

CONSTRAINT	CHECKS
`Is.EqualTo(5)`	Equality
`Is.Not.Null`	Not null
`Has.Count.EqualTo(3)`	Collection size
`Throws.TypeOf<E>()`	Exception type

⚠ COMMON MISTAKES

- Keeping state in NUnit fixture fields without `[SetUp]` — the instance is shared across tests
- Using MSTest's `ExpectedException` attribute — passes if any line throws; use `Assert.ThrowsExactly<T>`
- Mixing frameworks in one test project — pick one per project

✓ TIPS & BEST PRACTICES

- Choose by team familiarity; capabilities are similar
- Use `MSTest.Sdk` for the simplest MSTest project setup
- Use `[Parallelizable]` (NUnit) to speed up suites

QUICK SUMMARY

- NUnit — `[Test]`, `Assert.That` constraints
- MSTest — `[TestMethod]`, `Assert.AreEqual`
- Both — data rows, setup/teardown, `dotnet test`
- Attribute map ↔ xUnit

Mocking & Assertion Libraries

NSubstitute · Moq · Substitute.For · Returns · Received · Shouldly · AwesomeAssertions / FluentAssertions licensing

TEST DOUBLES AND READABLE ASSERTS

Mocking libraries create fake implementations of interfaces so a unit test can isolate the class under test and verify interactions. **NSubstitute** and **Moq** are the most common. **Assertion libraries** like **Shouldly** or **AwesomeAssertions** give fluent, readable assertions with clearer failure messages.

KEY FACTS

- Mock interfaces or virtual members only
- FluentAssertions v8+ needs a paid license for commercial use
- AwesomeAssertions is the Apache-licensed community fork
- Fake only what you own; wrap third-party APIs

NSUBSTITUTE, MOQ, SHOULDLY

```
// ── NSubstitute ──────────────────────────────────────
var repo = Substitute.For<IOrderRepo>();
repo.Find(1).Returns(new Order(1, 50m));            // stub
var svc = new OrderService(repo);
svc.Cancel(1);
repo.Received(1).Save(Arg.Is<Order>(o => o.Status == Status.Cancelled));

// ── Moq ──────────────────────────────────────────────
var mock = new Mock<IOrderRepo>();
mock.Setup(r => r.Find(1)).Returns(new Order(1, 50m));
new OrderService(mock.Object).Cancel(1);
mock.Verify(r => r.Save(It.IsAny<Order>()), Times.Once);

// ── Shouldly / AwesomeAssertions ─────────────────────
order.Total.ShouldBe(50m);                          // Shouldly
order.Items.ShouldContain(i => i.Sku == "A1");
order.Total.Should().Be(50m);                       // AwesomeAssertions
act.Should().Throw<InvalidOperationException>().WithMessage("*closed*");
```

CODE EXPLAINED

- → `Substitute.For<T>()` — create a fake
- → `.Returns(...)` — stub a return value
- → `Received(1)` + `Arg.Is` — verify a call with matching args
- → `Mock<T>.Setup/Verify` — Moq equivalents
- → `ShouldBe` — Shouldly assertion
- → `.Should().Be` — FluentAssertions-style API

NSUBSTITUTE VS MOQ

TASK	NSUBSTITUTE	MOQ
Create	`Substitute.For<I>()`	`new Mock<I>()`
Stub	`x.M(1).Returns(v)`	`m.Setup(x => x.M(1)).Returns(v)`
Any arg	`Arg.Any<T>()`	`It.IsAny<T>()`
Verify	`x.Received().M()`	`m.Verify(x => x.M())`
Throw	`x.M().Throws(ex)`	`m.Setup(…).Throws(ex)`

⚠ COMMON MISTAKES

- Mocking everything, including value objects — tests mirror the implementation and break on refactors
- Verifying every call — over-specified tests; verify only what matters
- Upgrading to FluentAssertions 8 in a commercial project without a license

✓ TIPS & BEST PRACTICES

- Prefer in-memory fakes for repositories when practical
- Use `FakeTimeProvider` and `FakeLogger` from Microsoft.Extensions.*.Testing
- Keep one assertion style across the codebase

QUICK SUMMARY

- NSubstitute / Moq — fakes for interfaces
- Stub returns, verify calls
- Shouldly / AwesomeAssertions — fluent asserts
- FluentAssertions 8+ — commercial license

Logging

STRUCTURED LOGGING

`Microsoft.Extensions.Logging` is the standard logging abstraction. Classes receive an **`ILogger<T>`** through DI and write messages at a **log level**. Message **templates** like `"Order {OrderId} placed"` produce **structured** logs where `OrderId` is a searchable field, not just text. Providers (console, Serilog, OpenTelemetry…) decide where logs go.

KEY FACTS

- Levels: Trace < Debug < Information < Warning < Error < Critical
- Placeholders are named, not interpolated
- Filter levels per category in `appsettings.json`
- `[LoggerMessage]` source gen avoids allocations

LOGGING IN A SERVICE

```
public partial class OrderService(ILogger<OrderService> log)
{
    public void Place(Order o)
    {
        log.LogInformation("Placing {OrderId} for {Total}", o.Id, o.Total);
        try { Save(o); }
        catch (Exception ex)
        {
            log.LogError(ex, "Save failed for {OrderId}", o.Id); // ex first
            throw;
        }
        using (log.BeginScope("Batch {BatchId}", o.BatchId))   // scope data
            OrderPlaced(log, o.Id);
    }
    [LoggerMessage(LogLevel.Information, "Order {OrderId} placed")]
    static partial void OrderPlaced(ILogger logger, int orderId); // generated
}
builder.Services.AddSerilog(lc => lc.WriteTo.Console());
```

CODE EXPLAINED

- → `ILogger<T>` — category = the class name
- → `{OrderId}` — structured field
- → `LogError(ex, …)` — exception goes first
- → `BeginScope` — attach data to nested logs
- → `[LoggerMessage]` — fast, generated log method
- → `AddSerilog` — plug in Serilog (Serilog.Extensions.Hosting)

LOG LEVELS

LEVEL	USE FOR
`Trace`	Very detailed internals
`Debug`	Developer diagnostics
`Information`	Normal business events
`Warning`	Unexpected but handled
`Error`	Operation failed
`Critical`	App-wide failure

POPULAR SINKS / PROVIDERS

PROVIDER	NOTES
`Console / Debug`	Built in
`Serilog`	Rich sinks: file, Seq, Elastic
`NLog`	Mature, config-driven
`OpenTelemetry`	Logs + traces + metrics
`Application Insights`	Azure monitoring

⚠ COMMON MISTAKES

- Interpolating messages `$"Order {id}"` — loses structure and allocates even when disabled
- Logging secrets or personal data — logs are widely accessible
- Logging and rethrowing at every layer — the same error appears many times

✓ TIPS & BEST PRACTICES

- Use PascalCase placeholder names consistently (`{OrderId}`)
- Use `[LoggerMessage]` in hot paths
- Check `log.IsEnabled(LogLevel.Debug)` before expensive log data

QUICK SUMMARY

- `ILogger<T>` — inject and log
- Templates — `{Name}` placeholders
- Levels — Trace → Critical
- Serilog / OpenTelemetry — providers

HttpClient

CALLING HTTP APIS

`HttpClient` sends HTTP requests and receives responses asynchronously. Create clients through **`IHttpClientFactory`** (`AddHttpClient`) so connections are pooled and DNS changes are respected. `System.Net.Http.Json` extensions like **`GetFromJsonAsync`** and **`PostAsJsonAsync`** handle serialization, and **`Microsoft.Extensions.Http.Resilience`** adds retries and timeouts.

KEY FACTS

- Don't create `new HttpClient()` per request
- Non-2xx responses don't throw unless you ask
- Default timeout is 100 seconds
- Typed clients wrap an API in a class

TYPED CLIENT WITH RESILIENCE

```
builder.Services.AddHttpClient<GitHubClient>(c =>
{
    c.BaseAddress = new Uri("https://api.github.com/");
    c.DefaultRequestHeaders.UserAgent.ParseAdd("my-app/1.0");
})
.AddStandardResilienceHandler();                    // retry, timeout, breaker

public class GitHubClient(HttpClient http)
{
    public Task<Repo?> GetRepo(string owner, string name) =>
        http.GetFromJsonAsync<Repo>($"repos/{owner}/{name}");

    public async Task<int> CreateIssue(string repo, NewIssue issue)
    {
        var url = $"repos/{repo}/issues";
        using var resp = await http.PostAsJsonAsync(url, issue);
        if (resp.StatusCode == HttpStatusCode.NotFound) return -1;
        resp.EnsureSuccessStatusCode();             // throw on other errors
        var created = await resp.Content.ReadFromJsonAsync<Issue>();
        return created!.Number;
    }
}
```

CODE EXPLAINED

- → `AddHttpClient<T>` — typed client via the factory
- → `BaseAddress` — relative URLs after this
- → `AddStandardResilienceHandler` — retries + timeouts
- → `GetFromJsonAsync<T>` — GET + deserialize
- → `PostAsJsonAsync` — serialize + POST
- → `EnsureSuccessStatusCode` — throw for non-2xx

HTTPCLIENT METHODS

METHOD	RETURNS
`GetStringAsync(url)`	string
`GetFromJsonAsync<T>(url)`	T?
`PostAsJsonAsync(url, obj)`	HttpResponseMessage
`PutAsJsonAsync / DeleteAsync`	HttpResponseMessage
`SendAsync(request)`	Full control

HTTPRESPONSEMESSAGE

MEMBER	USE
`StatusCode`	HttpStatusCode enum
`IsSuccessStatusCode`	2xx?
`Headers / Content.Headers`	Response headers
`Content.ReadFromJsonAsync<T>()`	Parse body
`EnsureSuccessStatusCode()`	Throw HttpRequestException

⚠ COMMON MISTAKES

- `using var client = new HttpClient()` per call — exhausts sockets under load
- A static `HttpClient` without `PooledConnectionLifetime` — misses DNS changes
- Ignoring status codes — `GetFromJsonAsync` throws on non-2xx, but `PostAsJsonAsync` doesn't

✓ TIPS & BEST PRACTICES

- Pass `CancellationToken` to every HTTP call
- Use typed clients to keep API details in one class
- Stream large downloads with `ResponseHeadersRead`

QUICK SUMMARY

- `IHttpClientFactory` — managed clients
- `GetFromJsonAsync` / `PostAsJsonAsync`
- `EnsureSuccessStatusCode` — error handling
- Resilience handler — retries & timeouts

Regular Expressions

PATTERN MATCHING ON TEXT

`System.Text.RegularExpressions.Regex` finds, validates, extracts and replaces text with regular expressions. Use **verbatim** or **raw** string literals so backslashes stay readable. For patterns known at compile time, **`[GeneratedRegex]`** (.NET 7) generates optimized code — faster startup and AOT-friendly.

KEY FACTS

- Static `Regex.IsMatch` caches recent patterns
- Named groups: `(?<year>\d{4})`
- Replacement tokens: `$1` , `${name}`
- Set a timeout for untrusted input

REGEX IN PRACTICE

```
string log = "2026-10-11 ERROR [db] timeout; 2026-10-12 WARN [api] slow";

bool hasError = Regex.IsMatch(log, @"\bERROR\b");            // True

Match m = Regex.Match(log, @"(?<date>\d{4}-\d{2}-\d{2}) (?<lvl>\w+)");
Console.WriteLine(m.Groups["date"].Value);                   // 2026-10-11
Console.WriteLine(m.Groups["lvl"].Value);                    // ERROR

foreach (Match x in Regex.Matches(log, @"\[(\w+)\]"))
    Console.Write(x.Groups[1].Value + " ");                  // db api

string iso = Regex.Replace("11/10/2026", @"(\d+)/(\d+)/(\d+)", "$3-$2-$1");
                                                             // 2026-10-11
string[] parts = Regex.Split("a1b22c", @"\d+");              // [a, b, c]

public static partial class Patterns
{
    [GeneratedRegex(@"^[\w.+-]+@[\w-]+\.[\w.]+$", RegexOptions.IgnoreCase)]
    public static partial Regex Email();
}
bool ok = Patterns.Email().IsMatch("ada@example.com");       // True
```

CODE EXPLAINED

- → `@"…"` — no double-escaping of `\`
- → `IsMatch` — yes/no test
- → `(?<name>…)` + `Groups["name"]` — named capture
- → `Matches` — all occurrences
- → `$3-$2-$1` — reorder captured groups
- → `[GeneratedRegex]` — source-generated, cached instance

PATTERN SYNTAX

TOKEN	MATCHES
`\d \w \s`	Digit, word char, whitespace
`\D \W \S`	Negations
`. ^ $`	Any char, start, end
`* + ? {2,4}`	Quantifiers
`[abc] [^abc]`	Set / negated set
`(…) (?:…) (?<n>…)`	Group, non-capturing, named

REGEXOPTIONS

OPTION	EFFECT
`IgnoreCase`	Case-insensitive
`Multiline`	^ $ match each line
`Singleline`	. matches \n
`ExplicitCapture`	Only named groups capture
`NonBacktracking`	Linear time, no backrefs

⚠ COMMON MISTAKES

- Writing `"\d+"` in a normal string — the C# escape eats the backslash; use `@"\d+"`
- Running user-supplied patterns without a timeout — catastrophic backtracking can hang (ReDoS)
- Creating `new Regex(...)` in a loop — cache it or use `[GeneratedRegex]`

✓ TIPS & BEST PRACTICES

- Pass `TimeSpan.FromSeconds(1)` as `matchTimeout` for untrusted input
- Prefer string methods (`Contains` , `Split`) for simple cases
- Test patterns with regex101.com (.NET flavor)

QUICK SUMMARY

- `IsMatch` / `Match` / `Matches`
- Named groups — `(?<n>…)`
- `Replace` with `$1` / `${n}`
- `[GeneratedRegex]` — fast, AOT-safe

async / await — Basics

ASYNCHRONOUS CODE

async/await lets methods wait for I/O (network, disk, databases) **without blocking a thread**. An `async` method returns a **Task** (no result) or **Task<T>**; `await` pauses the method until the task completes and then resumes, usually on a thread-pool thread. Async is about **scalability and responsiveness**, not making CPU work faster.

KEY FACTS

- `async` methods run synchronously until the first incomplete `await`
- Exceptions are stored in the task and rethrown at `await`
- Name async methods with an `Async` suffix
- `Main` can be `async Task` (or use top-level `await`)

ASYNC / AWAIT BASICS

```
static async Task<string> FetchTitleAsync(HttpClient http, string url)
{
    string html = await http.GetStringAsync(url);  // thread free while waiting
    int start = html.IndexOf("<title>") + 7;
    return html[start..html.IndexOf("</title>")];
}

static async Task SaveAsync(string path, string text)  // Task = no result
{
    await File.WriteAllTextAsync(path, text);
    Console.WriteLine("saved");
}

using var http = new HttpClient();
string title = await FetchTitleAsync(http, "https://example.com");
await SaveAsync("title.txt", title);

Task<string> pending = FetchTitleAsync(http, "https://dot.net");   // starts now
DoOtherWork();                                                     // runs meanwhile
string second = await pending;                                     // then wait

ValueTask<int> cached = GetCountAsync();      // cheaper when often synchronous
```

CODE EXPLAINED

- → `async Task<T>` — returns a value later
- → `await` — suspend without blocking the thread
- → `async Task` — like `void`, but awaitable
- → Top-level `await` — allowed in Program.cs
- → Start first, await later — overlap work
- → `ValueTask<T>` — avoids allocation on fast paths

RETURN TYPES

TYPE	USE
`Task`	Async work, no result
`Task<T>`	Async result
`ValueTask / ValueTask<T>`	Hot paths, often sync
`IAsyncEnumerable<T>`	Async stream
`async void`	Event handlers only

SYNC → ASYNC EQUIVALENTS

BLOCKING	ASYNC
`File.ReadAllText`	`File.ReadAllTextAsync`
`Thread.Sleep(ms)`	`await Task.Delay(ms)`
`stream.Read`	`await stream.ReadAsync`
`db.SaveChanges()`	`await db.SaveChangesAsync()`
`task.Result`	`await task`

⚠ COMMON MISTAKES

- Forgetting `await` — the method continues before the work finishes and errors are lost
- Using `async` for CPU-bound work without `Task.Run` — it still blocks the caller
- Mixing `.Result` / `.Wait()` with async code — can deadlock and wraps errors

✓ TIPS & BEST PRACTICES

- Go "async all the way" — from controller to database
- Accept and pass a `CancellationToken`
- Return the task directly (`=> http.GetAsync(u)`) when no `await` is needed

QUICK SUMMARY

- `async Task<T>` + `await`
- Non-blocking I/O, not faster CPU
- Start tasks, await later to overlap
- `ValueTask` for hot paths

async / await — Patterns

COORDINATING TASKS

Real programs run several async operations at once and need to **combine**, **race**, **cancel** and **time out** them. `Task.WhenAll` waits for everything, `Task.WhenAny` for the first, and `Task.WhenEach` (.NET 9) yields tasks **as they complete**. **`CancellationToken`** is the cooperative way to stop work, and `WaitAsync` adds a timeout to any task.

KEY FACTS

- `WhenAll` throws the first exception at `await`
- Cancellation is cooperative — code must check the token
- `Task.Run` offloads CPU work to the thread pool
- `CancellationTokenSource` must be disposed

COMBINING & CANCELLING

```
using var cts = new CancellationTokenSource(TimeSpan.FromSeconds(10));
CancellationToken ct = cts.Token;

string[] urls = ["https://a.com", "https://b.com", "https://c.com"];
var downloads = urls.Select(u => http.GetStringAsync(u, ct)).ToArray();

string[] pages = await Task.WhenAll(downloads);          // all, in parallel

Task<string> first = await Task.WhenAny(downloads);      // fastest one

await foreach (Task<string> done in Task.WhenEach(downloads))   // .NET 9
    Console.WriteLine($"finished: {(await done).Length}");

int hash = await Task.Run(() => ExpensiveHash(data), ct); // CPU work off-thread

try { await SlowOpAsync().WaitAsync(TimeSpan.FromSeconds(2)); }  // timeout
catch (TimeoutException) { Console.WriteLine("too slow"); }

var opts = new ParallelOptions { MaxDegreeOfParallelism = 2 };   // throttled
await Parallel.ForEachAsync(urls, opts,
    async (u, tok) => await http.GetStringAsync(u, tok));
```

CODE EXPLAINED

- → `CancellationTokenSource(timeout)` — auto-cancel
- → Start all tasks first, then `WhenAll`
- → `WhenAny` — returns the completed task
- → `WhenEach` — process results in completion order
- → `Task.Run` — CPU-bound work
- → `WaitAsync` — timeout without cancelling the work itself

COMBINATORS

API	COMPLETES WHEN
`Task.WhenAll(ts)`	All done (or one faults)
`Task.WhenAny(ts)`	First done
`Task.WhenEach(ts)`	Yields each as done
`Task.Delay(t, ct)`	Timer elapses
`task.WaitAsync(t)`	Task done or timeout

CANCELLATION TOOLKIT

API	USE
`ct.ThrowIfCancellationRequested()`	Check in loops
`cts.Cancel() / CancelAfter(t)`	Trigger
`CreateLinkedTokenSource(a, b)`	Combine tokens
`ct.Register(callback)`	React to cancel
`OperationCanceledException`	Thrown on cancel

⚠ COMMON MISTAKES

- Awaiting tasks one by one in a loop when they're independent — serial instead of parallel
- Firing thousands of requests with `WhenAll` — throttle with `Parallel.ForEachAsync` or `SemaphoreSlim`
- Accepting a `CancellationToken` but never passing it down

✓ TIPS & BEST PRACTICES

- Make `CancellationToken ct = default` the last parameter
- Use `WhenEach` to show progress as results arrive
- Inspect `task.Exception` after `WhenAll` to see every failure

QUICK SUMMARY

- `WhenAll` / `WhenAny` / `WhenEach`
- `CancellationToken` — cooperative cancel
- `Task.Run` — CPU-bound offload
- `WaitAsync` — timeouts

async / await — Pitfalls

async void · .Result / .Wait() deadlocks · ConfigureAwait(false) · fire-and-forget · WhenAll exceptions · await foreach

ASYNC TRAPS TO AVOID

Most async bugs come from a few patterns: **`async void`**, **blocking on tasks** (`.Result`, `.Wait()`), **fire-and-forget** without error handling, and misunderstanding where code resumes. Library code adds **`ConfigureAwait(false)`** to avoid capturing a UI or legacy ASP.NET context. These rules keep async code correct and debuggable.

KEY FACTS

- ASP.NET Core has no sync context — deadlocks are rarer there
- UI apps (WPF, WinForms, MAUI) resume on the UI thread
- Unobserved task exceptions are silently lost
- `await using` / `await foreach` exist for async resources

WRONG VS RIGHT

```
async void Save() { await db.SaveChangesAsync(); }     // ✗ crashes on error
async Task SaveAsync() { await db.SaveChangesAsync(); } // ✓ awaitable

var data = LoadAsync().Result;                          // ✗ blocks, may deadlock
var data2 = await LoadAsync();                          // ✓

// ✗ fire-and-forget: errors vanish
_ = SendEmailAsync(user);
// ✓ queue it to a background service, or at least observe errors
_ = SendEmailAsync(user).ContinueWith(t => log.LogError(t.Exception, "email"),
        TaskContinuationOptions.OnlyOnFaulted);

// Library code: don't resume on the caller's context
public async Task<string> ReadAsync(string p) =>
    await File.ReadAllTextAsync(p).ConfigureAwait(false);

// ✗ lock + await isn't allowed; ✓ use SemaphoreSlim
await _gate.WaitAsync();
try { await WriteAsync(); } finally { _gate.Release(); }
```

CODE EXPLAINED

- → `async void` — only for event handlers
- → `.Result` — blocks a thread, may deadlock
- → `_ =` — discards the task; exceptions unobserved
- → `ContinueWith(OnlyOnFaulted)` — minimal error logging
- → `ConfigureAwait(false)` — for libraries
- → `SemaphoreSlim` — async-friendly lock

RULES

DO	DON'T
`async Task`	async void
`await task`	.Result / .Wait()
`Task.Delay`	Thread.Sleep
`SemaphoreSlim.WaitAsync`	lock around await
`Background queue / service`	Unobserved fire-and-forget
`Pass CancellationToken`	Ignore it

SYMPTOMS → CAUSE

SYMPTOM	LIKELY CAUSE
`UI freezes`	Blocking on a task
`App hangs forever`	Sync-over-async deadlock
`Crash with no stack`	async void exception
`Missing side effects`	Missing await
`Thread-pool starvation`	Many blocking waits

⚠ COMMON MISTAKES

- Wrapping sync APIs in `Task.Run` inside libraries — let callers decide
- Returning `null` from a `Task`-returning method — callers' `await` throws; return `Task.CompletedTask`
- Assuming `await` always switches threads — it may complete synchronously

✓ TIPS & BEST PRACTICES

- Turn on analyzers (CA2007, VSTHRD*) to catch async misuse
- Use `IHostedService` + `Channel<T>` for background jobs
- In ASP.NET Core apps `ConfigureAwait(false)` is optional

QUICK SUMMARY

- No `async void` (except events)
- No `.Result` / `.Wait()`
- Observe fire-and-forget tasks
- `SemaphoreSlim` instead of `lock` with await

Threads & Synchronization

THREADS AND SYNCHRONIZATION

When several threads touch shared data, you need **synchronization**. `lock` gives one thread exclusive access to a block; .NET 9 adds a dedicated **`System.Threading.Lock`** type that `lock` uses efficiently. **`Interlocked`** performs atomic updates on numbers, and **`SemaphoreSlim`** limits concurrency and works with `await`. Most apps use the **thread pool** via tasks rather than raw `Threads`.

KEY FACTS

- Lock on a private, dedicated object — never `this` or a string
- You can't `await` inside a `lock` block
- Keep locked sections short
- `volatile` only stops reordering; it's not a lock

LOCKS, ATOMICS, SEMAPHORES

```
public class Counter
{
    private readonly Lock _gate = new();          // .NET 9 / C# 13
    private int _value;
    public void Add(int n) { lock (_gate) { _value += n; } }
    public int Value { get { lock (_gate) return _value; } }
}

private static int _hits;
Interlocked.Increment(ref _hits);                       // atomic ++
Interlocked.Add(ref _hits, 5);
int old = Interlocked.CompareExchange(ref _hits, 0, 10); // set 0 if it was 10

var throttle = new SemaphoreSlim(initialCount: 3);      // max 3 at once
await Parallel.ForEachAsync(files, async (f, ct) =>
{
    await throttle.WaitAsync(ct);
    try { await UploadAsync(f, ct); } finally { throttle.Release(); }
});

var t = new Thread(() => Console.WriteLine("raw")) { IsBackground = true };
t.Start(); t.Join();                                    // rarely needed
```

CODE EXPLAINED

- → `Lock` + `lock` — mutual exclusion
- → Lock in getters too — consistent reads
- → `Interlocked` — lock-free numeric updates
- → `CompareExchange` — atomic compare-and-swap
- → `SemaphoreSlim` — N concurrent holders; async wait
- → `Thread` — manual threads; prefer tasks

PRIMITIVES

TYPE	USE
`lock / Lock`	Short exclusive sections
`Interlocked`	Counters, flags
`SemaphoreSlim`	Throttle; async-safe
`ReaderWriterLockSlim`	Many readers, few writers
`ManualResetEventSlim`	Signal threads
`Monitor.Wait / Pulse`	Low-level coordination

HAZARDS

PROBLEM	CAUSE
`Race condition`	Unsynchronized shared writes
`Deadlock`	Locks taken in different orders
`Lock contention`	Long work inside lock
`Starvation`	Blocking pool threads

⚠ COMMON MISTAKES

- `lock (this)` or `lock ("key")` — outside code can lock the same object and deadlock you
- `_count++` from several threads — not atomic; use `Interlocked.Increment`
- Calling out to unknown code while holding a lock — invites deadlocks

✓ TIPS & BEST PRACTICES

- Prefer immutable data and message passing over shared state
- Use concurrent collections instead of locking around `Dictionary`
- Acquire multiple locks in a consistent global order

QUICK SUMMARY

- `lock (_gate)` — exclusive section
- `Lock` type — .NET 9
- `Interlocked` — atomic numbers
- `SemaphoreSlim` — async throttling

Parallelism & Channels

DATA PARALLELISM AND PIPELINES

The **Task Parallel Library** spreads CPU-bound loops across cores with `Parallel.For` / `ForEach`, and `Parallel.ForEachAsync` handles async work with a concurrency limit. **PLINQ** (`AsParallel()`) parallelizes LINQ queries. **Channels** (`System.Threading.Channels`) are async producer/consumer queues for building pipelines between tasks.

KEY FACTS

- Parallel loops only help for CPU-heavy, independent work
- Iterations run in no particular order
- Bounded channels apply **back-pressure** to producers
- Complete the writer so readers can finish

PARALLEL & CHANNELS

```
long[] results = new long[1000];
Parallel.For(0, results.Length, i => results[i] = Fib(i % 40));   // CPU-bound

var opts = new ParallelOptions { MaxDegreeOfParallelism = 4 };
await Parallel.ForEachAsync(urls, opts, async (url, ct) =>
    await DownloadAsync(url, ct));                                  // async, max 4

int primes = Enumerable.Range(2, 5_000_000)
    .AsParallel()
    .Count(IsPrime);                                                // PLINQ

var channel = Channel.CreateBounded<string>(capacity: 100);
var producer = Task.Run(async () =>
{
    foreach (var line in File.ReadLines("big.csv"))
        await channel.Writer.WriteAsync(line);                      // waits if full
    channel.Writer.Complete();                                      // no more items
});
var consumer = Task.Run(async () =>
    { await foreach (var l in channel.Reader.ReadAllAsync()) Process(l); });
await Task.WhenAll(producer, consumer);
```

CODE EXPLAINED

- → `Parallel.For` — split a range across cores
- → `MaxDegreeOfParallelism` — cap concurrency
- → `ForEachAsync` — parallel async I/O
- → `AsParallel()` — PLINQ query
- → `CreateBounded` — queue with back-pressure
- → `Complete()` + `ReadAllAsync()` — clean shutdown

WHICH TOOL?

WORKLOAD	USE
`CPU loop over data`	Parallel.For / ForEach
`Many async I/O calls`	Parallel.ForEachAsync
`CPU-heavy LINQ query`	PLINQ AsParallel()
`Producer → consumer stages`	Channel<T>
`One background job`	Task.Run

CHANNEL API

MEMBER	USE
`Channel.CreateUnbounded<T>()`	No limit
`Channel.CreateBounded<T>(n)`	Limit + back-pressure
`Writer.WriteAsync / TryWrite`	Produce
`Reader.ReadAllAsync()`	Consume as a stream
`Writer.Complete()`	Signal the end

⚠ COMMON MISTAKES

- Using `Parallel.For` for I/O — it blocks pool threads; use `ForEachAsync`
- Writing to a shared `List<T>` from parallel iterations — not thread-safe; write by index or use concurrent collections
- Forgetting `Writer.Complete()` — consumers wait forever

✓ TIPS & BEST PRACTICES

- Measure first — parallel overhead can make small loops slower
- Use `AsOrdered()` when PLINQ output order matters
- Run multiple consumers on one channel to scale processing

QUICK SUMMARY

- `Parallel.For` — CPU parallelism
- `ForEachAsync` — throttled async
- PLINQ — `AsParallel()`
- `Channel<T>` — async pipelines

Span, Memory & Performance

ZERO-COPY AND LOW-ALLOCATION CODE

Span<T> and **ReadOnlySpan<T>** are views over contiguous memory — arrays, strings, stack memory or native buffers — that slice without copying. Because they're `ref structs` they live only on the stack; **Memory<T>** is the heap-storable counterpart for async code. **stackalloc** and **ArrayPool<T>** avoid GC allocations, and **C# 14** adds implicit conversions to spans.

KEY FACTS

- Spans can't be fields of classes, captured, or used across `await`
- `stackalloc` is for small buffers (≈ < 1 KB)
- Always `Return` rented arrays to the pool
- Measure with BenchmarkDotNet before optimizing

SPAN, STACKALLOC, ARRAYPOOL

```
static int SumDigits(ReadOnlySpan<char> text)  // accepts string, char[], slices
{
    int sum = 0;
    foreach (char c in text) if (char.IsAsciiDigit(c)) sum += c - '0';
    return sum;
}
Console.WriteLine(SumDigits("a1b2c3"));             // 6 (string → span)
Console.WriteLine(SumDigits("x9y9z9".AsSpan(1, 3))); // 9 (slice, no copy)

Span<char> buf = stackalloc char[32];               // stack memory, no GC
Guid.NewGuid().TryFormat(buf, out int written, "N"); // no string allocated

byte[] rented = ArrayPool<byte>.Shared.Rent(16_384); // reuse big buffers
try { int n = await stream.ReadAsync(rented.AsMemory(0, 16_384)); }
finally { ArrayPool<byte>.Shared.Return(rented); }

Memory<byte> mem = rented.AsMemory();            // OK in async methods / fields
Span<byte> view = mem.Span[..10];                   // get a span when needed
```

CODE EXPLAINED

- → `ReadOnlySpan<char>` — one API for many sources
- → C# 14 — arrays convert to spans implicitly (e.g. span extension methods)
- → `AsSpan(start, len)` — zero-copy slice
- → `stackalloc` — temporary stack buffer
- → `ArrayPool<T>.Shared` — rent/return large arrays
- → `Memory<T>` — span-like, but storable and async-friendly

MEMORY TYPES

TYPE	LIVES	ASYNC OK
`Span<T>`	Stack only	No
`ReadOnlySpan<T>`	Stack only	No
`Memory<T>`	Heap / field	Yes
`ReadOnlyMemory<T>`	Heap / field	Yes
`T[]`	Heap	Yes

BENCHMARKDOTNET

```
[MemoryDiagnoser]
public class ParseBench
{
    [Benchmark] public int Sub() =>
        int.Parse(s.Substring(0, 4));
    [Benchmark] public int Span() =>
        int.Parse(s.AsSpan(0, 4));
}
// BenchmarkRunner.Run<ParseBench>();  (Release build)
```

⚠ COMMON MISTAKES

- Large `stackalloc` buffers — can overflow the stack; fall back to `ArrayPool` above a size limit
- Using a rented array after returning it — another caller may be writing to it
- Optimizing without measuring — spans add complexity; benchmark first

✓ TIPS & BEST PRACTICES

- Accept `ReadOnlySpan<T>` in parsing helpers
- Use `string.Create` / `TryFormat` to build strings without temporaries
- Add `[MemoryDiagnoser]` to see allocations per call

QUICK SUMMARY

- `Span<T>` — stack-only view
- `Memory<T>` — async-safe view
- `stackalloc` / `ArrayPool` — avoid GC
- Benchmark with BenchmarkDotNet

Sheet 117

Garbage Collection & Finalizers

generations 0/1/2 · Large Object Heap · finalizers ~T() · GC.SuppressFinalize · WeakReference · event-handler leaks

AUTOMATIC MEMORY MANAGEMENT

The **garbage collector** reclaims objects that are no longer reachable. It's **generational**: new objects start in **Gen 0**, survivors move to **Gen 1** and **Gen 2**, and objects ≥ 85 KB go to the **Large Object Heap**. Most apps never call the GC directly; your job is to avoid **leaks** (unexpected references) and **dispose** unmanaged resources.

KEY FACTS

- Gen 0 collections are frequent and cheap
- Finalizers delay collection and run on a separate thread
- Static references and event subscriptions keep objects alive
- Server GC is the default for ASP.NET Core

GC IN PRACTICE

```
var data = new byte[100];
Console.WriteLine(GC.GetGeneration(data));        // 0
GC.Collect();                                     // don't do this in real apps
Console.WriteLine(GC.GetGeneration(data));        // 1 (survived)

Console.WriteLine(GC.CollectionCount(0));         // # of Gen 0 GCs
public class NativeBuffer : IDisposable
{
    private IntPtr _ptr = Marshal.AllocHGlobal(1024);
    public void Dispose() { Free(); GC.SuppressFinalize(this); }
    ~NativeBuffer() => Free();                    // finalizer: last resort
    private void Free()
    {
        if (_ptr == IntPtr.Zero) return;
        Marshal.FreeHGlobal(_ptr); _ptr = IntPtr.Zero;
    }
}

var cache = new ConditionalWeakTable<object, Metadata>();   // no leak
var weak = new WeakReference<BigThing>(new BigThing());
if (weak.TryGetTarget(out var thing)) Use(thing);           // may be collected
```

CODE EXPLAINED

- → `GetGeneration` — which generation an object is in
- → `GC.Collect` — forces a collection (diagnostics only)
- → Finalizer `~T()` — safety net for unmanaged memory
- → `SuppressFinalize` — skip finalizer after Dispose
- → `ConditionalWeakTable` — attach data without keeping keys alive
- → `WeakReference<T>` — reference that allows collection

HEAP AREAS

AREA	HOLDS	COLLECTED
`Gen 0`	New objects	Very often
`Gen 1`	Short-lived survivors	Often
`Gen 2`	Long-lived objects	Rarely
`LOH`	Objects ≥ 85 KB	With Gen 2
`POH`	Pinned objects (.NET 5)	With Gen 2

COMMON LEAKS

CAUSE	FIX
`Event handlers never removed`	Unsubscribe / weak events
`Static collections growing`	Bound or evict entries
`Undisposed timers / CTS`	Dispose them
`Captured closures in caches`	Avoid capturing large objects

⚠ COMMON MISTAKES

- Calling `GC.Collect()` to "fix" memory — usually hurts performance and hides the real leak
- Writing finalizers for classes without unmanaged resources — adds overhead for nothing
- Allocating large temporary arrays repeatedly — fills the LOH; use `ArrayPool`

✓ TIPS & BEST PRACTICES

- Investigate memory with `dotnet-counters` and `dotnet-gcdump`
- Prefer `SafeHandle` over hand-written finalizers
- Reduce allocations in hot paths (spans, pooling, structs)

QUICK SUMMARY

- Generational GC — Gen 0/1/2 + LOH
- Leaks = unwanted references
- Finalizers — last resort; use `SafeHandle`
- Diagnose with dotnet-counters / gcdump

Reflection & Attributes

INSPECTING TYPES AT RUN TIME

Reflection (`System.Reflection`) reads type metadata at run time — properties, methods, attributes — and can create objects or invoke members dynamically. **Attributes** attach declarative metadata to code (`[Obsolete]`, `[Required]`, `[JsonIgnore]`) that frameworks read via reflection. Reflection is powerful but **slow** and limited under Native AOT.

KEY FACTS

- Start from `typeof(T)` or `obj.GetType()`
- Attribute classes end with `Attribute`; the suffix is optional in use
- `[AttributeUsage]` controls where an attribute can go
- Source generators often replace reflection today

REFLECTION & CUSTOM ATTRIBUTES

```
[AttributeUsage(AttributeTargets.Property)]
public sealed class ColumnAttribute(string name) : Attribute
{
    public string Name { get; } = name;
}
public class Customer
{
    [Column("cust_name")] public string Name { get; set; } = "";
    public int Age { get; set; }
}
Type t = typeof(Customer);
foreach (PropertyInfo p in t.GetProperties())
{
    var col = p.GetCustomAttribute<ColumnAttribute>();
    Console.WriteLine($"{p.Name} → {col?.Name ?? p.Name}");
}   // Name → cust_name   Age → Age

var c = (Customer)Activator.CreateInstance(t)!;        // create dynamically
t.GetProperty("Age")!.SetValue(c, 42);                 // set by name
MethodInfo m = t.GetMethod(nameof(ToString))!;
Console.WriteLine(m.Invoke(c, null));                   // invoke by reflection
```

CODE EXPLAINED

- → `: Attribute` — define a custom attribute
- → `[AttributeUsage]` — restrict targets
- → `[Column("…")]` — apply it
- → `GetProperties()` — list public properties
- → `GetCustomAttribute<T>()` — read the attribute
- → `Activator` / `SetValue` / `Invoke` — dynamic access

REFLECTION API

CALL	RETURNS
`t.GetProperties()`	PropertyInfo[]
`t.GetMethods()`	MethodInfo[]
`t.GetFields(BindingFlags…)`	FieldInfo[] (incl. private)
`t.GetInterfaces()`	Type[]

BUILT-IN ATTRIBUTES

ATTRIBUTE	EFFECT
`[Obsolete("msg")]`	Compiler warning on use
`[Flags]`	Enum bit flags
`[Conditional("DEBUG")]`	Calls removed in Release
`[CallerMemberName]`	Caller name injected
`[Required], [Range]`	Validation (DataAnnotations)

⚠ COMMON MISTAKES

- Reflection in hot loops — orders of magnitude slower; cache `PropertyInfo` or use delegates
- Reflecting over private members of other libraries — breaks on updates
- Relying on reflection in Native AOT/trimmed apps — members may be trimmed away

✓ TIPS & BEST PRACTICES

- Prefer source generators for serialization, mapping and DI
- Use `[DynamicallyAccessedMembers]` to keep members when trimming
- Use `nameof` with reflection lookups to stay rename-safe

QUICK SUMMARY

- `typeof(T)` / `GetType()` — entry points
- `GetProperties` / `GetMethods` — metadata
- Custom attributes — `: Attribute`
- Slow; limited under AOT

Union Types & Closed Hierarchies

union keyword (C# 15 preview) · case types · closed modifier · exhaustive switch · IUnion · sealed-record workaround for C# 14

UNIONS AND CLOSED HIERARCHIES (C# 15 PREVIEW)

C# 15 (preview, shipping with .NET 11 in November 2026) adds **union types** — `union Pet(Cat, Dog, Bird);` — whose values are exactly one of several **case types**, and the **`closed`** modifier, which fixes a class hierarchy to the types declared in the same assembly. In both cases the compiler knows every possibility, so `switch` expressions are **exhaustive without a default arm**.

KEY FACTS

- Needs the .NET 11 SDK and `LangVersion` preview
- Unions convert implicitly from each case type
- `closed` classes are implicitly `abstract`
- Syntax may still change before release

UNIONS & CLOSED (PREVIEW)

```
public record Cat(string Name); public record Dog(string Name);
public record Bird(string Name);
public union Pet(Cat, Dog, Bird);                  // union type

Pet pet = new Dog("Rex");                          // implicit conversion
string sound = pet switch
{
    Dog d  => $"{d.Name}: woof",
    Cat c  => $"{c.Name}: meow",
    Bird b => $"{b.Name}: tweet",                   // exhaustive, no _ arm
};

public closed record class GateState;              // closed hierarchy
public record class Closed : GateState;
public record class Open(float Percent) : GateState;

string Describe(GateState s) => s switch
{
    Closed => "closed",
    Open(var pct) => $"{pct}% open",            // no warning: all cases handled
};
```

CODE EXPLAINED

- → `union Pet(Cat, Dog, Bird)` — exactly one of three
- → Assign any case type directly
- → Switch covers every case — no default needed
- → `closed` — subclasses only in this assembly
- → Positional pattern on `Open` — extract fields

C# 14 WORKAROUND TODAY

```
public abstract record Shape
{
    private Shape() { }             // no outside subclasses
    public sealed record Circle(double R) : Shape;
    public sealed record Square(double S) : Shape;
}
// Switches over Shape still need:
//   _ => throw new UnreachableException()
```

FEATURE COMPARISON

FEATURE	UNION	CLOSED
`Cases`	Any existing types	Derived classes
`Shared base needed`	No	Yes
`Exhaustive switch`	Yes	Yes (direct children)
`Runtime support`	IUnion, UnionAttribute	—

⚠ COMMON MISTAKES

- Shipping preview language features in production code — syntax and behavior can change
- Assuming `closed` is transitive — mark intermediate classes `closed` too for deep exhaustiveness
- Combining `closed` with `sealed`, `static` or explicit `abstract` — not allowed

✓ TIPS & BEST PRACTICES

- Try previews in a side project with `<LangVersion>preview</LangVersion>`
- Use the private-constructor nested-record pattern on C# 14
- Libraries like OneOf offer union-like types today

QUICK SUMMARY

- `union U(A, B)` — one of several types
- `closed` — fixed hierarchy
- Exhaustive switches without `_`
- C# 15 preview — verify before relying on it

LINQ — Basics

LANGUAGE-INTEGRATED QUERY

LINQ queries any data source — arrays, lists, XML, databases — with the same operators. You can write **method syntax** (`xs.Where(…).Select(…)`) or **query syntax** (`from x in xs where … select …`); they compile to the same calls. Most operators use **deferred execution**: nothing runs until you enumerate the result.

KEY FACTS

- Operators are extension methods in `System.Linq`
- `ToList()`, `Count()`, `First()` execute immediately
- `IEnumerable<T>` runs in memory; `IQueryable<T>` is translated (e.g. to SQL)
- Re-enumerating a query re-runs it

TWO SYNTAXES, DEFERRED EXECUTION

```
int[] nums = [5, 3, 8, 1, 9, 2];

var big = nums.Where(n => n > 4).OrderBy(n => n).Select(n => n * 10);

var big2 = from n in nums                  // query syntax — same result
           where n > 4
           orderby n
           select n * 10;

Console.WriteLine(string.Join(",", big)); // 50,80,90

var list = new List<int> { 1, 2 };
var query = list.Where(x => x > 1);        // nothing runs yet
list.Add(3);
Console.WriteLine(query.Count());          // 2 — sees the new item

var snapshot = list.Where(x => x > 1).ToList();   // runs now, fixed copy
var slow = GetOrders().Where(o => o.Total > 100);
int c = slow.Count(); var f = slow.First();       // ✗ query runs twice
```

CODE EXPLAINED

- → Method syntax — chain extension methods with lambdas
- → Query syntax — SQL-like keywords, same IL
- → `string.Join` — enumerates the query
- → Deferred — the query sees later changes to `list`
- → `ToList()` — execute once and keep results
- → Multiple enumeration — cache with `ToList()`

EXECUTION TIMING

DEFERRED	IMMEDIATE
`Where, Select, SelectMany`	`ToList, ToArray, ToDictionary`
`OrderBy, ThenBy`	`Count, Sum, Average, Max`
`Skip, Take, Distinct`	`First, Single, Last`
`GroupBy, Join, Zip`	`Any, All, Contains`

QUERY KEYWORDS

KEYWORD	METHOD
`where`	Where
`select`	Select
`orderby … descending`	OrderByDescending
`group … by`	GroupBy
`join … on … equals`	Join
`let`	Select into a new shape

⚠ COMMON MISTAKES

- Enumerating an expensive query several times — call `ToList()` once
- Assuming a query captured the data when defined — it reads the source when enumerated
- Calling `ToList()` on an `IQueryable` too early — pulls the whole table into memory

✓ TIPS & BEST PRACTICES

- Prefer method syntax; use query syntax for joins and `let`
- Keep `IQueryable` until the last step so filters run in the database
- Use `Enumerable.Range`, `Repeat`, `Empty` to generate sequences

QUICK SUMMARY

- LINQ — one query model for all data
- Method vs query syntax — same result
- Deferred execution — runs on enumeration
- `ToList()` — materialize once

LINQ — Filtering & Projection

FILTERING AND SHAPING DATA

Where keeps elements that match a predicate. **Select** transforms each element into a new shape — a value, a tuple, a record or an anonymous type. **SelectMany** flattens nested collections. **OfType<T>** filters by type, and **Distinct** / **DistinctBy** remove duplicates.

KEY FACTS

- `Where` / `Select` overloads also give the index
- Anonymous types (`new { … }`) are great for projections
- `Cast<T>` throws on a wrong type; `OfType<T>` skips it
- `DistinctBy` (.NET 6) keeps the first per key

WHERE, SELECT, SELECTMANY

```
var orders = new[]
{
    new Order(1, "Ada", ["pen", "ink"], 30m),
    new Order(2, "Bob", ["pad"],         8m),
    new Order(3, "Ada", ["pen"],         12m),
};

var big = orders.Where(o => o.Total >= 10);                  // ids 1, 3
var names = orders.Select(o => o.Customer);                  // Ada, Bob, Ada
var shaped = orders.Select(o => new { o.Id, Items = o.Items.Length });
var withIdx = orders.Select((o, i) => $"{i}:{o.Id}");       // 0:1, 1:2, 2:3

var allItems = orders.SelectMany(o => o.Items);             // pen, ink, pad, pen
var unique = allItems.Distinct();                            // pen, ink, pad
var perCust = orders.DistinctBy(o => o.Customer);            // first Ada, Bob

object[] mixed = [1, "two", 3.0, "four"];
var strings = mixed.OfType<string>();                        // two, four
```

CODE EXPLAINED

- → `Where(pred)` — filter
- → `Select(f)` — map to a new value
- → `new { o.Id, … }` — anonymous projection
- → `Select((x, i) => …)` — with index
- → `SelectMany` — flatten nested lists
- → `OfType<T>` — keep only matching types

OPERATORS

OPERATOR	RESULT
`Where(pred)`	Matching elements
`Select(f)`	One output per input
`SelectMany(f)`	Flattened outputs
`Distinct() / DistinctBy(key)`	Unique elements
`OfType<T>() / Cast<T>()`	Type filter / conversion
`Where(…).Select(…)`	Filter then project

QUERY SYNTAX

```
var q = from o in orders
        where o.Total >= 10
        from item in o.Items      // SelectMany
        select new { o.Id, item };
```

⚠ COMMON MISTAKES

- Projecting before filtering on large data — filter first so less work follows
- Using `Cast<T>()` on mixed collections — throws `InvalidCastException`; use `OfType<T>()`
- Returning anonymous types from methods — they can't be named; use a record

✓ TIPS & BEST PRACTICES

- Project to small DTOs in database queries to fetch fewer columns
- Use tuples `(o.Id, o.Total)` for quick internal projections
- Combine `Where` conditions in one lambda when they're related

QUICK SUMMARY

- `Where` — filter
- `Select` — transform
- `SelectMany` — flatten
- `Distinct(By)` / `OfType<T>`

LINQ — Ordering & Paging

SORTING AND PAGING

`OrderBy` / `OrderByDescending` sort by a key and **`ThenBy`** adds tie-breakers; LINQ sorting is **stable**. `Order()` (.NET 7) sorts by natural order. **`Skip`** and **`Take`** implement paging, `TakeWhile`/`SkipWhile` stop on a condition, **`Chunk`** splits into batches, and **`Index()`** (.NET 9) pairs each item with its position.

KEY FACTS

- Sorting needs the whole sequence before yielding
- `Take(2..5)` accepts a range (.NET 6)
- C# 14: `array.Reverse()` binds to the in-place span version
- Page formula: `Skip((page - 1) * size).Take(size)`

ORDERING, PAGING, BATCHING

```
var people = new[] { ("Ada", 36), ("Bob", 25), ("Cy", 36), ("Di", 19) };

var sorted = people.OrderByDescending(p => p.Item2)   // age desc
                   .ThenBy(p => p.Item1);             // then name
// (Ada,36) (Cy,36) (Bob,25) (Di,19)

int[] nums = [4, 1, 3];
var asc = nums.Order();                               // 1, 3, 4

int page = 2, size = 2;
var paged = sorted.Skip((page - 1) * size).Take(size);  // (Bob,25) (Di,19)
var middle = Enumerable.Range(0, 10).Take(3..6);        // 3, 4, 5

var leading = new[] { 2, 4, 5, 6 }.TakeWhile(n => n % 2 == 0); // 2, 4

foreach (int[] batch in Enumerable.Range(1, 7).Chunk(3))
    Console.Write($"[{string.Join(",", batch)}] ");   // [1,2,3] [4,5,6] [7]

foreach (var (i, p) in sorted.Index())                // .NET 9
    Console.WriteLine($"{i + 1}. {p.Item1}");
```

CODE EXPLAINED

- → `OrderByDescending` + `ThenBy` — multi-key sort
- → `Order()` — natural ascending order
- → `Skip` + `Take` — pages
- → `Take(range)` — slice by indices
- → `TakeWhile` — stop at the first non-match
- → `Chunk(n)` — arrays of up to n items

OPERATORS

OPERATOR	RETURNS
`OrderBy / OrderByDescending`	Sorted by key
`ThenBy / ThenByDescending`	Secondary keys
`Order / OrderDescending`	Natural order
`Skip(n) / Take(n)`	Paging
`SkipLast(n) / TakeLast(n)`	From the end
`Chunk(n) / Index()`	Batches / (index, item)

WATCH OUT

CODE	PROBLEM
`xs.OrderBy(a).OrderBy(b)`	Second sort replaces the first
`Skip/Take without OrderBy`	Unstable pages in databases
`TakeWhile vs Where`	TakeWhile stops at first miss
`Deep Skip on big tables`	Slow — use keyset paging

⚠ COMMON MISTAKES

- Chaining `OrderBy(...).OrderBy(...)` — use `ThenBy` for the second key
- Paging without a deterministic order — items repeat or vanish between pages
- Using `Take` when you mean "while" — `TakeWhile` stops at the first failing element

✓ TIPS & BEST PRACTICES

- Add a unique key (e.g. Id) as the last `ThenBy` for stable paging
- Use `Chunk` to batch API calls or database inserts
- Use `Index()` instead of `Select((x, i) => …)` for readability

QUICK SUMMARY

- `OrderBy` / `ThenBy` — stable sort
- `Skip` / `Take` — paging
- `Chunk` — batching
- `Index()` — positions (.NET 9)

LINQ — Aggregation

SUMMARIZING SEQUENCES

Aggregation operators reduce a sequence to a single value: **`Count`**, **`Sum`**, **`Average`**, **`Min`**, **`Max`**. **`MinBy`**/**`MaxBy`** (.NET 6) return the **element** with the smallest/largest key. **`Aggregate`** folds with any function, and .NET 9 adds **`CountBy`** and **`AggregateBy`** for per-key totals without `GroupBy`.

KEY FACTS

- All aggregates execute immediately
- `Sum` of empty = 0; `Average` / `Min` / `Max` of empty throw
- Use nullable selectors (`x => (int?)x`) to get null instead
- `Count(pred)` counts matching items

AGGREGATES

```
var sales = new[]
{
    new Sale("North", "pen", 12m), new Sale("South", "pad", 30m),
    new Sale("North", "ink", 8m),  new Sale("East", "pen", 20m),
};

Console.WriteLine(sales.Count());                      // 4
Console.WriteLine(sales.Count(s => s.Amount > 10));    // 3
Console.WriteLine(sales.Sum(s => s.Amount));           // 70
Console.WriteLine(sales.Average(s => s.Amount));       // 17.5
Console.WriteLine(sales.Max(s => s.Amount));           // 30
Console.WriteLine(sales.MaxBy(s => s.Amount)!.Region); // South (the element)

foreach (var (region, n) in sales.CountBy(s => s.Region))      // .NET 9
    Console.Write($"{region}:{n} ");                   // North:2 South:1 East:1

var totals = sales.AggregateBy(s => s.Region, 0m, (sum, s) => sum + s.Amount);
// North:20 South:30 East:20

string csv = sales.Select(s => s.Item).Aggregate((a, b) => $"{a},{b}");
// pen,pad,ink,pen
```

CODE EXPLAINED

- → `Count(pred)` — matches only
- → `Sum` / `Average` / `Max` with a selector
- → `MaxBy` — returns the whole element
- → `CountBy` — key → count pairs
- → `AggregateBy` — key → folded value
- → `Aggregate` — general fold

AGGREGATE OPERATORS

OPERATOR	EMPTY SEQUENCE
`Count` / `LongCount`	0
`Sum`	0
`Average`	InvalidOperationException
`Min` / `Max`	Throws (non-nullable)
`MinBy` / `MaxBy`	default (null)
`Aggregate(seed, f)`	seed

COUNTBY VS GROUPBY

OLD	.NET 9
`GroupBy(k) + g.Count()`	`CountBy(k)`
`GroupBy(k) + g.Sum(f)`	`AggregateBy(k, 0, f)`

⚠ COMMON MISTAKES

- Calling `Average()` or `Max()` on a possibly empty sequence — throws; check `Any()` or use `DefaultIfEmpty()`
- Using `Count() > 0` to test for items — `Any()` stops at the first element
- Summing `double` money values — rounding errors; use `decimal`

✓ TIPS & BEST PRACTICES

- Use `MaxBy` instead of `OrderByDescending(...).First()`
- Use `CountBy` / `AggregateBy` for quick dashboards
- Use `TryGetNonEnumeratedCount` to get counts without enumerating

QUICK SUMMARY

- `Count` / `Sum` / `Average` / `Min` / `Max`
- `MinBy` / `MaxBy` — element with extreme key
- `CountBy` / `AggregateBy` — per-key (.NET 9)
- `Aggregate` — custom fold

LINQ — Grouping & Joining

GROUPING AND JOINING

GroupBy collects elements that share a key into `IGrouping<TKey, T>` groups; **ToLookup** does the same but executes immediately and allows key lookups. **Join** performs an inner join between two sequences, **GroupJoin** a one-to-many join, and **.NET 10** adds **LeftJoin** and **RightJoin**. **Zip** pairs elements by position.

KEY FACTS

- Each group has a `Key` and is itself enumerable
- `Join` drops unmatched elements
- `LeftJoin` keeps all left items; the right side may be `null`
- `ToDictionary` throws on duplicate keys

GROUPBY, JOIN, LEFTJOIN, ZIP

```
var byRegion = sales.GroupBy(s => s.Region);
foreach (var g in byRegion)
    Console.WriteLine($"{g.Key}: {g.Count()} sales, {g.Sum(s => s.Amount):C}");

var lookup = sales.ToLookup(s => s.Region);            // immediate
Console.WriteLine(lookup["North"].Count());            // 2

var rows = customers.Join(orders,                      // inner join
    c => c.Id, o => o.CustomerId,
    (c, o) => new { c.Name, o.Total });

var all = customers.LeftJoin(orders,                   // .NET 10
    c => c.Id, o => o.CustomerId,
    (c, o) => new { c.Name, Total = o?.Total ?? 0 }); // o may be null

var perCustomer = customers.GroupJoin(orders,          // one-to-many
    c => c.Id, o => o.CustomerId,
    (c, os) => new { c.Name, Count = os.Count() });

var pairs = new[] { "a", "b" }.Zip([1, 2]);            // (a,1) (b,2)
var byId = customers.ToDictionary(c => c.Id);
```

CODE EXPLAINED

- → `GroupBy` — groups with `.Key`
- → Aggregate inside each group
- → `ToLookup` — indexable multi-map
- → `Join(inner, outerKey, innerKey, result)`
- → `LeftJoin` — keep customers without orders
- → `Zip` — combine two sequences pairwise

OPERATORS

OPERATOR	SHAPE
`GroupBy(k)`	IEnumerable<IGrouping<K,T>>
`ToLookup(k)`	ILookup<K,T>
`Join`	Inner join
`GroupJoin`	Outer item + matching inners
`LeftJoin / RightJoin`	Outer joins (.NET 10)
`Zip(a, b)`	(T1, T2) tuples

QUERY SYNTAX JOIN

```
var q = from c in customers
        join o in orders on c.Id equals o.CustomerId
        group o by c.Name into g
        select new { Name = g.Key,
                     Total = g.Sum(o => o.Total) };
```

⚠ COMMON MISTAKES

- Using `ToDictionary` when keys repeat — throws `ArgumentException`; use `ToLookup`
- Emulating left joins with `GroupJoin` + `SelectMany` + `DefaultIfEmpty` on .NET 10 — use `LeftJoin`
- Joining large in-memory lists with nested loops — `Join` uses hashing and is much faster

✓ TIPS & BEST PRACTICES

- Use `CountBy` / `AggregateBy` when you only need per-key totals
- Use query syntax for multi-source joins — it reads better
- Use `Zip` with three sequences: `a.Zip(b, c)`

QUICK SUMMARY

- `GroupBy` / `ToLookup` — group by key
- `Join` — inner join
- `LeftJoin` / `RightJoin` — .NET 10
- `Zip` — pair by position

LINQ — Elements, Quantifiers & Sets

PICKING ELEMENTS AND COMPARING SETS

Element operators pick a single item: `First`, `Last`, `Single`, `ElementAt` and their `OrDefault` versions. **Quantifiers** answer yes/no questions: `Any`, `All`, `Contains`. **Set operators** combine sequences: `Union`, `Intersect`, `Except`, with `…By` versions (.NET 6) that compare by key.

KEY FACTS

- `First()` on empty throws; `FirstOrDefault()` returns default
- `Single()` throws if there are 0 or 2+ matches
- `FirstOrDefault(fallback)` (.NET 6) sets your own default
- Set operators remove duplicates

ELEMENTS, QUANTIFIERS, SETS

```
int[] n = [3, 8, 5, 8, 1];

Console.WriteLine(n.First());                    // 3
Console.WriteLine(n.First(x => x > 4));          // 8
Console.WriteLine(n.FirstOrDefault(x => x > 9)); // 0 (default)
Console.WriteLine(n.FirstOrDefault(x => x > 9, -1)); // -1 (custom)
Console.WriteLine(n.Last());                     // 1
Console.WriteLine(n.ElementAt(2));               // 5
Console.WriteLine(n.Single(x => x == 5));        // 5
// n.Single(x => x == 8);                        // ✗ more than one match

Console.WriteLine(n.Any(x => x > 7));            // True
Console.WriteLine(n.All(x => x > 0));            // True
Console.WriteLine(n.Contains(4));                // False

int[] a = [1, 2, 3], b = [2, 3, 4];
Console.WriteLine(string.Join(",", a.Union(b)));     // 1,2,3,4
Console.WriteLine(string.Join(",", a.Intersect(b))); // 2,3
Console.WriteLine(string.Join(",", a.Except(b)));    // 1
var newUsers = incoming.ExceptBy(existing.Select(u => u.Email), u => u.Email);
```

CODE EXPLAINED

- → `First` / `Last` — throw when empty
- → `OrDefault` — default or custom fallback
- → `Single` — exactly one or throw
- → `Any` / `All` / `Contains` — boolean tests
- → `Union` / `Intersect` / `Except` — set math
- → `ExceptBy` — compare by key

ELEMENT OPERATORS

OPERATOR	0 MATCHES	2+ MATCHES
`First`	Throws	First one
`FirstOrDefault`	default	First one
`Single`	Throws	Throws
`SingleOrDefault`	default	Throws
`Last / LastOrDefault`	Throws / default	Last one

SET OPERATORS

OPERATOR	KEEPS
`Union / UnionBy`	Items in either
`Intersect / IntersectBy`	Items in both
`Except / ExceptBy`	Items only in first
`Concat`	All items, with duplicates
`SequenceEqual`	Same items, same order?

⚠ COMMON MISTAKES

- Using `First()` when the sequence may be empty — throws; use `FirstOrDefault` and check
- Using `Single()` as a fast `First()` — it scans everything to prove uniqueness
- Using `Concat` when you wanted `Union` — duplicates remain

✓ TIPS & BEST PRACTICES

- Use `Single` when duplicates indicate a bug you want surfaced
- Use `Any()` instead of `Count() > 0`
- Use a `HashSet<T>` for repeated `Contains` checks

QUICK SUMMARY

- `First` / `Single` / `Last` (+ `OrDefault`)
- `Any` / `All` / `Contains`
- `Union` / `Intersect` / `Except` (+ `By`)
- `Concat` keeps duplicates

EF Core — DbContext & Setup

GETTING STARTED WITH EF CORE

Entity Framework Core is .NET's object-relational mapper: you work with C# classes and LINQ, and EF Core translates to SQL for SQL Server, PostgreSQL, SQLite, MySQL and more. A **`DbContext`** represents a session with the database and exposes a **`DbSet<T>`** per table. Register it with DI and configure the provider and connection string.

KEY FACTS

- EF Core 10 ships with .NET 10 (LTS)
- One provider package per database
- `DbContext` is **not** thread-safe; register it **scoped**
- `dotnet-ef` CLI tool manages migrations

DBCONTEXT & REGISTRATION

```
// packages: Microsoft.EntityFrameworkCore.Sqlite (+ .Design); tool: dotnet-ef

public class Blog
{
    public int Id { get; set; }                    // primary key by convention
    public required string Title { get; set; }
    public List<Post> Posts { get; } = [];              // navigation
}
public class Post
{ public int Id { get; set; } public int BlogId { get; set; } }

public class AppDb(DbContextOptions<AppDb> options) : DbContext(options)
{
    public DbSet<Blog> Blogs => Set<Blog>();
    public DbSet<Post> Posts => Set<Post>();
}

// appsettings.json: "ConnectionStrings": { "Default": "Data Source=app.db" }
builder.Services.AddDbContext<AppDb>(o =>
    o.UseSqlite(builder.Configuration.GetConnectionString("Default")));
```

CODE EXPLAINED

- → Provider package — Sqlite, SqlServer, Npgsql…
- → `Design` package + `dotnet-ef` — migrations tooling
- → `Id` — key by convention
- → Navigation property — relationship
- → `DbSet<T>` — queryable table
- → `AddDbContext` — scoped registration with provider

PROVIDERS

DATABASE	PACKAGE / METHOD
`SQL Server / Azure SQL`	…SqlServer · UseSqlServer
`PostgreSQL`	Npgsql.EntityFrameworkCore.PostgreSQL · UseNpgsql
`SQLite`	…Sqlite · UseSqlite
`MySQL / MariaDB`	Pomelo… · UseMySql
`Cosmos DB`	…Cosmos · UseCosmos
`In-memory (tests)`	…InMemory · UseInMemoryDatabase

USEFUL OPTIONS

OPTION	USE
`EnableSensitiveDataLogging()`	Show parameter values (dev only)
`LogTo(Console.WriteLine)`	See generated SQL
`UseQueryTrackingBehavior(NoTracking)`	Read-heavy apps
`AddDbContextPool<T>`	Reuse context instances

⚠ COMMON MISTAKES

- Registering `DbContext` as a singleton — not thread-safe and grows its change tracker forever
- Hard-coding connection strings — keep them in configuration / secrets
- Using the in-memory provider to test SQL behavior — it isn't relational; use SQLite or Testcontainers

✓ TIPS & BEST PRACTICES

- Use `IDbContextFactory<T>` in background services and Blazor
- Log SQL in development to learn what EF generates
- Keep entities simple; put rules in services or the domain

QUICK SUMMARY

- `DbContext` — database session
- `DbSet<T>` — table
- `AddDbContext` + provider — setup
- `dotnet-ef` — tooling

EF Core — Modeling & Migrations

conventions · data annotations · Fluent API · relationships · migrations add · database update

MAPPING THE MODEL AND EVOLVING THE SCHEMA

EF Core builds a **model** from **conventions** (names, keys, navigations), then applies **data annotations** and the **Fluent API** in `OnModelCreating` for anything else. **Migrations** turn model changes into versioned schema scripts: add a migration after each change, review it, then apply it to the database.

KEY FACTS

- Fluent API overrides annotations, which override conventions
- Set decimal precision explicitly
- Migrations live in your project as C# files
- Review generated migrations before applying

ANNOTATIONS, FLUENT API, MIGRATIONS

```
public class Product
{
    public int Id { get; set; }
    [MaxLength(100)] public required string Name { get; set; }
    public decimal Price { get; set; }
    public int CategoryId { get; set; }                 // FK by convention
    public Category Category { get; set; } = null!;
}
protected override void OnModelCreating(ModelBuilder mb)   // in AppDb
{
    mb.Entity<Product>(e =>
    {
        e.Property(p => p.Price).HasPrecision(18, 2);
        e.HasIndex(p => p.Name).IsUnique();
        e.HasOne(p => p.Category).WithMany(c => c.Products)
         .HasForeignKey(p => p.CategoryId)
         .OnDelete(DeleteBehavior.Restrict);
    });
}
```

CODE EXPLAINED

- → `[MaxLength]` — annotation → `nvarchar(100)`
- → `CategoryId` + navigation — relationship by convention
- → `OnModelCreating` — Fluent API configuration
- → `HasPrecision(18, 2)` — exact money columns
- → `HasIndex().IsUnique()` — unique index
- → `HasOne/WithMany` — explicit one-to-many + delete rule

MIGRATION COMMANDS

COMMAND	DOES
`dotnet ef migrations add Init`	Create a migration
`dotnet ef database update`	Apply pending migrations
`dotnet ef migrations remove`	Undo last (unapplied)
`dotnet ef migrations script --idempotent`	SQL script for prod
`dotnet ef migrations bundle`	Self-contained migrator exe

RELATIONSHIPS

KIND	FLUENT API
`One-to-many`	HasOne(…).WithMany(…)
`One-to-one`	HasOne(…).WithOne(…)
`Many-to-many`	HasMany(…).WithMany(…)
`Owned type`	OwnsOne(…)

⚠ COMMON MISTAKES

- Editing a migration that's already applied in other environments — add a new migration instead
- Leaving decimal precision unset — EF warns and SQL Server defaults may truncate values
- Running `database update` from developer machines against production — use scripts or bundles in CI/CD

✓ TIPS & BEST PRACTICES

- Put large configurations in `IEntityTypeConfiguration<T>` classes
- Name migrations after the change: `AddProductIndex`
- Use `--idempotent` scripts for safe deployments

QUICK SUMMARY

- Conventions → annotations → Fluent API
- `OnModelCreating` — model configuration
- `migrations add` / `database update`
- Scripts or bundles for production

EF Core — Querying & Saving

LINQ to Entities · Include / ThenInclude · AsNoTracking · Add / Update / Remove · SaveChangesAsync · ExecuteUpdate / ExecuteDelete

READING AND WRITING DATA

Queries on `DbSet<T>` are **`IQueryable`**: EF Core translates the LINQ into SQL when you call an executing method like `ToListAsync`. Loaded entities are **tracked**; changes are saved with **`SaveChangesAsync`**, which runs inside a transaction. Use **`AsNoTracking`** for read-only queries, **`Include`** for related data, and **`ExecuteUpdateAsync/ExecuteDeleteAsync`** for set-based changes.

KEY FACTS

- Use the `…Async` methods in web apps
- Projections (`Select`) fetch only needed columns
- `FromSql($"…")` parameterizes interpolated values
- Each `SaveChanges` is one transaction

QUERY & SAVE

```
var recent = await db.Blogs
    .AsNoTracking()                                  // read-only, faster
    .Where(b => b.Posts.Any(p => p.Published > since))
    .OrderBy(b => b.Title)
    .Select(b => new { b.Id, b.Title, Count = b.Posts.Count })
    .ToListAsync(ct);                                // SQL runs here

var blog = await db.Blogs
    .Include(b => b.Posts).ThenInclude(p => p.Tags)    // eager load
    .FirstOrDefaultAsync(b => b.Id == id, ct);

db.Blogs.Add(new Blog { Title = "New" });              // insert
blog!.Title = "Renamed";                               // tracked update
db.Posts.Remove(blog.Posts[0]);                        // delete
await db.SaveChangesAsync(ct);                         // one transaction

await db.Posts.Where(p => p.Published < cutoff)
    .ExecuteDeleteAsync(ct);                           // bulk delete, no load
await db.Posts.Where(p => p.BlogId == id)
    .ExecuteUpdateAsync(s => s.SetProperty(p => p.Views, 0), ct);
var rows = await db.Blogs
    .FromSql($"SELECT * FROM Blogs WHERE Id = {id}").ToListAsync(ct);
```

CODE EXPLAINED

- → `AsNoTracking` — skip change tracking
- → `Select` projection — only needed columns
- → `ToListAsync` — executes the SQL
- → `Include` / `ThenInclude` — load related rows
- → Add / modify / `Remove` + `SaveChangesAsync`
- → `ExecuteDelete/UpdateAsync` — set-based SQL

ENTITY STATES

STATE	ON SAVECHANGES
Added	INSERT
Modified	UPDATE changed columns
Deleted	DELETE
Unchanged	Nothing
Detached	Not tracked

PERFORMANCE CHECKLIST

PROBLEM	FIX
N+1 queries in loops	Include or projection
Huge Include graphs	AsSplitQuery()
Loading to update many rows	ExecuteUpdateAsync
Tracking read-only data	AsNoTracking()
ToList() before Where	Filter in the query

⚠ COMMON MISTAKES

- Building SQL with string concatenation — injection risk; use `FromSql($"…")` interpolation
- Calling `ToList()` early, then filtering in memory — pulls entire tables
- Lazy-loading navigation properties in loops — N+1 queries

✓ TIPS & BEST PRACTICES

- Return DTOs from APIs, not tracked entities
- Use `TagWith("…")` to label queries in SQL logs
- Wrap multi-step work in `db.Database.BeginTransactionAsync()` when needed

QUICK SUMMARY

- LINQ → SQL via `IQueryable`
- `Include` / projections — related data
- `SaveChangesAsync` — tracked changes
- `ExecuteUpdate/DeleteAsync` — bulk ops

Sheet 129

Quick Reference — Language & Types

dotnet CLI · types & nullability · strings · operators · control flow · pattern matching · exceptions · methods & delegates · OOP & records

DOTNET CLI

COMMAND	DOES	#
`dotnet new console -n App`	new project	003
`dotnet run / dotnet watch`	run / hot reload	003
`dotnet build / test`	build / test	003
`dotnet add package X`	NuGet package	099
`dotnet run app.cs`	file-based app	006

TYPES & NULLS

SYNTAX	DOES	#
`int / long / double / decimal`	numbers	012
`var x = 5;`	inferred type	011
`int? n`	nullable value type	016
`string? s / #nullable enable`	nullable reference	017
`(int)x / x as T / int.Parse`	cast / safe cast / parse	018
`int.TryParse(s, out var n)`	safe parse	018

STRINGS

SYNTAX	DOES	#
`$"{name} {x:N2}"`	interpolation + format	022
`"""raw text"""`	raw string literal	021
`s.Contains / IndexOf`	search	023
`s.Split(',') / string.Join`	split / join	024
`s[1..4] / s[^1]`	range / from end	025
`StringBuilder`	build in loops	026

OPERATORS

OPERATOR	DOES	#
`a ?? b`	null-coalescing	032
`a ??= b`	assign if null	032
`obj?.Prop / arr?[0]`	null-conditional	032
`x!`	null-forgiving	017
`is / is not null`	type / null test	019
`== vs Equals`	operator vs method	028

CONTROL FLOW

SYNTAX	DOES	#
`if / else if / else`	branch	034
`c ? a : b`	conditional	035
`switch (x) { case 1: break; }`	statement	036
`x switch { 1 => "a", _ => "b" }`	expression	037
`for / foreach (var x in xs)`	loops	041
`break / continue / goto case`	jumps	043

PATTERN MATCHING

PATTERN	DOES	#
`o is string s`	type pattern	038
`{ Age: > 18 }`	property pattern	038
`> 0 and < 10`	relational + logical	038
`[1, .., var last]`	list pattern	039
`case X when cond`	guard	039

EXCEPTIONS

SYNTAX	DOES	#
`try { } catch (IOException ex)`	catch	044
`catch (Exception ex) when (…)`	filter	046
`finally { }`	always	044
`throw;`	rethrow, keep stack	045
`using var f = File.OpenRead(p);`	dispose	048

METHODS & DELEGATES

SYNTAX	DOES	#
`int Add(int a, int b = 1)`	optional param	050
`ref / out / in`	by-reference	050
`Func<int, int> f = x => x * 2`	lambda	055
`Action / Predicate<T>`	delegate types	054
`event EventHandler Changed`	event	057
`static int Ext(this string s)`	extension method	060

OOP & RECORDS

SYNTAX	DOES	#
`class Dog : Animal, IPet`	inherit + implement	081
`public string Name { get; init; }`	init-only property	078
`required string Name`	must set	078
`virtual / override / sealed`	polymorphism	082
`record Point(int X, int Y)`	value-equal type	086
`p with { X = 1 }`	non-destructive copy	086

⚠ COMMON MISTAKES

- `catch (Exception ex) { throw ex; }` resets the stack trace — use `throw;`
- Ignoring nullable warnings instead of fixing them
- `async void` methods except event handlers

✓ TIPS & BEST PRACTICES

- Turn on `<Nullable>enable</Nullable>` and `<TreatWarningsAsErrors>`
- Prefer switch expressions and patterns over long if chains
- Use records for DTOs and value objects

QUICK SUMMARY

- `dotnet new / run / test / add package`
- Nullable types + `?.` + `??` keep nulls explicit
- Switch expressions and patterns
- Classes for behaviour, records for data

Sheet 130

Quick Reference — Collections, LINQ, Async & Libraries

arrays & collection expressions · List & Dictionary · generics · LINQ · files & JSON · DI & configuration · testing · async/await · EF Core

ARRAYS & LISTS

SYNTAX	DOES	#
`int[] xs = [1, 2, 3];`	collection expression	063
`new List<string>()`	growable list	064
`list.Add / Remove / Insert`	mutate	065
`list.Contains / Find`	search	065
`Array.Sort(xs) / list.Sort()`	sort	074

DICTIONARIES & SETS

SYNTAX	DOES	#
`new Dictionary<string, int>()`	map	066
`d.TryGetValue(k, out var v)`	safe lookup	066
`d[k] = v`	add or replace	066
`new HashSet<int>() / UnionWith`	set	068
`Queue / Stack / PriorityQueue`	other collections	069
`FrozenDictionary`	read-optimized	071

GENERICS & TUPLES

SYNTAX	DOES	#
`class Box<T>`	generic type	088
`where T : class, new()`	constraints	089
`(int, string) t = (1, "a")`	tuple	070
`var (a, b) = t;`	deconstruct	070

LINQ

METHOD	DOES	#
`Where(x => …)`	filter	121
`Select(x => …)`	project	121
`OrderBy / ThenByDescending`	sort	122
`Skip(20).Take(10)`	page	122
`Sum / Average / Count`	aggregate	123
`GroupBy / Join`	group / join	124
`First / FirstOrDefault / Any`	elements	125

FILES & JSON

API	DOES	#
`File.ReadAllText(p)`	read	091
`File.WriteAllText / AppendAllText`	write	092
`Path.Combine(a, b)`	build path	093
`Directory.EnumerateFiles`	list files	093
`JsonSerializer.Serialize(obj)`	to JSON	094
`JsonSerializer.Deserialize<T>(s)`	from JSON	094

HOSTING & LIBRARIES

API	DOES	#
`Host.CreateApplicationBuilder()`	generic host	104
`services.AddScoped<IRepo, Repo>()`	DI registration	104
`builder.Configuration["Key"]`	config	103
`ILogger<T>`	logging	108
`IHttpClientFactory`	HttpClient	109

TESTING

ITEM	DOES	#
`[Fact] / [Theory]`	xUnit tests	105
`[InlineData(1, 2)]`	test data	105
`Assert.Equal(exp, act)`	assertion	105
`Substitute.For<IRepo>()`	NSubstitute mock	107

ASYNC

SYNTAX	DOES	#
`async Task<int> GetAsync()`	async method	111
`await Task.WhenAll(a, b)`	parallel awaits	112
`CancellationToken ct`	cancellation	112
`await foreach (var x in s)`	async stream	112
`Parallel.ForEachAsync`	parallel work	115

EF CORE

API	DOES	#
`class AppDb : DbContext`	context	126
`DbSet<Order> Orders`	table	126
`dotnet ef migrations add Init`	migration	127
`db.Orders.Where(…).ToListAsync()`	query	128
`Include(o => o.Items)`	eager load	128
`SaveChangesAsync()`	persist	128

⚠ COMMON MISTAKES

- `.Result` / `.Wait()` on tasks — deadlocks and thread starvation; `await` instead
- Enumerating a LINQ query twice — call `ToList()` once
- Creating `new HttpClient()` per request — use `IHttpClientFactory`

✓ TIPS & BEST PRACTICES

- Pass `CancellationToken` through every async call chain
- Use `AsNoTracking()` for read-only EF Core queries
- Register services by interface and inject via constructors

QUICK SUMMARY

- `List<T>`, `Dictionary<K,V>`, `HashSet<T>`, collection expressions
- LINQ: `Where` → `Select` → `ToList`
- `async` / `await` with `Task` and cancellation
- Generic host + DI + config + logging